LAPORAN TUGAS BESAR 1

IF3070 Dasar Inteligensi Artifisial

Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo Menggunakan *Local Search*



Disusun oleh:

Kelompok G19 - Mantaplah Pokoknya

18224068	Tiffany Virya Chu
18224078	Francis Galton
18224092	Zahrah Nur Azizah

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN 6
1.1 Latar Belakang 6
1.2 Entitas dan Atribut 7
1.3 Ketentuan Representasi 8
1.4 Objective Function dan Constraint 8
1.5 Interpretasi dan Asumsi 9
1.6 Dataset 10
1.7 Local Search 11
BAB II PEMBAHASAN 12
2.1 Struktur Program 12
2.2 Representasi State 13
2.3 Perhitungan Objective Function 14
2.4 Initial State 16
2.5 Neighbor (Move) 17
2.6 Implementasi Algoritma 20
2.6.1 Hill-Climbing 20
a. Deskripsi 20
b. Source Code 21
c. Penjelasan 25
Penjelasan Implementasi Hill-Climbing 25
2.6.2 Simulated Annealing 27
a. Deskripsi 27
b. Source Code 27
c. Penjelasan 31
2.6.3 Genetic Algorithm 33
a. Deskripsi 33
b. Source Code 33
c. Penjelasan 43
2.6.4 Visualisasi 46
2.6.5 Heuristik yang Digunakan 51
2.7 Hasil Eksperimen 52
2.7.1 Setup Eksperimen 52
2.7.2 Hill-Climbing 53
2.7.3 Simulated Annealing 68
2.7.4 Genetic Algorithm 75
2.8 Bonus: Objective Function Alternatif Berbasis ETA 82
2.8.1 Deskripsi 82
2.8.2 Source Code 82

2.8.3 Penjelasan....83
2.8.4 Hasil Eksperimen....84
2.8.5 Analisis....87
BAB III ANALISIS....89
3.1 Kedekatan dengan Global Optimum....89
3.2 Perbandingan Hasil Antaralgoritma....90
3.3 Perbandingan Durasi....90
3.4 Konsistensi Hasil....91
3.5 Pengaruh Banyak Iterasi dan Jumlah Populasi pada Genetic Algorithm...91
3.6 Analisis Tambahan....91
BAB IV KESIMPULAN DAN SARAN....93
4.1 Kesimpulan....93
4.2 Saran....93
BAB V PEMBAGIAN TUGAS....95
REFERENSI....96
LAMPIRAN....97
Lampiran A. Form Penggunaan Generative AI....97
Lampiran B. Lampiran Lainnya....98

DAFTAR TABEL

Tabel 1.1 Entitas dan Atribut Permasalahan..... 8
Tabel 1.2 Ringkasan Dataset..... 11
Tabel 2.1 Initial State Setiap Percobaan.....53
Tabel 2.2 Hasil Eksperimen Steepest-Ascent Hill-Climbing.....54
Tabel 2.3 Hasil Eksperimen Stochastic Hill-Climbing..... 54
Tabel 2.4 Hasil Eksperimen Sideways Move.....54
Tabel 2.5 Hasil Eksperimen Random-Restart Hill-Climbing.....56
Tabel 2.6 Rincian Iterasi Random-Restart Hill-Climbing.....56
Tabel 2.7 Hasil Eksperimen Simulated Annealing.....76
Tabel 2.8 Rincian Langkah Simulated Annealing.....76
Tabel 2.9 Hasil Eksperimen Genetic Algorithm..... 83
Tabel 2.10 Rincian Langkah Genetic Algorithm..... 83
Tabel 2.11 Hasil Simulated Annealing dengan Objective ETA..... 92
Tabel 2.12 Perbandingan Objective Fee dan Objective ETA.....92
Tabel 3.1 Perbandingan Hasil Terhadap Upper Bound (1673)..... 97
Tabel 5.1 Pembagian Tugas Anggota Kelompok.....106

DAFTAR GAMBAR

Gambar 2.1 State Awal Steepest-Ascent Percobaan 1..... 57
Gambar 2.2 State Akhir Steepest-Ascent Percobaan 1..... 57
Gambar 2.3 Plot Objective Function Steepest-Ascent Percobaan 1..... 58
Gambar 2.4 State Awal Steepest-Ascent Percobaan 2..... 58
Gambar 2.5 State Akhir Steepest-Ascent Percobaan 2..... 59
Gambar 2.6 Plot Objective Function Steepest-Ascent Percobaan 2..... 59
Gambar 2.7 State Awal Steepest-Ascent Percobaan 3..... 60
Gambar 2.8 State Akhir Steepest-Ascent Percobaan 3..... 60
Gambar 2.9 Plot Objective Function Steepest-Ascent Percobaan 3..... 61
Gambar 2.10 State Awal Stochastic Percobaan 1..... 61
Gambar 2.11 State Akhir Stochastic Percobaan 1.....62
Gambar 2.12 Plot Objective Function Stochastic Percobaan 1.....62
Gambar 2.13 State Awal Stochastic Percobaan 2..... 63
Gambar 2.14 State Akhir Stochastic Percobaan 2..... 63
Gambar 2.15 Plot Objective Function Stochastic Percobaan 2.....64
Gambar 2.16 State Awal Stochastic Percobaan 3..... 64
Gambar 2.17 State Akhir Stochastic Percobaan 3..... 65
Gambar 2.18 Plot Objective Function Stochastic Percobaan 3.....65
Gambar 2.19 State Awal Sideways Percobaan 1.....66
Gambar 2.20 State Akhir Sideways Percobaan 1.....66
Gambar 2.21 Plot Objective Function Sideways Percobaan 1.....67
Gambar 2.22 State Awal Sideways Percobaan 2.....67
Gambar 2.23 State Akhir Sideways Percobaan 2.....68
Gambar 2.24 Plot Objective Function Sideways Percobaan 2.....68
Gambar 2.25 State Awal Sideways Percobaan 3.....69
Gambar 2.26 State Akhir Sideways Percobaan 3.....69
Gambar 2.27 Plot Objective Function Sideways Percobaan 3.....70
Gambar 2.28 State Awal Random Restart Percobaan 1..... 70
Gambar 2.29 State Akhir Random Restart Percobaan 1..... 71
Gambar 2.30 Plot Objective Function Random Restart Percobaan 1..... 71
Gambar 2.31 State Awal Random Restart Percobaan 2..... 72
Gambar 2.32 State Akhir Random Restart Percobaan 2..... 72
Gambar 2.33 Plot Objective Function Random Restart Percobaan 2..... 73
Gambar 2.34 State Awal Random Restart Percobaan 3..... 73
Gambar 2.35 State Akhir Random Restart Percobaan 3..... 74
Gambar 2.36 Plot Objective Function Random Restart Percobaan 3..... 74
Gambar 2.37 State Awal Simulated Annealing Percobaan 1..... 76

Gambar 2.38 State Akhir Simulated Annealing Percobaan 1 77
Gambar 2.39 Plot Objective Function Simulated Annealing Percobaan 1 77
Gambar 2.40 Plot e^(ΔE/T) Simulated Annealing Percobaan 1 77
Gambar 2.41 State Awal Simulated Annealing Percobaan 2 78
Gambar 2.42 State Akhir Simulated Annealing Percobaan 2 78
Gambar 2.43 Plot Objective Function Simulated Annealing Percobaan 2 79
Gambar 2.44 Plot e^(ΔE/T) Simulated Annealing Percobaan 2 79
Gambar 2.45 State Awal Simulated Annealing Percobaan 3 80
Gambar 2.46 State Akhir Simulated Annealing Percobaan 3 80
Gambar 2.47 Plot Objective Function Simulated Annealing Percobaan 3 81
Gambar 2.48 Plot e^(ΔE/T) Simulated Annealing Percobaan 3 81
Gambar 2.49 State Awal Genetic Algorithm Percobaan 1 84
Gambar 2.50 State Akhir Genetic Algorithm Percobaan 1 84
Gambar 2.51 Plot Objective Function Genetic Algorithm Percobaan 1 85
Gambar 2.52 State Awal Genetic Algorithm Percobaan 2 85
Gambar 2.53 State Akhir Genetic Algorithm Percobaan 2 86
Gambar 2.54 Plot Objective Function Genetic Algorithm Percobaan 2 86
Gambar 2.55 State Awal Genetic Algorithm Percobaan 3 87
Gambar 2.56 State Akhir Genetic Algorithm Percobaan 3 87
Gambar 2.57 Plot Objective Function Genetic Algorithm Percobaan 3 88
Gambar 2.58 State Akhir Objective ETA Percobaan 1 93
Gambar 2.59 Plot Objective Function Objective ETA Percobaan 1 93
Gambar 2.60 State Akhir Objective ETA Percobaan 2 94
Gambar 2.61 Plot Objective Function Objective ETA Percobaan 2 94
Gambar 2.62 State Akhir Objective ETA Percobaan 3 95
Gambar 2.63 Plot Objective Function Objective ETA Percobaan 3 95

BAB I
DESKRIPSI PERSOALAN

1.1 Latar Belakang

PT Intim Line merupakan perusahaan yang mengoperasikan kapal kargo pengangkut kendaraan. Pada setiap jadwal keberangkatan kapal, perusahaan menerima kendaraan dengan dimensi, berat, dan tarif pengiriman yang berbeda. Sementara itu, luas geladak dan kapasitas angkut kapal terbatas sehingga tidak seluruh kendaraan selalu dapat diangkut. Penentuan kendaraan yang dipilih juga berkaitan dengan penataannya, karena posisi dan orientasi suatu kendaraan mempengaruhi ruang yang tersedia bagi kendaraan lain. Oleh karena itu, pemilihan kendaraan dan pengaturan susunannya perlu dipertimbangkan secara bersamaan.

Dalam studi kasus ini, penempatan kendaraan masih dilakukan secara manual sehingga berbagai kemungkinan konfigurasi belum dapat dievaluasi secara menyeluruh. Persoalan tersebut kemudian dimodelkan sebagai penataan kendaraan pada grid dua dimensi dengan tujuan memaksimalkan total tarif pengiriman. Setiap solusi harus memastikan bahwa kendaraan tidak saling bertumpuk, seluruh bagian kendaraan yang diangkut berada di dalam geladak, dan total beratnya tidak melebihi kapasitas kapal.

Persoalan ini termasuk persoalan optimasi kombinatorial. Dari sisi pemilihan kendaraan, persoalannya mirip *0/1 knapsack* karena ada batas berat. Dari sisi penempatan, persoalannya mirip *2D bin packing* karena setiap kendaraan menempati sejumlah sel grid dan tidak boleh bertumpuk. Gabungan keduanya membuat ruang pencarian sangat besar, karena setiap kendaraan bisa berada di luar kapal atau di salah satu dari ratusan posisi dengan dua kemungkinan orientasi. Mencoba semua kombinasi satu per satu jelas tidak mungkin, sehingga digunakan pendekatan *local search*.

Tugas besar ini menerapkan *Hill-Climbing*, *Simulated Annealing*, dan *Genetic Algorithm* untuk mencari konfigurasi penempatan kendaraan. Ketiga algoritma dibandingkan berdasarkan nilai solusi yang diperoleh, perkembangan nilai selama pencarian, durasi proses, dan konsistensi hasil pada beberapa

percobaan. Melalui perbandingan tersebut, karakteristik masing-masing algoritma dapat diamati dalam menyelesaikan persoalan yang sama.

1.2 Entitas dan Atribut

Persoalan ini melibatkan dua entitas utama, yaitu kendaraan dan kapal. Dari deskripsi yang diberikan, setiap kendaraan memiliki identitas, dimensi, tarif pengiriman, berat, dan perkiraan waktu kedatangan, sedangkan setiap kapal memiliki dimensi geladak serta kapasitas berat maksimum. Atribut masing-masing entitas ditunjukkan pada tabel berikut.

Tabel 1.1 Entitas dan Atribut Permasalahan

Entitas	Atribut	Contoh	Keterangan
Kendaraan	id	V001	Identitas unik kendaraan
Kendaraan	w, l	1, 2	Lebar dan panjang kendaraan dalam satuan grid
Kendaraan	orientation	H atau V	Orientasi penempatan kendaraan, data ini dapat berubah selama pencarian
Kendaraan	diangkut	0 atau 1	Menunjukkan apakah kendaraan dimuat ke dalam kapal
Kendaraan	koordinat	(0, 0)	Posisi pojok kiri atas kendaraan pada grid geladak. Kendaraan yang tidak diangkut tidak memiliki koordinat penempatan
Kendaraan	shipping_fee	13	Tarif pengiriman kendaraan dalam juta rupiah
Kendaraan	weight	11	Berat kendaraan dalam satuan berat yang digunakan dataset
Kendaraan	eta	1	Perkiraan waktu kedatangan dalam sisa hari, mengikuti definisi spesifikasi
Kapal	w, l	10, 20	Lebar dan panjang geladak dalam satuan grid
Kapal	max_capacity	500	Batas total berat kendaraan yang dapat diangkut kapal

Di antara semua atribut tersebut, terdapat tiga atribut yang dinamis, yaitu status diangkut, *orientation*, dan koordinat penempatan. Representasi *state* yang memuat informasi tersebut dijelaskan lebih dalam pada Bab II.

1.3 Ketentuan Representasi

Spesifikasi memberikan beberapa ketentuan representasi yang harus diikuti, yaitu:

- Kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang.
- Tidak semua kendaraan harus dimuat. Kendaraan yang tidak dimuat berada sepenuhnya di luar kapal.
- Koordinat berupa bilangan bulat, sedangkan cara representasi koordinat dan struktur data dibebaskan.
- *Initial state* dibangkitkan secara acak.
- Langkah *neighbor* yang boleh dilakukan setiap iterasi ada tiga, yaitu menukar dua kendaraan (*swap*), memindahkan sebuah kendaraan ke koordinat lain (*move*), dan mengubah orientasi kendaraan (*rotate*).
- *Final state* harus bisa menunjukkan keadaan semua kendaraan (di dalam maupun di luar), keadaan geladak, dan nilai dari *state* tersebut.

1.4 Objective Function dan Constraint

Tujuan dari persoalan ini adalah memaksimalkan total tarif pengiriman kendaraan yang diangkut dalam satu keberangkatan kapal. Kendaraan yang tidak diangkut tidak memberikan kontribusi terhadap nilai tersebut. Dengan z_i sebagai indikator kendaraan yang diangkut dan v_i sebagai tarif pengirimannya, fungsi objektif dirumuskan sebagai berikut.

$$\textbf{max } R(s) = \Sigma\, z_i\, v_i, \textbf{ untuk } i = 1, 2, ..., n$$

dengan:

- R(s): total tarif pengiriman pada *state* s.
- n: jumlah seluruh kendaraan.
- z_i: bernilai 1 jika kendaraan ke-i diangkut dan 0 jika tidak diangkut.
- v_i: tarif pengiriman kendaraan ke-i.

Untuk mendukung objektif tersebut, ada beberapa batasan yang harus dipenuhi oleh solusi akhir:

1. **Tidak terjadi tumpang tindih.** $C_s(x, y) \leq 1$ untuk seluruh sel geladak, dengan $C_s(x, y)$ adalah banyak kendaraan yang menempati sel (x, y).
2. **Kendaraan berada sepenuhnya di dalam geladak.** $0 \leq x_i$ dan $x_i + d_x,i \leq L$, serta $0 \leq y_i$ dan $y_i + d_y,i \leq W$, dengan d_x,i dan d_y,i adalah banyak kolom dan baris yang ditempati kendaraan ke-i sesuai orientasinya.
3. **Kapasitas berat tidak terlampaui.** $\Sigma\ z_i\ b_i \leq B_max$, dengan b_i adalah berat kendaraan ke-i dan B_max adalah kapasitas kapal.

Rumus di atas adalah tujuan persoalan. Cara program menghitung nilai sebuah *state*, termasuk ketika *state* masih melanggar batasan, dijelaskan pada subbab 2.3.

1.5 Interpretasi dan Asumsi

Untuk menjaga konsistensi pemodelan dan implementasi, digunakan interpretasi dan asumsi sebagai berikut.

1. Penyebutan "truk", "peti kemas", dan "paket" pada beberapa bagian spesifikasi diinterpretasikan sebagai kapal, geladak, dan kendaraan sesuai konteks persoalan.
2. Geladak direpresentasikan sebagai grid berukuran W baris x L kolom. Sumbu x mengikuti arah panjang geladak, sedangkan sumbu y mengikuti arah lebar geladak. Koordinat (0, 0) berada di pojok kiri atas, dengan x bertambah ke kanan dan y bertambah ke bawah. Posisi kendaraan dinyatakan melalui koordinat sel pojok kiri atasnya.
3. Kendaraan berorientasi horizontal menempati l kolom x w baris. Pada orientasi vertikal, ukurannya ditukar menjadi w kolom x l baris. Perubahan orientasi merepresentasikan rotasi sebesar 90°.
4. Kendaraan yang tidak diangkut tidak memiliki koordinat penempatan pada geladak dan tidak berkontribusi terhadap total tarif pengiriman maupun berat muatan. Kendaraan tersebut tetap dicatat sebagai bagian dari *state*.
5. ETA dipahami sebagai perkiraan waktu kedatangan kendaraan dalam sisa hari. Atribut ini tetap disimpan, tetapi tidak digunakan untuk menentukan

kelayakan kendaraan atau menghitung fungsi objektif utama karena spesifikasi belum menetapkan aturan penggunaannya. ETA baru digunakan pada *objective function* alternatif untuk bonus (subbab 2.8).

6. Dimensi kendaraan dan geladak menggunakan bilangan bulat positif dalam satuan grid. Berat kendaraan dan kapasitas kapal menggunakan bilangan bulat positif dalam satuan berat yang sama.
7. Model utama menggunakan satu kapal. Kendaraan diperbolehkan bersentuhan pada sisi-sisinya selama tidak menempati sel yang sama. Jalur keluar kendaraan, jarak pengaman, dan keseimbangan muatan kapal tidak dimodelkan karena tidak tercantum sebagai batasan dalam spesifikasi.

1.6 Dataset

Seluruh eksperimen menggunakan satu dataset yang disimpan pada data/data.json. Dataset ini berisi data kapal dan daftar kendaraan dalam format JSON dan divalidasi oleh data_loader.py sebelum dipakai, misalnya untuk memastikan id kendaraan tidak duplikat dan semua dimensi berupa bilangan bulat positif. Ringkasan dataset ditunjukkan pada tabel berikut.

Tabel 1.2 Ringkasan Dataset

Parameter	Nilai
Jumlah kendaraan	90
Ukuran geladak (W x L)	10 x 20 (200 sel)
MaxCapacity	500
Total luas kendaraan / luas geladak	1020 / 200 (sekitar 5,1 kali)
Total berat kendaraan / MaxCapacity	3203 / 500 (sekitar 6,4 kali)
Rentang shipping fee	10 sampai 214
Rentang berat kendaraan	5 sampai 95
Rentang ETA	0 sampai 14 hari
Ukuran kendaraan (w x l)	1x2, 1x3, 2x3, 2x4, 2x5, 3x4, 3x5, 3x6, 4x7
Format file	JSON (`data/data.json`)

Total luas kendaraan sekitar lima kali luas geladak dan total beratnya sekitar enam kali kapasitas kapal. Artinya hanya sebagian kecil kendaraan yang bisa diangkut, sehingga pemilihan kendaraan benar-benar menentukan kualitas solusi.

1.7 Local Search

Local search adalah kelompok algoritma pencarian yang tidak menyimpan jalur dari *initial state* ke solusi, tetapi hanya fokus pada *state* yang sedang dikunjungi dan tetangganya (*neighbor*). Setiap langkah, algoritma berpindah ke *state* tetangga berdasarkan aturan tertentu dan nilai *objective function*. Pendekatan ini cocok untuk persoalan optimasi yang ruang pencariannya terlalu besar untuk ditelusuri seluruhnya, dengan konsekuensi solusi yang didapat belum tentu *global optimum*.

BAB II
PEMBAHASAN

2.1 Struktur Program

Program ditulis dalam bahasa Python dan disimpan pada *repository* GitHub kelompok. Algoritma *local search* hanya memakai *standard library* Python, sedangkan pustaka matplotlib dipakai untuk membuat visualisasi. Struktur *repository* adalah sebagai berikut.

Struktur repository

```
IF3070_G19_MantaplahPokoknya/
├── .github/workflows/
├── data/
│   ├── data.json             dataset utama 90 kendaraan dan satu kapal
│   └── fleet.json            konfigurasi bonus multi-kapal
├── output/
│   └── bonus/                hasil HC, SA, dan GA untuk bonus 1 dan 3
├── src/
│   ├── Algorithm/            implementasi HC, SA, dan GA
│   ├── Core/                 loader, state, objective, constraint, dan
tetangga
│   ├── Visualization/        ekspor visualisasi state dan riwayat objective
│   ├── experiment.py         runner bersama untuk CLI dan Streamlit
│   ├── ui_support.py         grafik interaktif dan ringkasan perbandingan
│   └── main.py               CLI utama eksperimen
├── streamlit_app.py          UI web local
├── .streamlit/config.toml    color palette UI
├── requirements.txt
└── README.md
```

Pembagian modul ini dibuat supaya semua algoritma memakai representasi *state*, *objective function*, dan operasi *neighbor* yang sama dari folder Core. Dengan begitu perbandingan antara algoritma menjadi adil karena yang berbeda hanya strategi pencariannya. Eksperimen dijalankan melalui main.py, misalnya:

```
python -m src.main --data data/data.json --seeds 1 2 3
--output output/eksperimen
```

2.2 Representasi State

Satu *state* direpresentasikan sebagai *dictionary* Python. *Key*-nya adalah id kendaraan, sedangkan *value*-nya berisi posisi dan orientasi kendaraan tersebut. Semua kendaraan selalu ada di dalam *state*, termasuk yang tidak diangkut. Kendaraan yang berada di luar kapal ditandai dengan position = None, sehingga atribut diangkut pada Tabel 1.1 tidak perlu disimpan terpisah.

Contoh representasi state

```
{
    "V001": {"position": (0, 0), "orientation": "H"},
# di geladak, pojok kiri atas di kolom 0 baris 0
    "V002": {"position": None,   "orientation": "V"},
# di luar kapal
    ...
}
```

Koordinat mengikuti asumsi pada subbab 1.5, yaitu (x, y) dengan x sebagai kolom dan y sebagai baris, serta titik acuan (*anchor*) di sel pojok kiri atas. Fungsi footprint menghitung berapa kolom dan baris yang ditempati kendaraan sesuai orientasinya, dan fungsi within_bounds memastikan kendaraan tidak keluar dari geladak.

Source code footprint dan within_bounds (src/Core/state.py)

```
def footprint(vehicle, orientation):
    """Return (columns, rows) occupied after
orientation."""
    if orientation not in ('H', 'V'):
        raise ValueError('orientation harus H atau V')
    return ((vehicle['l'], vehicle['w']) if orientation
== 'H'
            else (vehicle['w'], vehicle['l']))


def within_bounds(vehicle, position, orientation, ship):
```

```
    dx, dy = footprint(vehicle, orientation)
    if position is None:
        return True
    if not isinstance(position, (tuple, list)) or
len(position) != 2:
        return False
    x, y = position
    return (type(x) is int and type(y) is int
            and 0 <= x and 0 <= y
            and x + dx <= ship['l'] and y + dy <=
ship['w'])
```

Representasi ini membuat operasi *neighbor* mudah dilakukan, karena memindahkan, memutar, atau menukar kendaraan cukup dengan mengganti nilai pada satu atau dua *key*. Selain itu *dictionary* mudah disimpan ke JSON sehingga *initial state* dan *final state* setiap eksperimen bisa disimpan di folder `output`.

2.3 Perhitungan Objective Function

Program menghitung nilai *state* dengan pendekatan penalti. *State* di tengah pencarian boleh melanggar batasan tumpang tindih dan kapasitas, tetapi pelanggaran tersebut dikurangkan dari nilai *state* dengan bobot yang besar. Batasan geladak tetap dijaga secara langsung, artinya operasi yang membuat kendaraan keluar geladak tidak pernah dihasilkan. Nilai yang dihitung program adalah:

$$\mathbf{f(s) = R(s) - P \times (O(s) + E(s))}$$

dengan:

- R(s) adalah total tarif pengiriman kendaraan yang berada di geladak, sesuai rumus pada subbab 1.4.
- O(s) adalah jumlah kelebihan penghuni sel. Kalau satu sel ditempati k kendaraan, sel tersebut menyumbang k - 1. Jadi yang dihitung adalah kelebihan penghuni, bukan jumlah pasangan kendaraan yang bertumpuk.
- E(s) adalah kelebihan berat, yaitu max(0, total berat - MaxCapacity).

- P adalah koefisien penalti, yaitu 1 + *shipping fee* terbesar. Pada dataset kami nilainya 1 + 214 = 215.

Nilai P dipilih supaya satu sel tumpang tindih atau satu unit kelebihan berat selalu lebih merugikan daripada keuntungan memuat kendaraan termahal sekalipun. Dengan begitu algoritma selalu terdorong untuk membersihkan pelanggaran terlebih dahulu, dan *state* dengan nilai tertinggi pasti merupakan *state* yang valid. Sebuah *state* disebut valid kalau O(s) = 0 dan E(s) = 0, dan untuk *state* valid berlaku f(s) = R(s).

Source code objective function (src/Core/evaluation.py)

```
def evaluate(state, vehicles, ship):
    validate_state(state, vehicles, ship)
    penalty = 1 + max((v['shipping_fee'] for v in
vehicles.values()), default=0)
    grid = [[0] * ship['l'] for _ in range(ship['w'])]
    revenue = total_weight = loaded_count = 0
    for vid, placement in state.items():
        if placement['position'] is None:
            continue
        vehicle = vehicles[vid]
        x, y = placement['position']
        dx, dy = footprint(vehicle,
placement['orientation'])
        for yy in range(y, y + dy):
            for xx in range(x, x + dx):
                grid[yy][xx] += 1
        revenue += vehicle['shipping_fee']
        total_weight += vehicle['weight']
        loaded_count += 1
    overlap = sum(max(0, count - 1) for row in grid for
count in row)
    excess_weight = max(0, total_weight -
ship['max_capacity'])
```

```
    return {'revenue': revenue, 'total_weight':
total_weight,
            'overlap': overlap, 'excess_weight':
excess_weight,
            'penalty_coefficient': penalty,
            'score': revenue - penalty * (overlap +
excess_weight),
            'is_valid': overlap == 0 and excess_weight
== 0,
            'loaded_count': loaded_count, 'grid': grid}
```

Fungsi evaluate membangun grid geladak berukuran w x l, lalu setiap kendaraan yang berada di geladak menambah hitungan pada sel-sel yang ditempatinya. Setelah itu tumpang tindih dan kelebihan berat dihitung untuk mendapatkan *score*. Fungsi ini juga mengembalikan informasi tambahan seperti *revenue*, total berat, dan status is_valid yang nantinya dipakai untuk visualisasi dan analisis.

2.4 Initial State

Sesuai spesifikasi, *initial state* dibangkitkan secara acak dengan fungsi random_state. Setiap kendaraan diberi orientasi acak (dari orientasi yang muat di geladak), lalu dengan peluang 0,5 (load_probability) kendaraan tersebut ditaruh di posisi acak di geladak. Kalau tidak terpilih, kendaraan tetap berada di luar kapal.

Source code random initial state (src/Core/state.py)

```
def random_state(vehicles, ship, rng=None,
load_probability=0.5):
    if not 0 <= load_probability <= 1:
        raise ValueError('load_probability harus antara
0 dan 1')
    rng = rng if rng is not None else random.Random()
    result = empty_state(vehicles)
    for vid, vehicle in vehicles.items():
        orientations = [o for o in ('H', 'V')
```

```
                        if footprint(vehicle, o)[0] <=
ship['l']
                        and footprint(vehicle, o)[1] <=
ship['w']]
        orientation = rng.choice(orientations) if
orientations else rng.choice(('H', 'V'))
        result[vid]['orientation'] = orientation
        if orientations and rng.random() <
load_probability:
            dx, dy = footprint(vehicle, orientation)
            result[vid]['position'] =
(rng.randrange(ship['l'] - dx + 1),

rng.randrange(ship['w'] - dy + 1))
    return result
```

Karena posisi dipilih tanpa memperhatikan kendaraan lain, *initial state* hampir selalu tidak valid. Pada dataset kami, sekitar 37 sampai 49 kendaraan langsung ditaruh di geladak sehingga terjadi ratusan sel tumpang tindih dan kelebihan berat lebih dari 900 unit. Akibatnya nilai awal sangat negatif, sekitar -259 ribu sampai -363 ribu. Inisialisasi yang lebih pintar (misalnya *greedy*) sengaja tidak dipakai supaya sesuai dengan spesifikasi dan supaya kemampuan setiap algoritma untuk keluar dari *state* yang buruk terlihat jelas.

Agar perbandingan adil, *initial state* dibangkitkan dari *seed* yang sama untuk semua algoritma, yaitu 101, 202, dan 303. Artinya percobaan ke-1 setiap algoritma selalu dimulai dari *state* awal yang sama, begitu juga percobaan ke-2 dan ke-3.

2.5 Neighbor (Move)

Operasi *neighbor* diimplementasikan di src/Core/neighbors.py dan semuanya menghasilkan *state* baru tanpa mengubah *state* lama. Ada tiga operasi yang sesuai dengan spesifikasi:

- **move**: memindahkan sebuah kendaraan ke koordinat lain di geladak, atau mengeluarkannya dari kapal (position = None). Kendaraan yang sedang di luar juga bisa dimasukkan ke geladak lewat operasi ini.
- **rotate**: mengubah orientasi kendaraan dari H ke V atau sebaliknya pada posisi yang sama.
- **swap**: menukar posisi dua kendaraan, baik keduanya di geladak, keduanya di luar, maupun satu di dalam dan satu di luar. Setiap kendaraan tetap memakai orientasinya masing-masing.

Operasi yang membuat kendaraan keluar batas geladak akan memunculkan ValueError dan dianggap tidak valid. Modul ini menyediakan dua fungsi, yaitu iter_neighbors untuk membangkitkan seluruh tetangga dan random_neighbor untuk mengambil satu tetangga acak.

Source code random_neighbor (src/Core/neighbors.py)

```
def random_neighbor(state, vehicles, ship, rng=None,
max_attempts=100):
    validate_state(state, vehicles, ship)
    if type(max_attempts) is not int or max_attempts <
0:
        raise ValueError('max_attempts harus integer >=
0')
    if not vehicles:
        return None
    rng = rng if rng is not None else random.Random()
    ids = list(vehicles)
    for _ in range(max_attempts):
        vid = rng.choice(ids)
        operation = rng.choice(('move', 'rotate',
'swap'))
        try:
            if operation == 'rotate':
                candidate = rotate(state, vid, vehicles,
ship)
```

```
            elif operation == 'swap':
                if len(ids) < 2:
                    continue
                first, second = rng.sample(ids, 2)
                candidate = swap(state, first, second,
vehicles, ship)
            else:
                dx, dy = footprint(vehicles[vid],
state[vid]['orientation'])
                if rng.random() < 0.5:
                    position = None
                elif dx <= ship['l'] and dy <=
ship['w']:
                    position = (rng.randrange(ship['l']
- dx + 1),
                                rng.randrange(ship['w']
- dy + 1))
                else:
                    continue
                candidate = move(state, vid, position,
vehicles, ship)
        except ValueError:
            continue
        if candidate != state:
            return candidate
    return next(iter_neighbors(state, vehicles, ship),
None)
```

Fungsi random_neighbor memilih satu kendaraan dan satu jenis operasi secara acak. Untuk operasi *move*, kendaraan dikeluarkan dari kapal dengan peluang 0,5 atau dipindahkan ke posisi acak di geladak. Kalau operasi yang terpilih tidak valid, fungsi mencoba lagi sampai max_attempts kali.

2.6 Implementasi Algoritma

2.6.1 Hill-Climbing

a. Deskripsi

Hill-Climbing (HC) adalah algoritma pencarian lokal yang berusaha menemukan solusi yang lebih baik dengan berpindah dari suatu *state* ke *state* tetangga. Pada setiap langkah, algoritma mengevaluasi sejumlah solusi yang dapat dicapai dari solusi saat ini, kemudian memilih salah satu solusi berdasarkan strategi tertentu. Pencarian biasanya berhenti ketika tidak ditemukan tetangga yang lebih baik atau ketika kondisi penghentian lainnya terpenuhi.

Algoritma ini memiliki beberapa varian umum.

1. **Steepest-Ascent Hill-Climbing.** Pada setiap iterasi, seluruh tetangga dievaluasi. Algoritma kemudian memilih tetangga yang memberikan peningkatan nilai objektif paling besar. Pencarian berhenti ketika tidak ada tetangga yang lebih baik daripada *state* saat ini.
2. **Stochastic Hill-Climbing.** Algoritma mengidentifikasi tetangga yang memiliki nilai lebih baik, kemudian memilih salah satunya secara acak. Strategi ini tidak selalu memilih peningkatan terbesar sehingga dapat menghasilkan jalur pencarian yang berbeda pada setiap eksekusi.
3. **Hill-Climbing with Sideways Move.** Selain berpindah ke tetangga yang lebih baik, algoritma dapat berpindah ke tetangga dengan nilai yang sama. Langkah tersebut digunakan untuk melewati kondisi *plateau*, yaitu keadaan ketika beberapa solusi bertetangga memiliki nilai objektif yang setara. Jumlah langkah setara biasanya dibatasi untuk mencegah pencarian berlangsung tanpa kemajuan.
4. **Random-Restart Hill-Climbing.** Algoritma menjalankan Hill-Climbing beberapa kali dengan titik awal yang berbeda. Setiap pencarian menghasilkan solusi lokal, kemudian solusi terbaik dari seluruh pencarian dipilih sebagai hasil akhir. Strategi ini membantu mengurangi risiko terjebak pada *local optimum* yang kurang baik.

Hill-Climbing memiliki keunggulan berupa proses pencarian yang relatif sederhana dan efisien karena hanya berfokus pada solusi di sekitar *state* saat ini. Namun, algoritma ini dapat terjebak pada *local optimum*, *plateau*, atau kondisi ketika tidak ada arah perbaikan yang langsung terlihat. Penggunaan *sideways move* dan *random restart* dapat membantu mengatasi sebagian permasalahan tersebut, meskipun tidak menjamin ditemukannya solusi global terbaik.

b. Source Code

(src/Algorithm/hill_climbing.py)

Source code inisiasi pencarian dan solusi terbaik

```
def _search(initial_state, vehicles, ship, *, mode, rng,
max_iterations,
            max_sideways=0, neighborhood=None,
objective="shipping_fee"):
    objective = normalize_objective(objective)
    if max_iterations is not None:
        _nonnegative_int(max_iterations,
"max_iterations")
    _nonnegative_int(max_sideways, "max_sideways")
    started = perf_counter()
    current = copy_state(initial_state)
    initial_metrics = evaluate(current, vehicles, ship,
objective)
    metrics = initial_metrics

    best_valid = empty_state(vehicles, ship)
    best_valid_metrics = evaluate(best_valid, vehicles,
ship, objective)
    best_valid_source = "empty_baseline"
    if metrics["is_valid"]:
        best_valid, best_valid_metrics =
copy_state(current), metrics
        best_valid_source = "visited"
```

```
    history = [_point(0, metrics)]
    iterations = scans = evaluated = sideways =
improvements = 0
    space = neighborhood if neighborhood is not None
else Neighborhood(vehicles, ship, objective)
```

Source code evaluasi tetangga dan pemilihan Steepest/Stochastic

```
        space.bind(current)
        scans += 1
        selected = lateral = None
        selected_score = metrics["score"]
        improving_count = lateral_count = 0
        for change in space.changes():
            evaluated += 1
            score = space.score(change)
            if score > metrics["score"] and not
math.isclose(score, metrics["score"], rel_tol=1e-12,
abs_tol=1e-8):
                if mode == "stochastic":
                    improving_count += 1
                    if rng.randrange(improving_count) ==
0:
                        selected, selected_score =
change, score
                elif score > selected_score:
                    selected, selected_score = change,
score
```

Source code pemilihan dan batas Sideways Move

```
            elif mode == "sideways" and
math.isclose(score, metrics["score"], rel_tol=1e-12,
abs_tol=1e-8):
```

```
                lateral_count += 1
                if rng.randrange(lateral_count) == 0:
                    lateral = change
        if selected is None:
            if mode == "sideways" and lateral is not
None and sideways < max_sideways:
                selected = lateral
                selected_score = metrics["score"]
                sideways += 1
            else:
                reason = ("sideways_limit" if mode ==
"sideways" and lateral is not None
                          else "local_optimum")
                break
        else:
            improvements += 1
```

Source code pemanggilan Algoritma

```
def steepest_ascent_hill_climbing(initial_state,
vehicles, ship, *,
                                  seed=0,
max_iterations=None, objective="shipping_fee"):
    """Tetangga terbaik, tie terbaik mengikuti urutan
enumerasi core."""
    result = _search(initial_state, vehicles, ship,
mode="steepest",
                     rng=random.Random(seed),
max_iterations=max_iterations, objective=objective)
    result["parameters"]["seed"] = seed
    return result

def stochastic_hill_climbing(initial_state, vehicles,
ship, *,
```

```
                                seed=0, max_iterations=None,
objective="shipping_fee"):
    """Pilih uniform di antara tetangga dengan score
lebih tinggi."""
    result = _search(initial_state, vehicles, ship,
mode="stochastic",
                     rng=random.Random(seed),
max_iterations=max_iterations, objective=objective)
    result["parameters"]["seed"] = seed
    return result

def hill_climbing_sideways(initial_state, vehicles,
ship, *, seed=0,
                           max_iterations=None,
max_sideways=20, objective="shipping_fee"):
    """Steepest + langkah setara, berhenti saat batas
TOTAL sideways tercapai.

    max_sideways=0 berarti tidak menerima sideways,
tetapi perbaikan boleh.
    Penghitung tidak di-reset setelah langkah membaik.
    """
    result = _search(initial_state, vehicles, ship,
mode="sideways",
                     rng=random.Random(seed),
max_iterations=max_iterations,
                     max_sideways=max_sideways,
objective=objective)
    result["parameters"]["seed"] = seed
    return result

    space = Neighborhood(vehicles, ship, objective)
    runs = []
    for index in range(max_restarts + 1):
        initial = (initial_state if index == 0 else
```

```
                    random_state(vehicles, ship, rng,
load_probability))
        run = _search(initial, vehicles, ship,
mode="steepest", rng=rng,
                      max_iterations=max_iterations,
neighborhood=space, objective=objective)
        run["restart_index"] = index
        runs.append(run)
    best_index = max(range(len(runs)), key=lambda i:
runs[i]["final_metrics"]["score"])
    best = runs[best_index]
```

c. Penjelasan

Implementasi Hill-Climbing menggunakan fungsi _search sebagai inti pencarian untuk varian Steepest-Ascent, Stochastic, dan Sideways Move. Fungsi ini menerima *initial state*, data kendaraan dan kapal, mode pencarian, serta parameter penghentian. Pada awal pencarian, *initial state* disalin agar data masukan tidak berubah, lalu nilainya dihitung menggunakan fungsi evaluate. Program juga menyiapkan *state* kapal kosong sebagai solusi valid cadangan. Jika *initial state* sudah valid, *state* tersebut menjadi solusi valid terbaik sementara.

Pada setiap iterasi, objek Neighborhood dihubungkan dengan *state* saat ini melalui space.bind(current). Seluruh perubahan penempatan yang mungkin dibangkitkan oleh space.changes() dan dinilai menggunakan space.score(change). Penilaian ini menghitung perubahan *score* tanpa menyalin dan mengevaluasi ulang seluruh *state* untuk setiap tetangga. Setelah satu perubahan dipilih, program menerapkannya dengan space.apply(selected) dan menghitung ulang nilainya melalui evaluate. Hasil kedua cara perhitungan diperiksa agar tetap konsisten.

Perbedaan ketiga varian terletak pada cara memilih tetangga. Pada mode **Steepest-Ascent**, program memeriksa seluruh tetangga dan menyimpan kandidat dengan *score* tertinggi yang lebih baik daripada *state* saat ini. Pada mode **Stochastic**, program juga memeriksa seluruh tetangga yang lebih baik, tetapi memilih salah satunya secara acak dengan peluang yang sama. Karena itu,

Stochastic dapat mengambil peningkatan yang lebih kecil walaupun tersedia kandidat dengan peningkatan lebih besar. Kedua mode berhenti ketika tidak ditemukan tetangga yang memperbaiki *score*.

Pada mode **Sideways Move**, kandidat yang lebih baik tetap diprioritaskan. Jika tidak ada kandidat yang lebih baik, program dapat berpindah ke tetangga dengan *score* setara. Perpindahan ini memungkinkan pencarian melintasi *plateau*, yaitu kumpulan *state* bertetangga dengan nilai yang sama. Jumlah langkah setara dibatasi oleh max_sideways, penghitungnya berlaku untuk keseluruhan satu pencarian dan tidak diatur ulang setelah terjadi peningkatan. Program berhenti ketika tidak ada langkah yang dapat diambil atau batas langkah setara telah tercapai.

Setelah suatu langkah diterima, program menambah penghitung iterasi, mencatat nilai *state* ke dalam history, dan memperbarui best_valid apabila ditemukan solusi valid dengan *score* yang tidak lebih rendah. Solusi yang dikembalikan adalah *state* terminal apabila valid. Jika *state* terminal tidak valid, program menggunakan solusi valid terbaik yang disimpan sebelumnya. Informasi hasil meliputi *initial state*, *final state*, riwayat nilai, jumlah iterasi, jumlah tetangga yang diperiksa, jumlah *sideways move*, alasan penghentian, dan durasi pencarian.

Varian **Random-Restart** menggunakan pencarian Steepest-Ascent tersebut beberapa kali. Pencarian pertama dimulai dari *initial state* eksperimen, sedangkan pencarian berikutnya dimulai dari *state* acak baru. Parameter max_restarts menyatakan jumlah pencarian tambahan, sehingga max_restarts = 3 berarti empat pencarian secara keseluruhan. Program membandingkan hasil valid dari keempat pencarian dan mengembalikan hasil dengan *score* tertinggi. Jumlah iterasi masing-masing pencarian dicatat dalam iterations_per_restart, sementara iterations menyimpan jumlah seluruh langkah yang diterima.

2.6.2 Simulated Annealing

a. Deskripsi

Simulated Annealing (SA) terinspirasi dari proses pendinginan logam. Berbeda dengan *Hill-Climbing* yang memeriksa tetangga untuk memilih langkah yang membaik, setiap iterasi SA hanya mengambil satu tetangga acak. Kalau

tetangga lebih baik atau sama, tetangga langsung diterima. Kalau lebih buruk, tetangga tetap bisa diterima dengan peluang e^(ΔE/T), dengan ΔE adalah selisih nilai tetangga dan nilai sekarang, dan T adalah suhu. Saat suhu masih tinggi, langkah buruk cukup sering diterima sehingga pencarian bisa keluar dari *local optimum*. Semakin lama suhu diturunkan, peluang tersebut makin kecil dan perilaku SA makin mirip *Hill-Climbing*.

b. Source Code

Source code parameter dan pemilihan objective (src/Algorithm/simulated_annealing.py)

```
def simulated_annealing(initial_state, vehicles, ship,
*, seed=0,
                        t0=100.0, alpha=0.9999,
t_min=0.1,
                        stuck_window=1000,
history_step=10, objective="fee"):
    #stuck_window: brp iterasi tanpa perbaikan baru
dianggep stuck 1x
    #history_step: history di json disimpen tiap n
iterasi aja biar filenya ga gede
    #objective="fee" -> evaluate() biasa (spek wajib)
    #objective="eta" -> evaluate_eta(), fee dikali bobot
urgensi (bonus 1)
    if objective == "fee":
        score_fn = evaluate
    elif objective == "eta":
        score_fn = evaluate_eta
    else:
        raise ValueError('objective harus "fee" atau
"eta"')
```

Source code inisialisasi Simulated Annealing

```
    rng = random.Random(seed)
    started = perf_counter()

    current = copy_state(initial_state)
    current_metrics = score_fn(current, vehicles, ship)
    initial_metrics = current_metrics

    #simpen state valid terbaik yg pernah dilewatin,
soalnya sa bisa aja
    #nerima langkah jelek terus ga balik lagi ke state
bagus tadi
    #defaultnya kapal kosong (score 0, pasti valid) biar
slalu ada jawaban valid
    best_valid = empty_state(vehicles)
    best_valid_metrics = score_fn(best_valid, vehicles,
ship)
    best_valid_source = "empty_baseline"
    if current_metrics["is_valid"]:
        best_valid, best_valid_metrics =
copy_state(current), current_metrics
        best_valid_source = "visited"

    #buat ngitung stuck, pake score terbaik apa aja
(valid / belom)
    best_score = current_metrics["score"]
    since_improve = 0
    stuck_count = 0

    #data lengkap tiap iterasi buat plot
    scores = [current_metrics["score"]]
    temperatures = [t0]
    acceptance = []  #isinya (iterasi, e^(delta/T)) cuma
utk langkah yg lebih jelek
    history = [_point(0, t0, current_metrics)]
```

```
    #counter langkah buat analisis di laporan
    accepted_better = accepted_equal = accepted_worse =
rejected = 0

    temperature = t0
    iteration = 0
    stop_reason = "t_min"
```

Source code loop utama Simulated Annealing

```
    while temperature > t_min:
        #ambil 1 neighbor yang random
        neighbor = random_neighbor(current, vehicles,
ship, rng)
        if neighbor is None:  #dataset tanpa kendaraan
            stop_reason = "no_neighbor"
            break
        neighbor_metrics = score_fn(neighbor, vehicles,
ship)

        #hitung delta E = score tetangga - score
sekarang
        delta = neighbor_metrics["score"] -
current_metrics["score"]

        #lebih baik / sama -> terima, lebih buruk ->
terima dengan peluang e^(delta/T)
        if delta > 0:
            accept = True
            accepted_better += 1
        elif delta == 0:
            #e^(0/T) = 1 jd langkah yg sama aja pasti
diterima
            accept = True
```

```
            accepted_equal += 1
        else:
            probability = math.exp(delta / temperature)
            #dicatet buat plot e^(delta/T), delta > 0 ga
dicatet krn hasilnya > 1
            acceptance.append((iteration + 1,
probability))
            accept = rng.random() < probability
            if accept:
                accepted_worse += 1
            else:
                rejected += 1

        if accept:
            current, current_metrics = neighbor,
neighbor_metrics

        #kalo state skrg valid dan lebih bagus dari
best, update bestnya
        #pake copy biar ga ikut berubah pas current
diganti
        if current_metrics["is_valid"] and
current_metrics["score"] > best_valid_metrics["score"]:
            best_valid, best_valid_metrics =
copy_state(current), current_metrics
            best_valid_source = "visited"

        #turunin suhu
        temperature = temperature * alpha
        iteration += 1

        #ngitung stuck: kalo udah stuck_window iterasi
score terbaik ga naik2,
        #itung 1x stuck. periode stuck yg panjang tetep
diitung 1x aja
```

```
        if current_metrics["score"] > best_score:
            best_score = current_metrics["score"]
            since_improve = 0
        else:
            since_improve += 1
            if since_improve == stuck_window:
                stuck_count += 1

        #catet data buat plot
        scores.append(current_metrics["score"])
        temperatures.append(temperature)
        if iteration % history_step == 0:
            history.append(_point(iteration,
temperature, current_metrics))
```

c. Penjelasan

Fungsi simulated_annealing menerima *initial state*, data kendaraan, data kapal, dan beberapa parameter. Parameter t0, alpha, dan t_min mengatur jadwal pendinginan, stuck_window dipakai untuk menghitung frekuensi *stuck*, history_step mengatur seberapa sering riwayat pencarian disimpan ke JSON, dan objective menentukan fungsi penilaian yang dipakai (fee untuk *objective function* utama, eta untuk bonus). Fungsi penilaian disimpan di variabel score_fn sehingga loop utama tidak perlu diubah untuk kedua *objective*.

Loop utama berjalan selama T > T_min. Setiap iterasi, satu tetangga acak dibangkitkan dengan random_neighbor, lalu dihitung ΔE = *score* tetangga - *score* sekarang. Kalau ΔE > 0, tetangga diterima. Kalau ΔE = 0, tetangga juga diterima karena e^(0/T) = 1. Kalau ΔE < 0, dibangkitkan bilangan acak antara 0 dan 1, dan tetangga diterima kalau bilangan tersebut lebih kecil dari e^(ΔE/T). Di akhir iterasi suhu diturunkan secara geometris dengan T = T x α.

Dengan parameter yang kami pakai (T0 = 100, α = 0,9999, T_min = 0,1), banyak iterasinya selalu sama, yaitu ln(0,1/100) / ln(0,9999) ≈ 69.075 iterasi. Nilai T0 = 100 kami pilih karena besarnya mirip dengan *shipping fee* satu kendaraan (10 sampai 214). Dengan begitu di awal pencarian, langkah yang

mengeluarkan satu kendaraan masih punya peluang yang cukup besar untuk diterima. Nilai $\alpha = 0{,}9999$ membuat pendinginan cukup lambat, dan pada uji coba awal kami, $\alpha = 0{,}9995$ (sekitar 13.800 iterasi) memberikan rata-rata hasil yang lebih rendah pada tiga *seed* yang sama.

Selain alur dasar tersebut, ada beberapa hal tambahan pada implementasi SA:

- **Menyimpan *state* valid terbaik.** Karena SA bisa menerima langkah buruk, *state* tempat SA berhenti belum tentu yang terbaik. Karena itu setiap kali *current state* valid dan lebih baik dari catatan sebelumnya, salinannya disimpan sebagai best_valid. *Final state* yang dilaporkan adalah best_valid, sedangkan *state* tempat SA berhenti disimpan sebagai terminal_state. Nilai awal best_valid adalah kapal kosong (*score* 0, pasti valid) supaya program selalu punya jawaban yang sah. Salinan dibuat dengan copy_state supaya catatan terbaik tidak ikut berubah ketika *current state* berganti.
- **Data plot e^(ΔE/T).** Nilai peluang hanya dicatat untuk langkah yang lebih buruk ($\Delta E < 0$). Untuk $\Delta E > 0$ nilainya lebih dari 1 sehingga tidak bermakna sebagai peluang.
- **Frekuensi *stuck*.** Spesifikasi tidak memberikan definisi *stuck*, sehingga kami mendefinisikannya sendiri. Satu kejadian *stuck* dihitung ketika *score* terbaik yang pernah dicapai (valid maupun tidak) tidak membaik selama 1000 iterasi berturut-turut (stuck_window = 1000). Satu periode *stuck* yang panjang tetap dihitung satu kali.
- **Penghitung langkah.** Jumlah langkah yang lebih baik, sama, lebih buruk tapi diterima, dan ditolak dicatat untuk bahan analisis.
- **Riwayat pencarian.** *Score* dan suhu setiap iterasi disimpan di trace untuk membuat plot, sedangkan history yang ditulis ke result.json hanya diambil setiap 10 iterasi supaya ukuran filenya tidak terlalu besar.

2.6.3 Genetic Algorithm

a. Deskripsi

Genetic Algorithm digunakan untuk mencari kombinasi kendaraan yang diangkut serta posisi dan orientasinya pada geladak kapal. Setiap individu merepresentasikan suatu konfigurasi penempatan lengkap, sedangkan populasi merupakan kumpulan beberapa konfigurasi alternatif. Gen dalam individu menyimpan posisi dan orientasi satu kendaraan. Kendaraan yang tidak diangkut tetap dicatat dengan posisi bernilai None.

Kualitas individu dihitung dengan fungsi evaluasi bersama, yaitu total tarif pengiriman dikurangi penalti tumpang tindih dan kelebihan berat. Implementasi ini melalui seleksi turnamen, uniform crossover, mutasi melalui operasi neighbor, serta elitism. Selain itu, dibuatkannya operasi repair untuk memperbaiki anak yang mengalami tumpang tindih atau kelebihan berat.

b. Source Code

Source code parameter Genetic Algorithm (src/Algorithm/genetic_algorithm.py)

```
def run_ga(ship, vehicles, population_size=100,
generations=300,
           crossover_rate=0.8, mutation_rate=0.5,
           tournament_size=3, elite_count=2, seed=42,
repair=False):

    validate_params(
        population_size, generations, crossover_rate,
mutation_rate,
        tournament_size, elite_count, seed
    )

    if seed is None:
        seed = random.SystemRandom().randrange(2 ** 31)
    rng = random.Random(seed)
```

Source code inisialisasi Genetic Algorithm

```
    start = perf_counter()

    population = [
        random_state(vehicles, ship, rng)
        for _ in range(population_size)
    ]
    initial_population = [copy_state(s) for s in
population]

    fitnesses = [
        metrics_of(s, vehicles, ship)['fitness']
        for s in population
    ]

    initial_best_index = max(
        range(population_size),
        key=lambda i: fitnesses[i]
    )
    initial_best_state =
copy_state(population[initial_best_index])

    best_fitness = {
        'fitness': None, 'state': None, 'generation':
None
    }
    best_valid = {
        'revenue': None, 'state': None, 'generation':
None
    }

    def observe(pop, fits, generation):
        valid_count = 0

        for state, fit in zip(pop, fits):
```

```
            if (
                best_fitness['fitness'] is None
                or fit > best_fitness['fitness']
            ):
                best_fitness.update(
                    fitness=fit,
                    state=copy_state(state),
                    generation=generation
                )

            m = evaluate(state, vehicles, ship)

            if m['is_valid']:
                valid_count += 1

                if (
                    best_valid['revenue'] is None
                    or m['revenue'] >
best_valid['revenue']
                ):
                    best_valid.update(
                        revenue=m['revenue'],
                        state=copy_state(state),
                        generation=generation
                    )

        so_far = (
            best_valid['revenue']
            if best_valid['revenue'] is not None else 0
        )

        return {
            'generation': generation,
            'candidate_peak': max(fits),
            'mean_fitness': sum(fits) / len(fits),
```

```
            'valid_count': valid_count,
            'best_valid_revenue_so_far': so_far
        }

    history = [observe(population, fitnesses, 0)]
```

Source code evaluasi fitness, seleksi induk, crossover, dan mutasi Genetic Algorithm

```
def metrics_of(state, vehicles, ship):
    result = evaluate(state, vehicles, ship)
    result.pop('grid')
    result['fitness'] = result['score']
    return result


def tournament_select(population, fitnesses,
tournament_size, rng):
    if not 1 <= tournament_size <= len(population):
        raise ValueError('tournament_size harus
1..ukuran populasi')

    contenders = rng.sample(
        range(len(population)), tournament_size
    )
    return max(contenders, key=lambda i: fitnesses[i])


def uniform_crossover(parent1, parent2, rng):
    child1, child2 = {}, {}

    for vid in parent1:
        if rng.random() < 0.5:
            child1[vid], child2[vid] = (
                deepcopy(parent1[vid]),
```

```
                deepcopy(parent2[vid])
            )
        else:
            child1[vid], child2[vid] = (
                deepcopy(parent2[vid]),
                deepcopy(parent1[vid])
            )

    return child1, child2


def mutate(state, vehicles, ship, rng):
    mutated = random_neighbor(state, vehicles, ship,
rng)
    return state if mutated is None else mutated
```

Source code perbaikan overlap dan kelebihan berat Genetic Algorithm

```
 def repair_state(state, vehicles, ship):
    w_ship, l_ship = ship['w'], ship['l']
    grid = [[0] * l_ship for _ in range(w_ship)]
    total_weight = 0
    overlap_cells = 0
    loaded = []

    for vid, p in state.items():
        if p['position'] is None:
            continue

        v = vehicles[vid]
        total_weight += v['weight']
        loaded.append(vid)

        x, y = p['position']
        dx, dy = footprint(v, p['orientation'])
```

```
        for yy in range(y, y + dy):
            for xx in range(x, x + dx):
                grid[yy][xx] += 1
                if grid[yy][xx] == 2:
                    overlap_cells += 1

    if overlap_cells == 0 and total_weight <=
ship['max_capacity']:
        return state

    result = copy_state(state)

    if overlap_cells > 0:
        candidates = sorted(
            loaded,
            key=lambda vid: (
                vehicles[vid]['shipping_fee']
                / max(1, vehicles[vid]['w'] *
vehicles[vid]['l']),
                vehicles[vid]['shipping_fee']
            )
        )

        for vid in candidates:
            if overlap_cells == 0:
                break

            p = result[vid]
            if p['position'] is None:
                continue

            v = vehicles[vid]
            x, y = p['position']
            dx, dy = footprint(v, p['orientation'])
```

```
            in_overlap = any(
                grid[yy][xx] > 1
                for yy in range(y, y + dy)
                for xx in range(x, x + dx)
            )

            if in_overlap:
                result[vid]['position'] = None
                total_weight -= v['weight']

                for yy in range(y, y + dy):
                    for xx in range(x, x + dx):
                        if grid[yy][xx] == 2:
                            overlap_cells -= 1
                        grid[yy][xx] -= 1

    if total_weight > ship['max_capacity']:
        loaded_now = [
            vid for vid, p in result.items()
            if p['position'] is not None
        ]

        loaded_now.sort(
            key=lambda vid: (
                vehicles[vid]['shipping_fee']
                / max(1, vehicles[vid]['weight']),
                vehicles[vid]['shipping_fee']
            )
        )

        for vid in loaded_now:
            if total_weight <= ship['max_capacity']:
                break
```

```
            result[vid]['position'] = None
            total_weight -= vehicles[vid]['weight']

    return result
```

Source code loop utama Genetic Algorithm

```
   for generation in range(1, generations + 1):
        order = sorted(
            range(population_size),
            key=lambda i: fitnesses[i],
            reverse=True
        )

        next_population = [
            copy_state(population[i])
            for i in order[:elite_count]
        ]
        next_fitnesses = [
            fitnesses[i]
            for i in order[:elite_count]
        ]

        children = []
        slots = population_size - elite_count

        while len(children) < slots:
            p1 = population[
                tournament_select(
                    population, fitnesses,
tournament_size, rng
                )
            ]
            p2 = population[
                tournament_select(
                    population, fitnesses,
tournament_size, rng
                )
            ]

            if rng.random() < crossover_rate:
```

```
                c1, c2 = uniform_crossover(p1, p2, rng)
            else:
                c1, c2 = copy_state(p1), copy_state(p2)

            for child in (c1, c2):
                if len(children) < slots:
                    if rng.random() < mutation_rate:
                        child = mutate(child, vehicles,
ship, rng)

                    if repair:
                        child = repair_state(child,
vehicles, ship)

                    children.append(child)

        for child in children:
            next_population.append(child)
            next_fitnesses.append(
                metrics_of(child, vehicles,
ship)['fitness']
            )

        population, fitnesses = next_population,
next_fitnesses

        history.append(
            observe(population, fitnesses, generation)
        )
```

Source code penentuan solusi akhir dan penyimpanan hasil Genetic Algorithm

```
   duration = perf_counter() - start

    from_fallback = best_valid['state'] is None
```

```
    if from_fallback:
        final_valid_state = empty_state(vehicles)
        best_valid_generation = None
    else:
        final_valid_state = best_valid['state']
        best_valid_generation = best_valid['generation']

    final_valid_metrics = metrics_of(
        final_valid_state, vehicles, ship
    )

    if not final_valid_metrics['is_valid']:
        raise RuntimeError(
            'Solusi akhir ternyata tidak valid menurut
evaluate()'
        )

    return {
        'params': {
            'population_size': population_size,
            'generations': generations,
            'crossover_rate': crossover_rate,
            'mutation_rate': mutation_rate,
            'tournament_size': tournament_size,
            'elite_count': elite_count,
            'seed': seed,
            'repair': repair
        },
        'ship': dict(ship),
        'vehicles': deepcopy(vehicles),
        'initial_population': initial_population,
        'initial_best_state': initial_best_state,
        'initial_best_metrics': metrics_of(
            initial_best_state, vehicles, ship
        ),
```

```
        'final_population': [
            copy_state(s) for s in population
        ],
        'best_fitness_state': best_fitness['state'],
        'best_fitness_metrics': metrics_of(
            best_fitness['state'], vehicles, ship
        ),
        'best_fitness_generation':
best_fitness['generation'],
        'best_valid_state':
copy_state(final_valid_state),
        'best_valid_metrics': final_valid_metrics,
        'best_valid_from_fallback': from_fallback,
        'best_valid_generation': best_valid_generation,
        'duration_seconds': duration,
        'generations_run': generations,
        'history': history,
    }
```

c. Penjelasan

Fungsi utama yang digunakan adalah run_ga, yaitu fungsi yang menerima data kapal, data kendaraan, dan parameter untuk mengatur proses pencarian. Parameter population_size menentukan jumlah alternatif susunan yang dipertahankan dalam setiap generasi, sedangkan generations menentukan banyaknya pergantian generasi. Parameter crossover_rate mengatur peluang penggabungan dua induk, mutation_rate mengatur peluang perubahan pada setiap anak, tournament_size menentukan jumlah peserta seleksi induk, dan elite_count menentukan jumlah individu terbaik yang dipertahankna. Parameter repair mengaktifkan perbaikan terhadap anak yang melanggar batasan, sementara seed digunakan untuk mengatur pengacakan agar percobaan dapat diulang berkali-kali.

Loop utama dijalankan sebanyak generations. Pada setiap generasi, individu diurutkan dari fitness tertinggi ke terendah. Sebanyak elite_count individu terbaik kemudian disalin langsung ke populasi berikutnya, sedangkan

sisanya diisi oleh anak hasil crossover dan mutasi. Dengan populasi 100 dan dua elite, setiap generasi mempertahankan dua individu lama dan membentuk 98 anak. Kedua elite tidak diubah agar susunan terbaik yang sudah ditemukan tetap terjaga.

Untuk menghasilkan anak, dua induk dipilih menggunakan *tournament selection.* Setiap pemilihan mengambil tiga individu secara acak, lalu memilih individu dengan fitness tertinggi. Misalnya, dari peserta dengan fitness −200, 425, dan 700, individu bernilai 700 terpilih sebagai induk. Cara ini tetap dapat digunakan untuk fitness negatif karena hanya membandingkan nilainya. Pemilihan dilakukan secara terpisah untuk setiap induk, sehingga individu yang sama bisa terpilih dua kali. Kedua induk tersebut digunakan untuk membentuk dua anak, yaitu susunan baru yang menggabungkan penempatan kendaraan dari kedua induk. Proses penggabungan ini disebut crossover dan dilakukan dengan peluang 0,8. Program membangkitkan bilangan acak r antara 0 dan 1, jika $r < 0{,}8$, fungsi uniform_crossover dijalankan. Jika tidak, kedua anak dibuat sebagai salinan masing-masing induk.

Dalam crossover, posisi dan orientasi setiap kendaraan diambil dari salah satu induk dengan peluang yang sama. Misalnya, jika anak pertama mengambil penempatan kendaraan A dari induk pertama, anak kedua mengambil penempatan A dari induk kedua. Pemilihan ini diulang untuk seluruh kendaraan. Posisi dan orientasi diwariskan sebagai satu paket agar setiap kendaraan tetap berada dalam batas geladak. Namun, gabungan penempatan dari dua susunan berbeda masih dapat menyebabkan overlap atau kelebihan berat.

Anak yang terbentuk kemudian memiliki peluang 0,5 untuk mengalami mutasi melalui random_neighbor. Mutasi mencoba satu perubahan berupa perpindahan, rotasi, atau pertukaran kendaraan agar muncul variasi susunan baru. Dengan 98 anak, rata-rata jumlah anak yang diharapkan menjalani percobaan mutasi adalah $98 \times 0{,}5 = 49$, tetapi jumlah aktualnya dapat berbeda pada setiap generasi. Jika tidak tersedia neighbor, anak tetap pada susunan sebelumnya. Hasil mutasi tetap diproses meskipun fitness-nya menurun.

Sebelum dimasukkan ke populasi berikutnya, setiap anak diperiksa melalui repair_state karena repair diaktifkan pada konfigurasi yang digunakan. Pemeriksaan ini berlaku juga untuk anak yang tidak mengalami mutasi. Fungsi

tersebut menghitung jumlah kendaraan pada setiap sel dan total berat muatan. Jika susunan sudah valid, anak tidak diubah. Perbaikan yang dilakukan terfokus pada perbaikan overlap berdasarkan tarif per luas dan perbaikan kapasitas berdasarkan tarif per berat. Jika dua kendaraan memiliki rasio yang sama, kendaraan dengan tarif lebih rendah dikeluarkan terlebih dahulu. Dengan aturan ini, repair berusaha mempertahankan kendaraan yang memberikan tarif lebih besar untuk ruang dan kapasitas berat yang digunakan. Namun, repair hanya mengeluarkan kendaraan, tanpa mencoba memindahkan atau memasukkannya kembali ke ruang yang masih kosong. Jadi, susunan anak memenuhi batasan, tetapi belum tentu menjadi kombinasi dengan tarif tertinggi.

Setelah proses tersebut selesai, fitness setiap anak dihitung. Anak-anak kemudian digabungkan dengan elite untuk menggantikan populasi lama. Jumlah individu tetap dijaga sebanyak population_size. Jika hanya tersisa satu slot, hanya satu anak dari pasangan terakhir yang diproses. Selain alur dasar ini, ada beberapa hal tambahan pada implementasi Genetic Algorithm ini:

- **Menyimpan state valid terbaik.** Fungsi observe memeriksa populasi awal dan setiap populasi baru, apabila ada yang paling bagus, maka akan disimpan dalam best_valid. Hal yang sama dilakukan pada best_fitness.
- **Menghitung perkembangan populasi.** Nilai candidate_peak dan mean_fitness digunakan untuk melihat individu terbaik dan populasi secara keseluruhan mengalami perbaikan.
- **Kondisi berhenti dan pemeriksaan hasil.** Pencarian berhenti setelah jumlah yang ditentukan tercapai, meskipun hasilnya tidak membaik selama beberapa generasi.

2.6.4 Visualisasi

Visualisasi dibuat dengan matplotlib dan disimpan otomatis sebagai file PNG di folder output setiap kali eksperimen dijalankan. Visualisasi yang dihasilkan untuk setiap percobaan *Hill Climbing* adalah sebagai berikut.

- **State awal dan state akhir.** Geladak digambar sebagai grid. Kendaraan yang dimuat ditampilkan sebagai persegi panjang berwarna dengan label ID dan orientasi. Sel yang ditempati lebih dari satu kendaraan diberi arsiran merah agar pelanggaran tumpang tindih pada *initial state* terlihat.

Bagian bawah gambar mencantumkan jumlah kendaraan di dalam dan di luar kapal serta daftar kendaraan yang tidak dimuat. Judul gambar menunjukkan status validitas, *score*, nilai objective, total *shipping fee*, jumlah sel tumpang tindih, dan kelebihan berat. Untuk armada multi-kapal, setiap kapal ditampilkan pada panel tersendiri.

- **Plot perkembangan nilai terhadap iterasi.** Panel atas menampilkan *score* sepanjang langkah yang diterima. *Score* mencakup penalti jika susunan pada iterasi tersebut melanggar batasan. Panel bawah menampilkan total *shipping fee* kendaraan yang sedang dimuat. Pemisahan kedua panel memudahkan pembaca melihat proses perbaikan susunan sekaligus perubahan tarif pengiriman. Titik dengan iterasi 0 adalah *initial state*.
- **Plot Random-Restart.** Setiap pencarian digambar sebagai garis tersendiri pada `history.png` utama, dengan label "Awal" untuk pencarian pertama dan "Restart 1", "Restart 2", serta "Restart 3" untuk pencarian ulang. Sumbu iterasi dimulai lagi dari 0 pada setiap garis karena setiap *restart* merupakan pencarian baru. Selain grafik gabungan, gambar *state* awal, *state* akhir, dan riwayat masing-masing pencarian disimpan dalam folder `restart_00` sampai `restart_03`.

Source code (src/Visualization/hill_climbing.py)

```
def plot_history(result, path):
    plt = _pyplot()
    fig, axes = plt.subplots(2, 1, figsize=(11, 8),
sharex=True)
    runs = result.get("runs", [result])
    for index, run in enumerate(runs):
        history = run["history"]
        xs = [point["iteration"] for point in history]
        label = "Awal" if index == 0 else f"Restart
{index}"
        axes[0].plot(xs, [p["score"] for p in history],
label=label, marker=".", markersize=3)
```

```
        axes[1].plot(xs, [p["revenue"] for p in
history], label=label, marker=".", markersize=3)
    objective = result["parameters"].get("objective",
"shipping_fee")
    axes[0].set_ylabel(f"Score ({objective} - penalti)")
    axes[1].set_ylabel("Total shipping fee")
    axes[1].set_xlabel("Langkah diterima (0 = initial
state, dihitung per pencarian)")
```

Visualisasi yang dihasilkan untuk setiap percobaan *Simulated Annealing* adalah sebagai berikut.

- **State awal dan state akhir.** Geladak digambar sebagai grid, dan setiap kendaraan di geladak digambar sebagai persegi panjang berwarna dengan label id dan orientasinya. Sel yang tumpang tindih diberi arsiran merah. Di bagian bawah gambar ditampilkan jumlah kendaraan di dalam dan di luar kapal beserta daftar kendaraan yang di luar. Judul gambar memuat *score*, total fee, berat terhadap kapasitas, tumpang tindih, dan kelebihan berat. Fungsi gambar ini dipakai bersama oleh semua algoritma.
- **Plot *objective function* terhadap iterasi.** Panel atas menunjukkan *score* dengan skala *symlog*, karena nilai awalnya bisa mencapai minus ratusan ribu akibat penalti. Dengan skala ini bagian akhir pencarian yang mendekati nilai terbaik tetap terlihat. Garis putus-putus merah menunjukkan nilai *best valid*. Panel bawah menunjukkan suhu dengan skala log, sehingga pendinginan geometris terlihat sebagai garis lurus.
- **Plot e^(ΔE/T) terhadap iterasi.** Setiap titik adalah satu langkah yang lebih buruk, dengan sumbu y menunjukkan peluang langkah tersebut diterima. Judul plot memuat jumlah langkah buruk yang diterima dan yang ditolak.

Source code plot e^(ΔE/T) (src/Visualization/simulated_annealing.py)

```
def plot_acceptance(result, trace, path):
    plt = _pyplot()
    fig, ax = plt.subplots(figsize=(11, 5))
    points = trace["acceptance"]
    if points:
        xs = [it for it, _ in points]
        ys = [prob for _, prob in points]
        #titiknya puluhan ribu, jadi dibikin kecil &
agak transparan
        ax.scatter(xs, ys, s=1, alpha=0.4)
    ax.set_ylim(-0.05, 1.05)
    ax.set_xlabel("iterasi")
    ax.set_ylabel("e^(ΔE / T)")
    ax.grid(alpha=0.25)
    ax.set_title(f"peluang nerima langkah lebih jelek
(ΔE < 0) | "
                 f"diterima
{result['accepted_worse_moves']}, "
                 f"ditolak {result['rejected_moves']}")
    fig.tight_layout()
    fig.savefig(path, dpi=130)
    plt.close(fig)
```

Visualisasi yang dihasilkan untuk setiap percobaan *Genetic Algorithm* adalah sebagai berikut.

1. **State awal dan state akhir.** Geladak digambar sebagai grid, dan setiap kendaraan di geladak digambar sebagai persegi panjang berwarna dengan label id dan orientasinya. Sel yang tumpang tindih diberi arsiran merah. Di bagian bawah gambar ditampilkan jumlah kendaraan di dalam dan di luar kapal beserta daftar kendaraan yang di luar. Judul gambar memuat *score*, total fee, berat terhadap kapasitas, tumpang tindih, dan kelebihan berat. Fungsi gambar ini dipakai bersama oleh semua algoritma.
2. **Plot fitness maksimum dan rata-rata populasi terhadap generasi.** Grafik ini menampilkan dua garis evaluasi evolusi populasi:
    - Garis biru menunjukkan perkembangan individu terbaik dengan fitness tertinggi dari setiap generasi.
    - Garis hijau menunjukkan perkembangan kualitas rata-rata seluruh populasi untuk melihat konvergensi.
    - Garis putus-putus merah menunjukkan nilai solusi valid terbaik yang ditemukan.

Source code plot fitness populasi (src/Visualization/genetic_algorithm_vis.py):

```
def history_figure(history, final_score=None,
duration=None, params=None):
    from matplotlib.figure import Figure

    fig = Figure(figsize=(8.5, 4.2), dpi=140)
    ax = fig.subplots()

    gens = [h['generation'] for h in history]
    peak = [h['candidate_peak'] for h in history]
    mean = [h['mean_fitness'] for h in history]

    # Dua garis utama: fitness terbaik & rata-rata
populasi
```

```
    ax.plot(gens, peak, label='Fitness maksimum populasi
(candidate_peak)', color='#1f77b4', lw=1.5)
    ax.plot(gens, mean, label='Fitness rata-rata
populasi (mean_fitness)', color='#2ca02c', lw=1.5)

    # Garis acuan best valid
    if final_score is not None:
        ax.axhline(final_score, color='red',
linestyle='--', lw=1.2, label=f'best valid =
{final_score}')

    # Header ringkasan hasil & parameter
    if params and duration is not None and final_score
is not None:
        ax.set_title(
            f"genetic algorithm | final score
{final_score} | {duration:.2f} detik\n"
            f"pop_size={params.get('population_size')},
generations={params.get('generations')}, "
            f"crossover={params.get('crossover_rate')},
mutation={params.get('mutation_rate')}, "

f"tournament={params.get('tournament_size')},
elitism={params.get('elite_count')}, "
            f"seed={params.get('seed')}",
            fontsize=9.5
        )

    ax.set_xlabel('Generasi', fontsize=9.5)
    ax.set_ylabel('Nilai fitness', fontsize=9.5)
    ax.grid(True, linestyle='-', color='#ebebeb',
lw=0.7)
    ax.legend(loc='lower right', fontsize=8.5,
framealpha=0.9)
    fig.tight_layout()
```

```
    return fig
```

Semua angka hasil eksperimen juga disimpan dalam result.json per percobaan dan summary.csv per algoritma, sehingga tabel pada subbab berikutnya bisa dicek ulang langsung dari *repository*.

2.6.5 Heuristik yang Digunakan

Spesifikasi memperbolehkan heuristik selama masih dalam lingkup *local search*. Heuristik umum yang kami pakai adalah sebagai berikut.

1. **Penalti dengan koefisien P = 1 + fee terbesar.** Seperti dijelaskan di subbab 2.3, nilai ini menjamin *state* terbaik selalu valid tanpa harus membuang tetangga yang tidak valid.
2. **Fallback ke *state* valid terbaik.** Kalau pencarian berhenti di *state* yang tidak valid, yang dilaporkan adalah *state* valid terbaik yang pernah dikunjungi, atau kapal kosong kalau belum pernah ada. Pada semua eksperimen *Simulated Annealing*, *final state* selalu valid.
3. **Pemilihan T0 berdasarkan skala fee.** T0 disesuaikan dengan besar *shipping fee* satu kendaraan supaya peluang menerima langkah buruk di awal pencarian tidak terlalu kecil maupun terlalu besar.
4. **Evaluasi delta untuk Hill-Climbing.** Kandidat direpresentasikan sebagai perubahan penempatan satu atau dua kendaraan, bukan sebagai salinan lengkap seluruh *state*. Saat menilai kandidat, program menghitung perubahan nilai kendaraan yang dimuat, jumlah sel tumpang tindih, dan kelebihan berat akibat perubahan tersebut. Nilai ini ditambahkan pada *score state* saat ini untuk memperoleh *score* tetangga. Setelah tetangga terpilih, program tetap menghitung ulang nilainya dengan evaluator utama sebagai pemeriksaan konsistensi.
5. **Repair pada Genetic Algorithm.** Setiap anak hasil crossover dan mutasi diperiksa menggunakan repair_state. Jika terjadi overlap, kendaraan yang masih terlibat konflik dikeluarkan secara bertahap berdasarkan rasio shipping_fee / (w × l) terendah. Jika berat muatan masih melebihi

kapasitas, kendaraan dikeluarkan berdasarkan rasio shipping_fee / weight terendah. Jika rasionya sama, kendaraan dengan tarif lebih rendah didahulukan. Heuristik ini berusaha mempertahankan kendaraan yang memberikan tarif lebih besar untuk ruang dan kapasitas berat yang digunakan. Repair menghasilkan anak yang memenuhi batasan, tetapi tidak menjamin kombinasi muatan optimal karena hanya mengeluarkan kendaraan tanpa menata ulang atau mengisi kembali ruang kosong.

2.7 Hasil Eksperimen

2.7.1 Setup Eksperimen

Semua eksperimen memakai dataset pada subbab 1.6. Setiap algoritma (selain *Genetic Algorithm*) dijalankan tiga kali dengan *seed* 101, 202, dan 303. *Initial state* dibangkitkan dari *seed* yang sama, sehingga percobaan dengan nomor yang sama pada algoritma berbeda selalu berangkat dari *state* awal yang sama. Kondisi *initial state* ketiga percobaan ditunjukkan pada tabel berikut.

Tabel 2.1 Initial State Setiap Percobaan

Percobaan (seed)	Score awal	Fee di geladak	Kendaraan di geladak	Sel tumpang tindih	Kelebihan berat
1 (101)	-355.819	3.661	49	401	1.271
2 (202)	-363.486	3.734	49	408	1.300
3 (303)	-258.960	2.695	37	271	946

Eksperimen dijalankan dengan perintah berikut. Folder *output* harus berbeda untuk setiap eksperimen karena `main.py` menolak menulis ke folder yang sudah berisi file, supaya hasil lama tidak tertimpa.

```
python -m src.main --algorithm steepest --seeds 42
--output output/hc_steepest
```

```
python -m src.main --algorithm simulated_annealing
--seeds 101 202 303 --output output/simulated_annealing
```

```
python -m src.main --algorithm simulated_annealing
--objective eta --seeds 101 202 303 --output
output/bonus_objective_eta
```

```
101, 202, 303 | ForEach-Object { python -m
src.Algorithm.genetic_algorithm --data data/data.json
--seed $_ }
```

2.7.2 Hill-Climbing

Tabel 2.2 Hasil Eksperimen Steepest-Ascent Hill-Climbing

Percobaan (seed)	Score awal	Score akhir (best valid)	Iterasi	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	−355.819	1.333	48	29	498/500	4,92
2 (202)	−363.486	1.503	50	27	499/500	5,28
3 (303)	−258.960	1.415	34	24	498/500	3,42
Rata-rata	−326.088,33	1.417,00	44,00	26,67	498,33/500	4,54

Tabel 2.3 Hasil Eksperimen Stochastic Hill-Climbing

Percobaan (seed)	Score awal	Score akhir (best valid)	Iterasi	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	−355.819	1.250	168	33	500/500	18,49
2 (202)	−363.486	1.211	194	32	500/500	20,95
3 (303)	−258.960	1.277	136	31	500/500	13,76
Rata-rata	−326.088,33	1.246,00	166,00	32,00	500/500	17,74

Tabel 2.4 Hasil Eksperimen Sideways Move

Percobaan (seed)	Score awal	Score akhir (best valid)	Iterasi	Sideways Move	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	−355.819	1.333	68	20	29	498/500	6,90

Percobaan (seed)	Score awal	Score akhir (best valid)	Iterasi	Sideways Move	Kendaraan dimuat	Berat	Durasi (s)
2 (202)	−363.486	1.503	70	20	27	499/500	7,16
3 (303)	−258.960	1.455	58	20	25	499/500	5,58
Rata-rata	−326.088,33	1.430,33	65,33	20	27,00	498,67/500	6,55

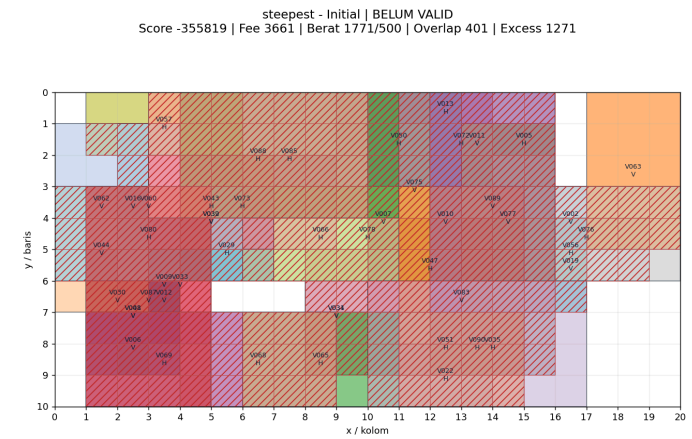
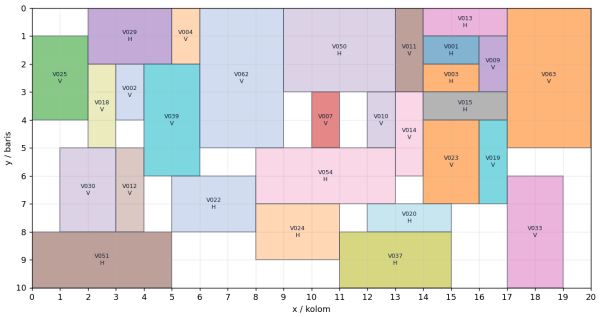
Tabel 2.5 Hasil Eksperimen Random-Restart Hill-Climbing

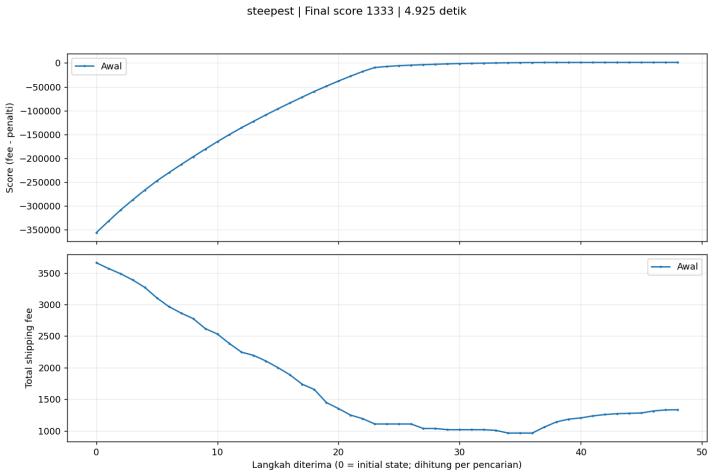
Percobaan (seed)	Score awal	Score akhir (best valid)	Pencarian	Total Iterasi	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	−355.819	1.333	4	182	29	498/500	18,60
2 (202)	−363.486	1.503	4	183	27	499/500	18,72
3 (303)	−258.960	1.415	4	147	24	498/500	14,98
Rata-rata	−326.088,33	1.417,00	4	170,67	26,67	498,33/500	17,43

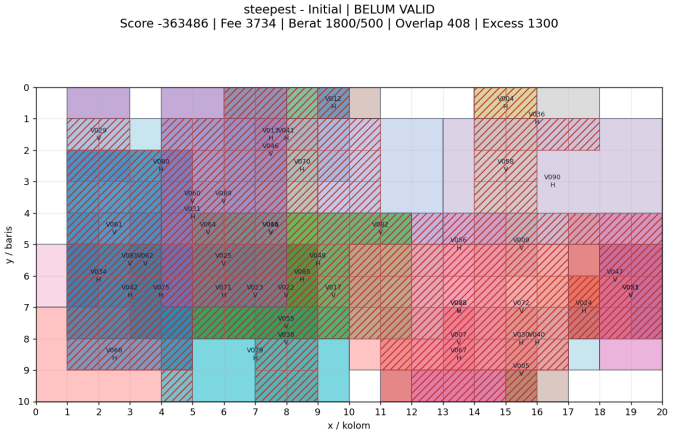
Tabel 2.6 Rincian Iterasi Random-Restart Hill-Climbing

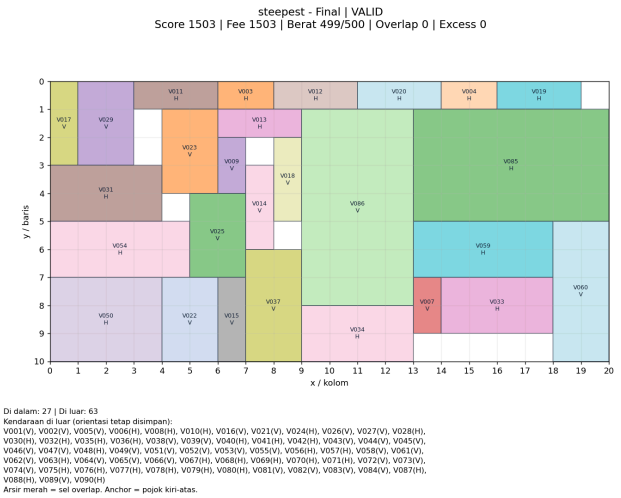
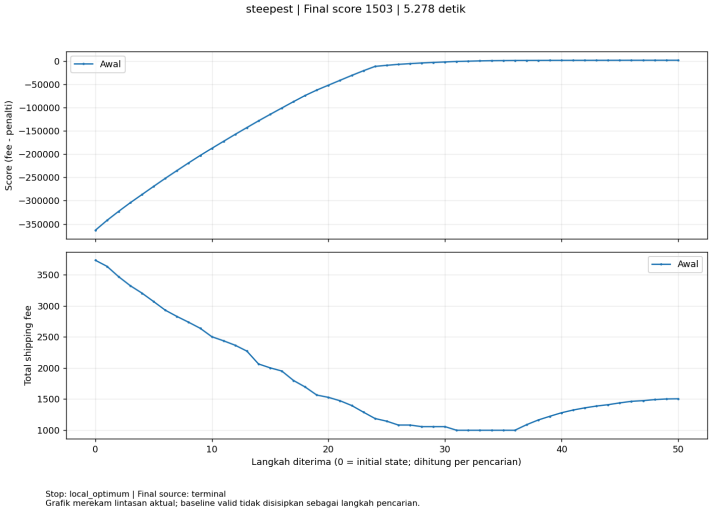
Percobaan (seed)	Pencarian Awal	Restart 1	Restart 2	Restart 3	Pencarian terbaik
1 (101)	48	48	46	40	Pencarian awal
2 (202)	50	50	38	45	Pencarian awal
3 (303)	34	34	33	46	Pencarian awal
(101)	48	48	46	40	Pencarian awal

Steepest-Ascent

Percobaan 1 (seed 101)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(H), V003(H), V004(V), V014(V), V015(H), V017(V), V018(V), V020(H), V021(H), V023(V), V024(H), V025(V), V026(H), V027(H), V028(V), V036(H), V037(H), V038(V), V040(V), V042(H), V045(H), V046(V), V048(V), V049(H), V052(V), V053(V), V054(H), V055(V), V058(V), V059(H), V061(V), V064(H), V067(H), V070(H), V071(V), V074(V), V079(V), V081(V), V082(H), V084(V), V086(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.1 State Awal Steepest-Ascent Percobaan 1**
State akhir	 Di dalam: 29 | Di luar: 61 Kendaraan di luar (orientasi tetap disimpan): V005(H), V006(V), V008(V), V016(V), V017(V), V021(H), V026(H), V027(H), V028(V), V031(V), V032(V), V034(V), V035(H), V036(H), V038(V), V040(V), V041(V), V042(H), V043(H), V044(V), V045(H), V046(V), V047(H), V048(V), V049(H), V052(V), V053(V), V055(V), V056(H), V057(H), V058(V), V059(H), V060(V), V061(V), V064(H), V065(H), V066(H), V067(H), V068(H), V069(H), V070(H), V071(V), V072(H), V073(H), V074(V), V075(V), V076(H), V077(V), V078(H), V079(V), V080(H), V081(V), V082(H), V083(V), V084(V), V085(H), V086(H), V087(V), V088(H), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.2 State Akhir Steepest-Ascent Percobaan 1**

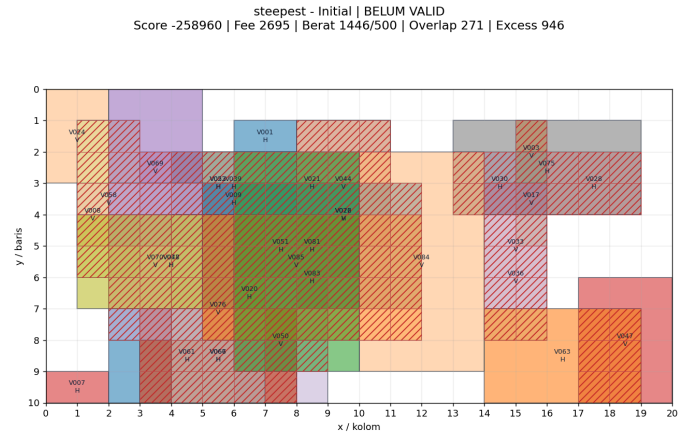
Plot objective function	 Stop: local_optimum | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.3 Plot Objective Function Steepest-Ascent Percobaan 1**

Percobaan 2 (seed 202)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V003(H), V006(H), V008(H), V010(H), V011(H), V015(V), V016(V), V018(V), V019(H), V020(H), V026(V), V027(V), V028(H), V032(H), V033(H), V035(H), V037(V), V039(V), V043(V), V044(V), V045(V), V049(V), V050(H), V051(V), V052(V), V054(H), V057(H), V059(H), V063(H), V065(V), V069(H), V074(V), V076(H), V077(H), V078(H), V083(V), V084(V), V086(V), V087(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.4 State Awal Steepest-Ascent Percobaan 2**

State akhir	 **Gambar 2.5 State Akhir Steepest-Ascent Percobaan 2**
Plot objective function	 **Gambar 2.6 Plot Objective Function Steepest-Ascent Percobaan 2**

Percobaan 3 (seed 303)

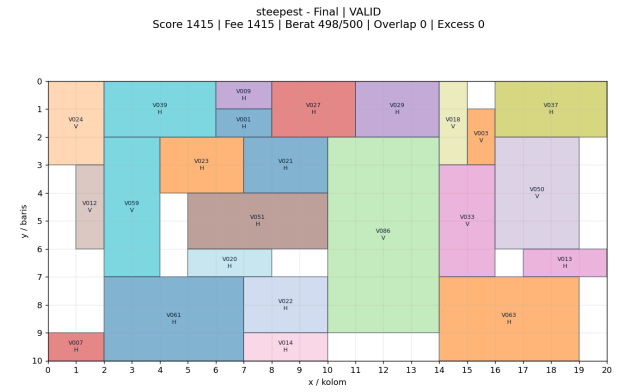
State awal



Di dalam: 37 | Di luar: 53
Kendaraan di luar (orientasi tetap disimpan):
V002(V), V004(V), V005(H), V006(H), V010(V), V011(H), V012(V), V013(H), V014(H), V015(V), V016(H), V018(V), V019(H), V022(H), V025(V), V027(H), V029(H), V031(H), V032(V), V034(H), V035(V), V037(H), V038(V), V040(V), V041(V), V042(H), V043(V), V045(V), V046(V), V049(V), V053(V), V054(V), V055(V), V056(V), V057(V), V059(V), V060(V), V062(V), V064(H), V065(H), V068(V), V071(V), V073(V), V074(V), V078(V), V079(V), V080(V), V082(V), V086(V), V087(V), V088(H), V089(V), V090(V)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

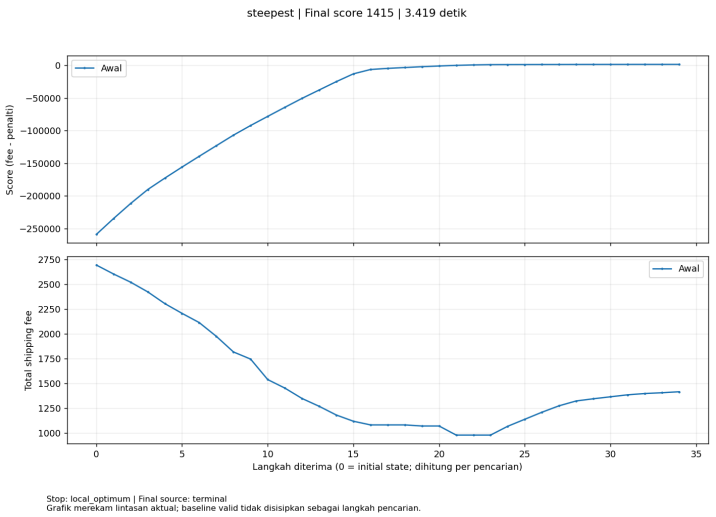
Gambar 2.7 State Awal Steepest-Ascent Percobaan 3

State akhir

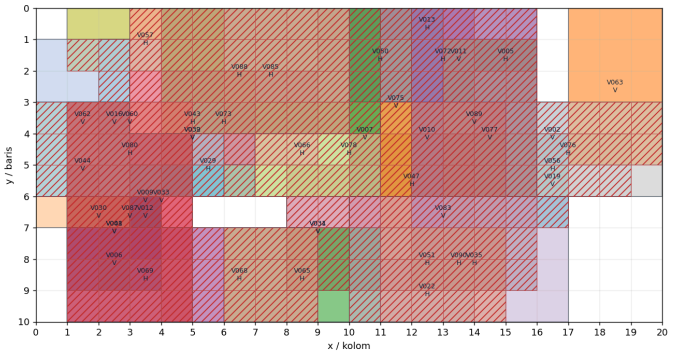


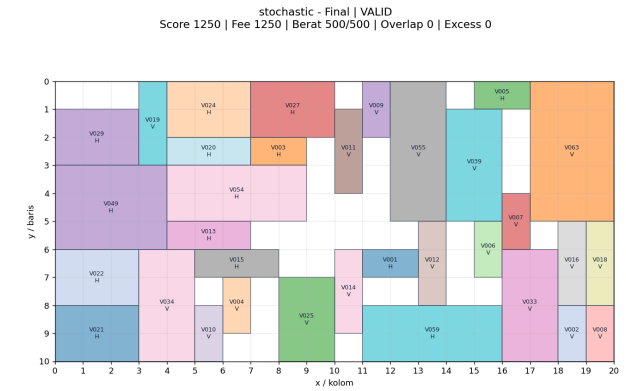
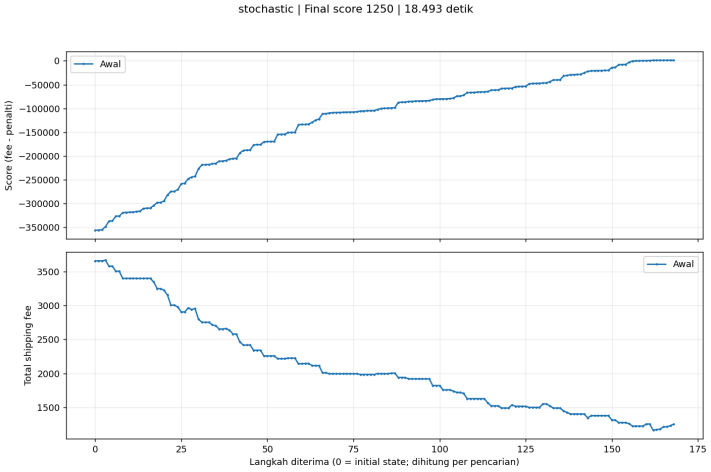
Di dalam: 24 | Di luar: 66
Kendaraan di luar (orientasi tetap disimpan):
V002(V), V004(V), V005(H), V006(H), V008(V), V010(V), V011(H), V015(V), V016(H), V017(V), V019(H), V025(V), V026(H), V028(H), V030(H), V031(H), V032(V), V034(H), V035(V), V036(V), V038(V), V040(V), V041(V), V042(H), V043(V), V044(V), V045(V), V046(V), V047(V), V048(H), V049(V), V052(H), V053(V), V054(V), V055(V), V056(V), V057(V), V058(V), V060(V), V062(V), V064(H), V065(H), V066(H), V067(H), V068(V), V069(V), V070(V), V071(V), V072(V), V073(V), V074(V), V075(H), V076(V), V077(H), V078(V), V079(V), V080(V), V081(H), V082(V), V083(H), V084(V), V085(V), V087(V), V088(H), V089(V), V090(V)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

Gambar 2.8 State Akhir Steepest-Ascent Percobaan 3

Plot objective function	 Stop: local_optimum | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.9 Plot Objective Function Steepest-Ascent Percobaan 3**

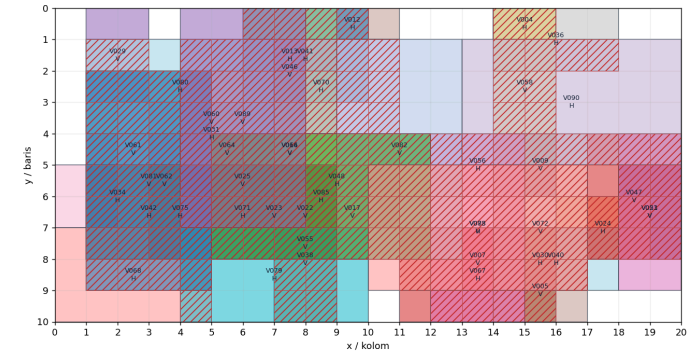
Stochastic

Percobaan 1 (seed 101)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(H), V003(H), V004(V), V014(V), V015(H), V017(V), V018(V), V020(H), V021(H), V023(V), V024(H), V025(V), V026(H), V027(H), V028(V), V036(H), V037(H), V038(V), V040(V), V042(H), V045(H), V046(V), V048(V), V049(H), V052(V), V053(V), V054(H), V055(V), V058(V), V059(H), V061(V), V064(H), V067(H), V070(H), V071(V), V074(V), V079(V), V081(V), V082(H), V084(V), V086(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.10 State Awal Stochastic Percobaan 1**

State akhir	 Di dalam: 33 | Di luar: 57 Kendaraan di luar (orientasi tetap disimpan): V017(V), V023(V), V026(H), V028(V), V030(V), V031(V), V032(V), V035(H), V036(H), V037(H), V038(V), V040(V), V041(V), V042(H), V043(H), V044(V), V045(H), V046(V), V047(H), V048(V), V050(H), V051(H), V052(V), V053(V), V056(H), V057(H), V058(V), V060(V), V061(V), V062(V), V064(H), V065(H), V066(H), V067(H), V068(H), V069(H), V070(H), V071(V), V072(H), V073(H), V074(V), V075(V), V076(H), V077(V), V078(H), V079(V), V080(H), V081(V), V082(H), V083(V), V084(V), V085(H), V086(H), V087(V), V088(H), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.11 State Akhir Stochastic Percobaan 1**
Plot objective function	 Stop: local_optimum | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.12 Plot Objective Function Stochastic Percobaan 1**

Percobaan 2 (seed 202)

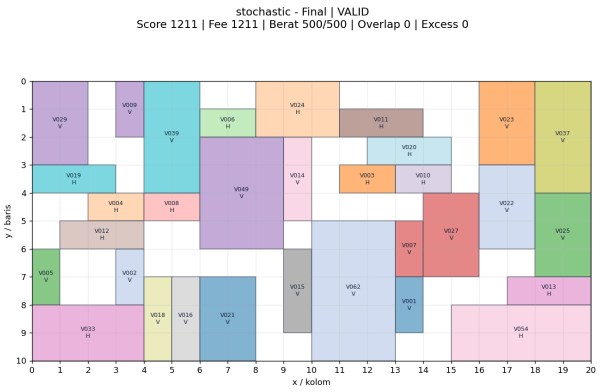
State awal



Di dalam: 49 | Di luar: 41
Kendaraan di luar (orientasi tetap disimpan):
V001(V), V002(V), V003(H), V006(H), V008(H), V010(H), V011(H), V015(V), V016(V), V018(V), V019(H), V020(H), V026(V), V027(V), V028(H), V032(H), V033(H), V035(H), V037(V), V039(V), V043(V), V044(V), V045(V), V049(V), V050(H), V051(V), V052(V), V054(H), V057(H), V059(H), V063(H), V065(V), V069(H), V074(V), V076(H), V077(H), V078(H), V083(V), V084(V), V086(V), V087(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

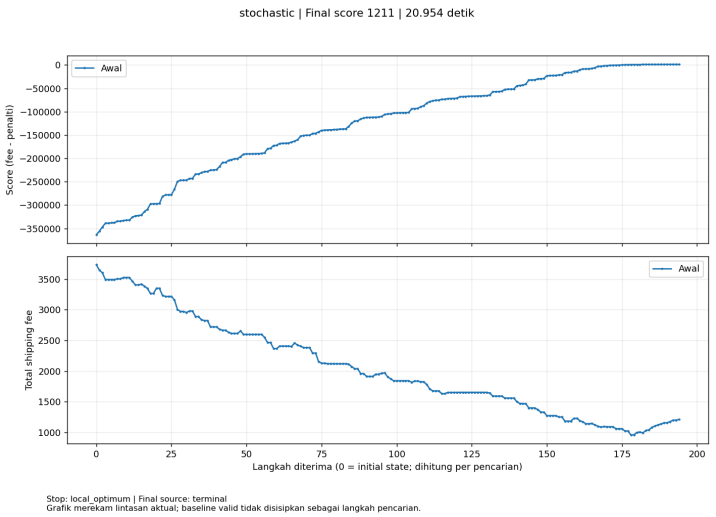
Gambar 2.13 State Awal Stochastic Percobaan 2

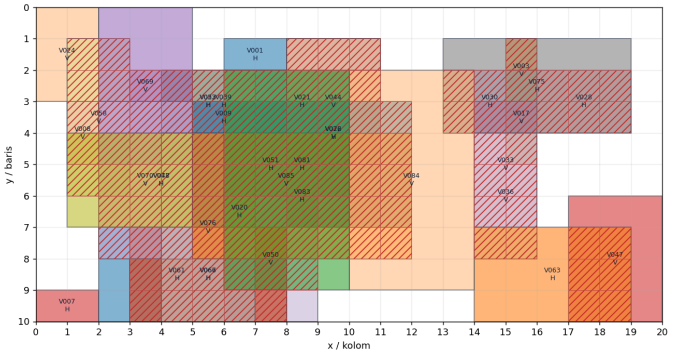
State akhir

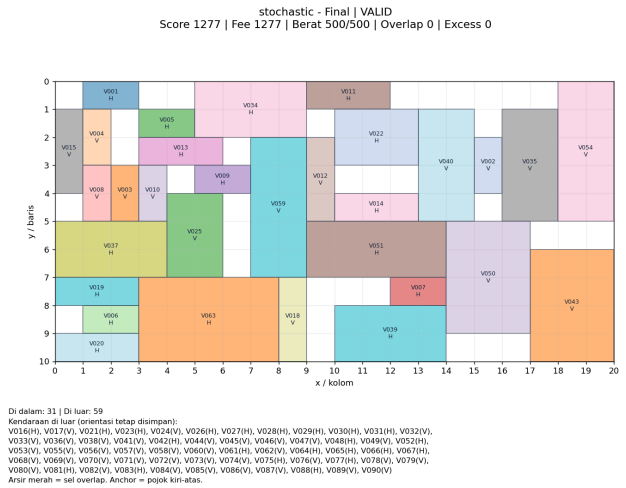
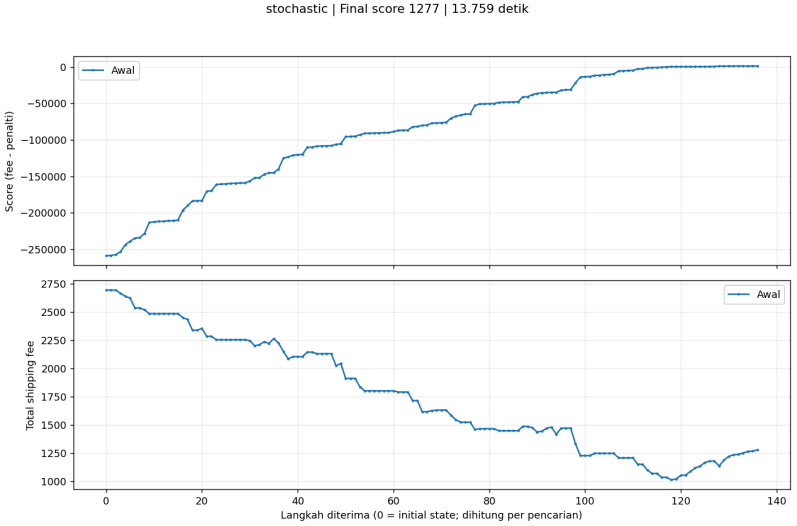


Di dalam: 32 | Di luar: 58
Kendaraan di luar (orientasi tetap disimpan):
V017(V), V026(V), V028(H), V030(H), V031(H), V032(H), V034(H), V035(H), V036(H), V038(V), V040(H), V041(H), V042(H), V043(V), V044(V), V045(V), V046(V), V047(V), V048(H), V050(H), V051(V), V052(V), V053(V), V055(V), V056(H), V057(H), V058(V), V059(H), V060(V), V061(V), V063(H), V064(V), V065(V), V066(V), V067(H), V068(H), V069(H), V070(H), V071(H), V072(V), V073(V), V074(V), V075(H), V076(H), V077(H), V078(H), V079(H), V080(H), V081(V), V082(V), V083(V), V084(V), V085(H), V086(V), V087(H), V088(H), V089(V), V090(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

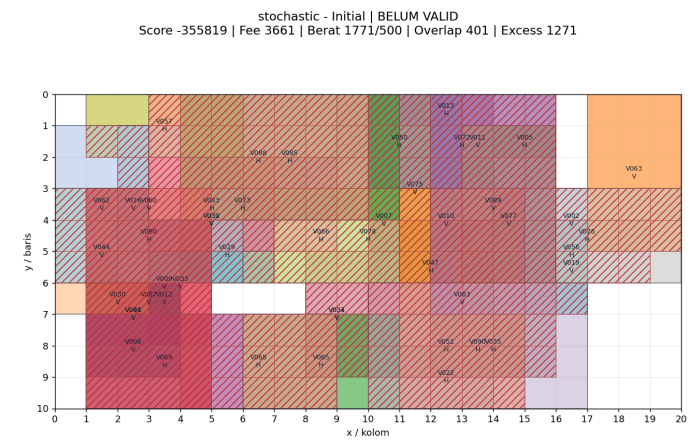
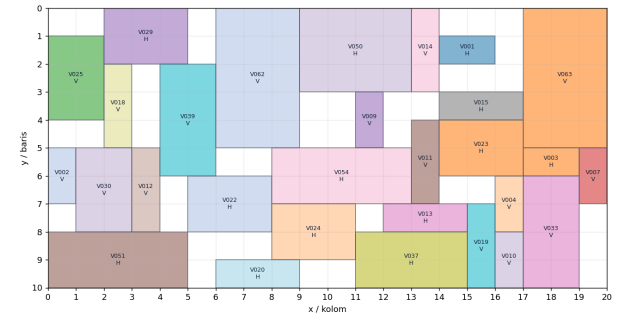
Gambar 2.14 State Akhir Stochastic Percobaan 2

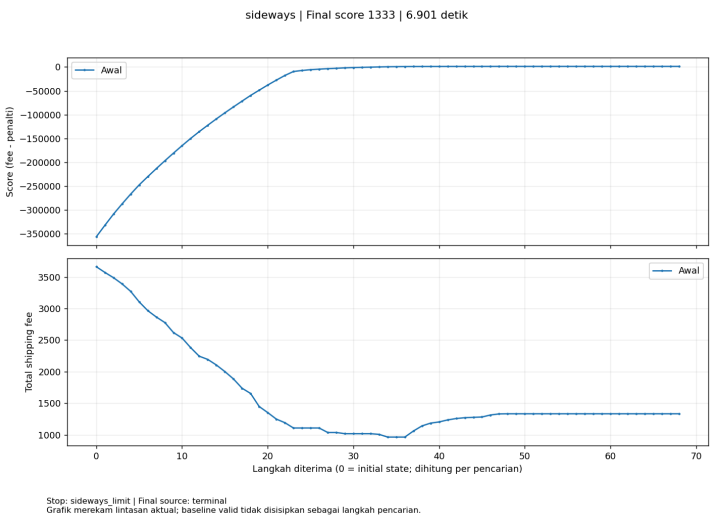
Plot objective function	 Stop: local_optimum | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.15 Plot Objective Function Stochastic Percobaan 2**

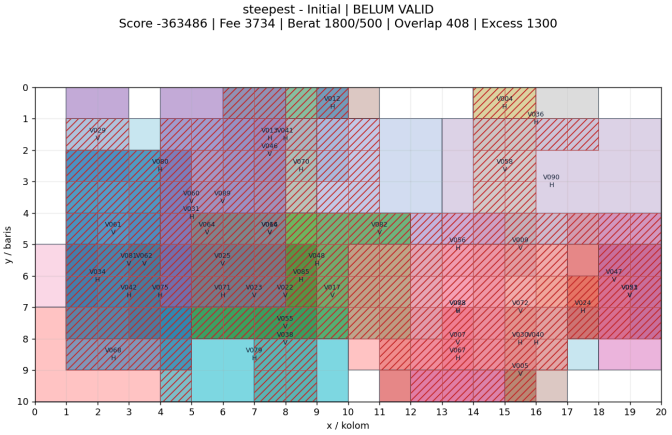
Percobaan 3 (seed 303)	
State awal	steepest - Initial | BELUM VALID Score -258960 | Fee 2695 | Berat 1446/500 | Overlap 271 | Excess 946  Di dalam: 37 | Di luar: 53 Kendaraan di luar (orientasi tetap disimpan): V002(V), V004(V), V005(H), V006(H), V010(V), V011(H), V012(V), V013(H), V014(H), V015(V), V016(H), V018(V), V019(H), V022(H), V025(V), V027(H), V029(H), V031(H), V032(V), V034(H), V035(V), V037(H), V038(V), V040(V), V041(V), V042(H), V043(V), V045(V), V046(V), V049(V), V053(V), V054(V), V055(V), V056(V), V057(V), V059(V), V060(V), V062(V), V064(H), V065(H), V068(V), V071(V), V073(V), V074(V), V078(V), V079(V), V080(V), V082(V), V086(V), V087(V), V088(H), V089(V), V090(V). Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.16 State Awal Stochastic Percobaan 3**

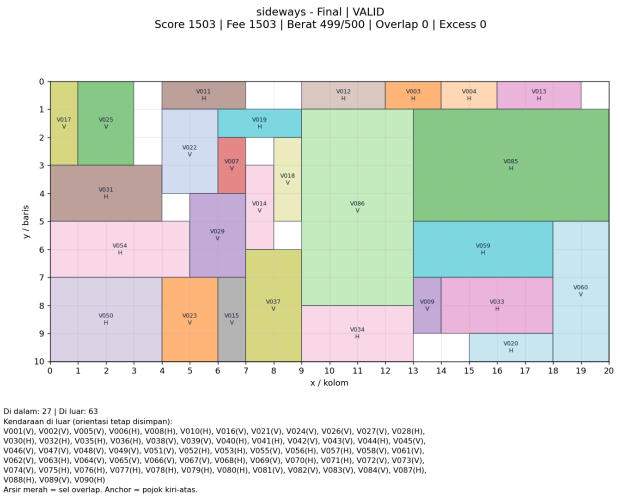
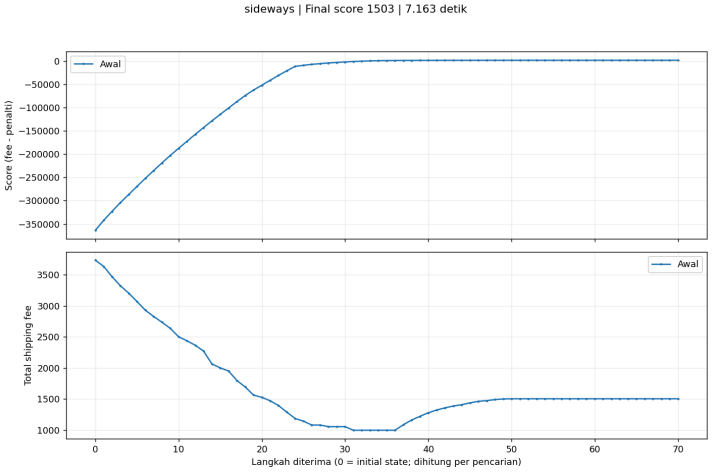
State akhir	 Di dalam: 31 | Di luar: 59 Kendaraan di luar (orientasi tetap disimpan): V016(H), V017(V), V021(H), V023(H), V024(V), V026(H), V027(H), V028(H), V029(H), V030(H), V031(H), V032(V), V033(V), V036(V), V038(V), V041(V), V042(H), V044(V), V045(V), V046(V), V047(V), V048(H), V049(V), V052(H), V053(V), V055(V), V056(V), V057(V), V058(V), V060(V), V061(H), V062(V), V064(H), V065(H), V066(H), V067(H), V068(V), V069(V), V070(V), V071(V), V072(V), V073(V), V074(V), V075(H), V076(V), V077(H), V078(V), V079(V), V080(V), V081(H), V082(V), V083(H), V084(V), V085(V), V086(V), V087(V), V088(H), V089(V), V090(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.17 State Akhir Stochastic Percobaan 3**
Plot objective function	 Stop: local_optimum | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.18 Plot Objective Function Stochastic Percobaan 3**

Sideways

Percobaan 1 (seed 101)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(H), V003(H), V004(V), V014(V), V015(H), V017(V), V018(V), V020(H), V021(H), V023(V), V024(H), V025(V), V026(H), V027(H), V028(V), V036(H), V037(H), V038(V), V040(V), V042(H), V045(H), V046(V), V048(V), V049(H), V052(V), V053(V), V054(H), V055(V), V058(V), V059(H), V061(V), V064(H), V067(H), V070(H), V071(V), V074(V), V079(V), V081(V), V082(H), V084(V), V086(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.19 State Awal Sideways Percobaan 1**
State akhir	 Di dalam: 29 | Di luar: 61 Kendaraan di luar (orientasi tetap disimpan): V005(V), V006(V), V008(V), V016(V), V017(V), V021(H), V026(H), V027(H), V028(V), V031(V), V032(V), V034(V), V035(H), V036(H), V038(V), V040(V), V041(V), V042(V), V043(H), V044(V), V045(H), V046(V), V047(H), V048(V), V049(H), V052(V), V053(V), V055(V), V056(H), V057(H), V058(V), V059(H), V060(V), V061(V), V064(H), V065(H), V066(H), V067(H), V068(H), V069(H), V070(H), V071(V), V072(H), V073(H), V074(V), V075(V), V076(H), V077(V), V078(H), V079(V), V080(H), V081(V), V082(V), V083(V), V084(V), V085(H), V086(H), V087(V), V088(H), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.20 State Akhir Sideways Percobaan 1**

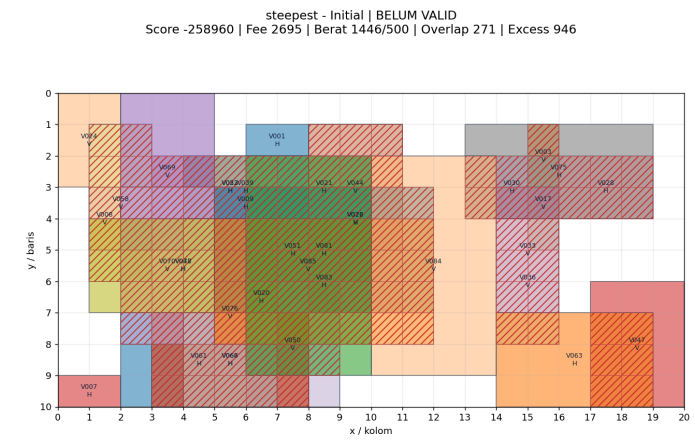
Plot objective function	 Stop: sideways_limit | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.21 Plot Objective Function Sideways Percobaan 1**

Percobaan 2 (seed 202)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V003(H), V006(H), V008(H), V010(H), V011(H), V015(V), V016(V), V018(V), V019(H), V020(H), V026(V), V027(V), V028(H), V032(H), V033(H), V035(H), V037(V), V039(V), V043(V), V044(V), V045(V), V049(V), V050(H), V051(V), V052(V), V054(H), V057(H), V059(H), V063(H), V065(V), V069(H), V074(V), V076(H), V077(H), V078(H), V083(V), V084(V), V086(V), V087(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.22 State Awal Sideways Percobaan 2**

State akhir	 Di dalam: 27 | Di luar: 63 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V005(V), V006(H), V008(H), V010(H), V016(V), V021(V), V024(V), V026(V), V027(V), V028(H), V030(H), V032(H), V035(H), V036(H), V038(V), V039(V), V040(H), V041(H), V042(V), V043(V), V044(H), V045(V), V046(V), V047(V), V048(V), V049(V), V051(V), V052(H), V053(H), V055(V), V056(H), V057(H), V058(V), V061(V), V062(V), V063(H), V064(V), V065(V), V066(V), V067(V), V068(H), V069(V), V070(H), V071(H), V072(V), V073(V), V074(V), V075(H), V076(H), V077(H), V078(H), V079(H), V080(H), V081(V), V082(V), V083(V), V084(V), V087(H), V088(H), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.23 State Akhir Sideways Percobaan 2**
Plot objective function	 Stop: sideways_limit | Final source: terminal Grafik merekam lintasan aktual; baseline valid tidak disisipkan sebagai langkah pencarian. **Gambar 2.24 Plot Objective Function Sideways Percobaan 2**

Percobaan 3 (seed 303)

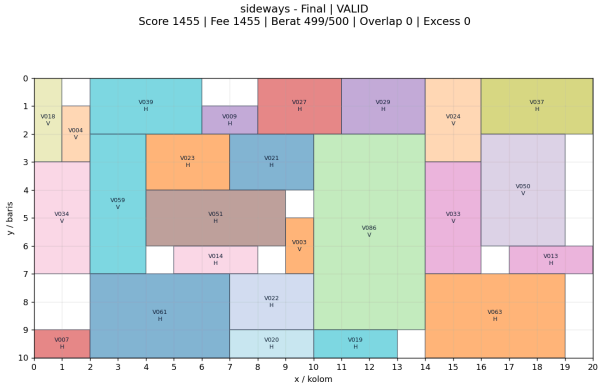
State awal



Di dalam: 37 | Di luar: 53
Kendaraan di luar (orientasi tetap disimpan):
V002(V), V004(V), V005(H), V006(H), V010(V), V011(H), V012(V), V013(H), V014(H), V015(V), V016(H), V018(V), V019(H), V022(H), V025(V), V027(H), V029(H), V031(H), V032(V), V034(H), V035(V), V037(H), V038(V), V040(V), V041(V), V042(H), V043(V), V045(V), V046(V), V049(V), V053(V), V054(V), V055(V), V056(V), V057(V), V059(V), V060(V), V062(V), V064(H), V065(H), V068(V), V071(V), V073(V), V074(V), V078(V), V079(V), V080(V), V082(V), V086(V), V087(V), V088(H), V089(V), V090(V)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

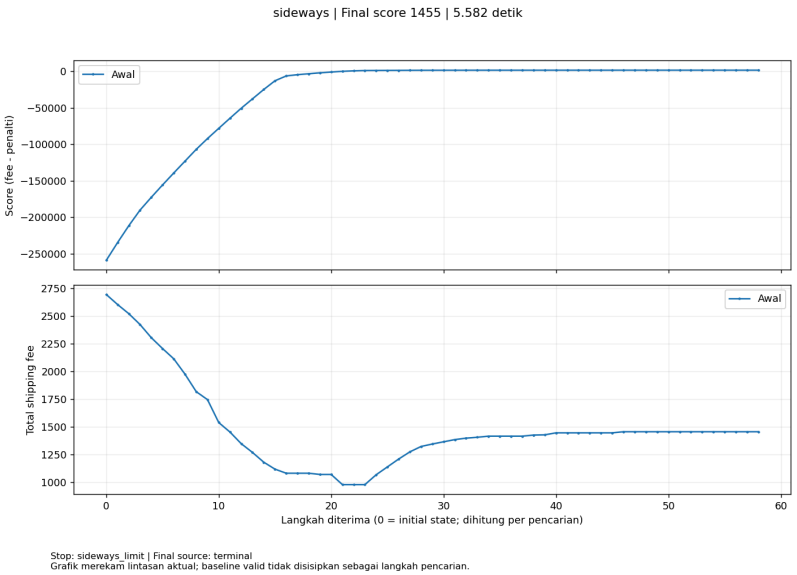
Gambar 2.25 State Awal Sideways Percobaan 3

State akhir

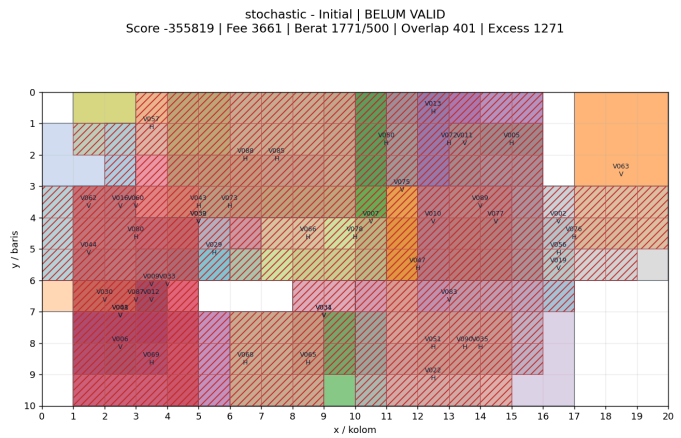


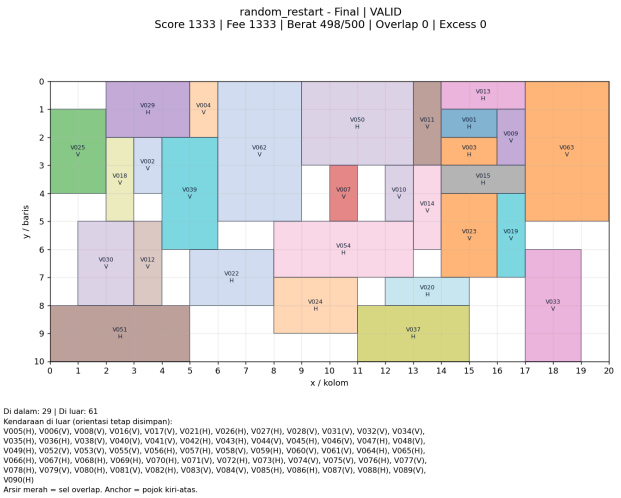
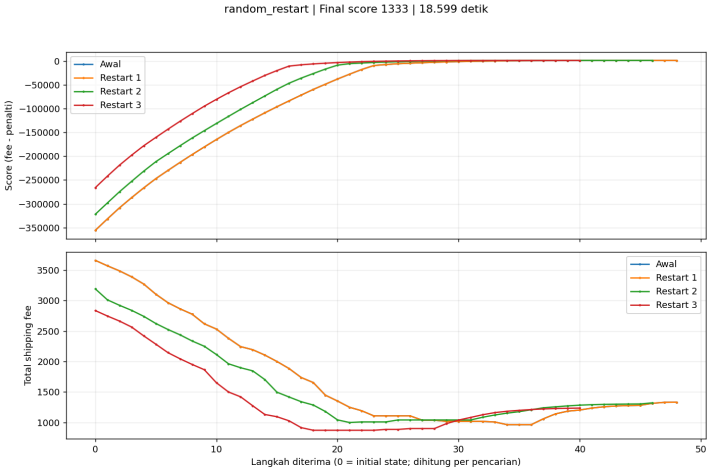
Di dalam: 25 | Di luar: 65
Kendaraan di luar (orientasi tetap disimpan):
V001(H), V002(V), V005(H), V006(H), V008(V), V010(V), V011(H), V012(V), V015(V), V016(H), V017(V), V025(V), V026(H), V028(H), V030(H), V031(H), V032(V), V035(V), V036(V), V038(H), V040(V), V041(V), V042(H), V043(V), V044(V), V045(V), V046(V), V047(V), V048(H), V049(V), V052(V), V053(V), V054(V), V055(H), V056(H), V057(V), V058(V), V060(V), V062(H), V064(H), V065(H), V066(H), V067(V), V068(V), V069(V), V070(V), V071(V), V072(V), V073(V), V074(V), V075(H), V076(V), V077(V), V078(V), V079(V), V080(V), V081(V), V082(V), V083(H), V084(V), V085(V), V087(V), V088(H), V089(V), V090(V)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

Gambar 2.26 State Akhir Sideways Percobaan 3

Plot objective function	 **Gambar 2.27 Plot Objective Function Sideways Percobaan 3**

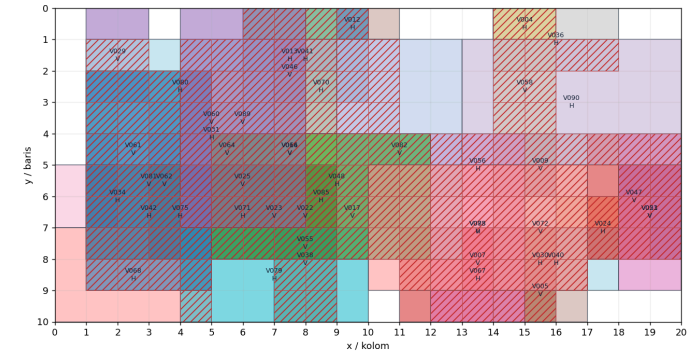
Random Restart

Percobaan 1 (seed 101)	
State awal	 **Gambar 2.28 State Awal Random Restart Percobaan 1**

State akhir	 **Gambar 2.29 State Akhir Random Restart Percobaan 1**
Plot objective function	 **Gambar 2.30 Plot Objective Function Random Restart Percobaan 1**

Percobaan 2 (seed 202)

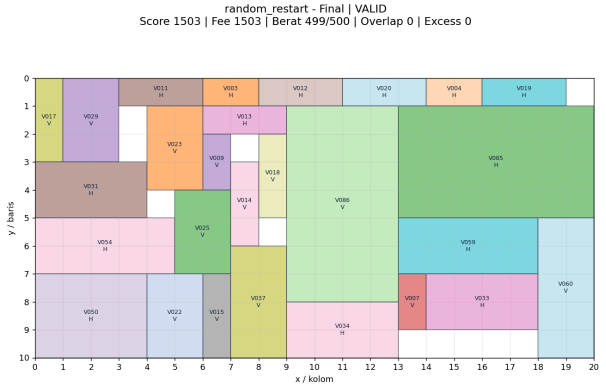
State awal



Di dalam: 49 | Di luar: 41
Kendaraan di luar (orientasi tetap disimpan):
V001(V), V002(V), V003(H), V006(H), V008(H), V010(H), V011(H), V015(V), V016(V), V018(V), V019(H), V020(H), V026(V), V027(V), V028(H), V032(H), V033(H), V035(H), V037(V), V039(V), V043(V), V044(V), V045(V), V049(V), V050(H), V051(V), V052(V), V054(H), V057(H), V059(H), V063(H), V065(V), V069(H), V074(V), V076(H), V077(H), V078(H), V083(V), V084(V), V086(V), V087(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

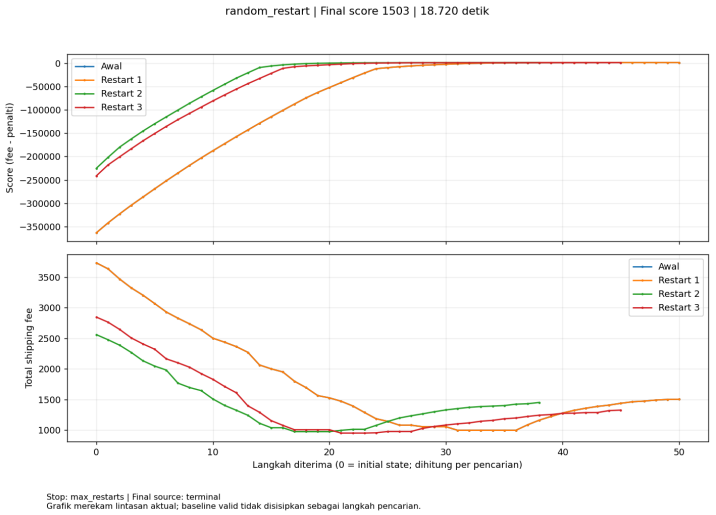
Gambar 2.31 State Awal Random Restart Percobaan 2

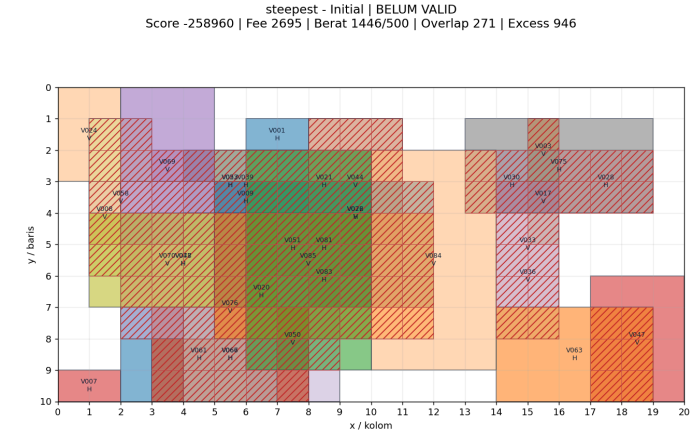
State akhir

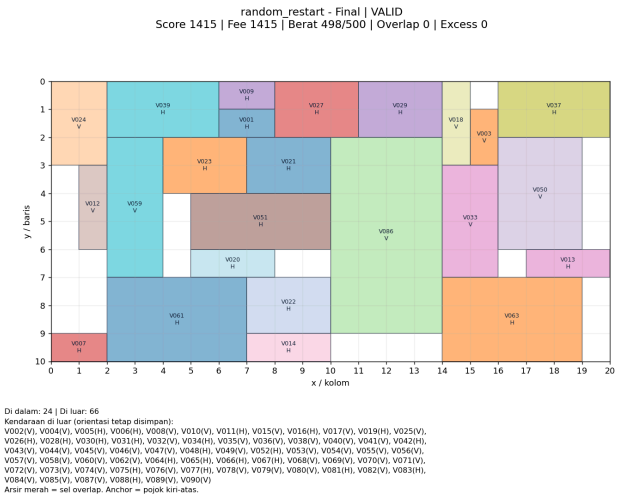
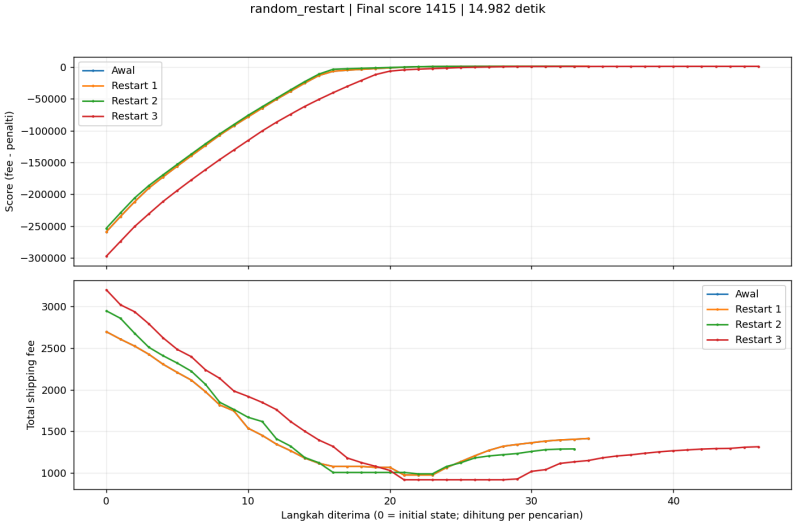


Di dalam: 27 | Di luar: 63
Kendaraan di luar (orientasi tetap disimpan):
V001(V), V002(V), V005(V), V006(H), V008(H), V010(H), V016(V), V021(V), V024(H), V026(V), V027(V), V028(H), V030(H), V032(H), V035(H), V036(H), V038(V), V039(V), V040(H), V041(H), V042(H), V043(V), V044(V), V045(V), V046(V), V047(V), V048(H), V049(V), V051(V), V052(V), V053(V), V055(V), V056(H), V057(H), V058(V), V061(V), V062(V), V063(H), V064(V), V065(V), V066(V), V067(H), V068(H), V069(H), V070(H), V071(H), V072(V), V073(V), V074(V), V075(H), V076(H), V077(H), V078(H), V079(H), V080(H), V081(V), V082(V), V083(V), V084(V), V087(H), V088(H), V089(V), V090(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

Gambar 2.32 State Akhir Random Restart Percobaan 2

Plot objective function	 **Gambar 2.33 Plot Objective Function Random Restart Percobaan 2**

Percobaan 3 (seed 303)	
State awal	 **Gambar 2.34 State Awal Random Restart Percobaan 3**

State akhir	 **Gambar 2.35 State Akhir Random Restart Percobaan 3**
Plot objective function	 **Gambar 2.36 Plot Objective Function Random Restart Percobaan 3**

Analisis Hasil Percobaan Hill Climbing

Steepest-Ascent memperoleh *score* 1.333, 1.503, dan 1.415 dengan rata-rata 1.417,00. Pencarian berhenti setelah 34 sampai 50 langkah yang diterima karena tidak ada tetangga yang lebih baik. Hasil terbaiknya diperoleh pada *seed* 202. Dari ketiga hasil akhir, berat muatan berada pada 498 sampai 499 dari kapasitas 500, sehingga kapasitas berat hampir terpakai seluruhnya.

Stochastic membutuhkan lebih banyak langkah, yaitu 136 sampai 194 iterasi, tetapi rata-rata *score*-nya hanya 1.246,00. Varian ini memilih salah satu tetangga yang lebih baik secara acak dan tidak selalu mengambil peningkatan terbesar. Oleh karena itu, lintasan pencariannya dapat masuk ke *local optimum* yang berbeda dari Steepest-Ascent. Pada ketiga *seed*, Stochastic berakhir dengan berat tepat 500/500, tetapi muatan yang penuh tidak otomatis menghasilkan total *shipping fee* tertinggi: kombinasi kendaraan yang dipilih juga menentukan nilai solusi.

Sideways memberikan rata-rata tertinggi di antara varian HC, yaitu 1.430,33. Pada *seed* 101 dan 202, hasilnya sama dengan Steepest-Ascent. Pada *seed* 303, hasilnya naik dari 1.415 menjadi 1.455. Ini menunjukkan bahwa perpindahan melalui *state* dengan nilai setara kadang dapat membuka jalur menuju perbaikan berikutnya. Akan tetapi, manfaat tersebut tidak terlihat pada setiap *seed*, sementara setiap percobaan memakai seluruh batas 20 *sideways move* dan memerlukan waktu sedikit lebih lama daripada Steepest-Ascent.

Random-Restart memperoleh hasil akhir 1.333, 1.503, dan 1.415, sama dengan Steepest-Ascent. Pada ketiga *seed*, pencarian awal menghasilkan solusi terbaik, sedangkan tiga pencarian ulang tidak memperbaikinya. Random-Restart membutuhkan total 147 sampai 183 langkah dan rata-rata waktu 17,43 detik karena empat pencarian dijalankan. Hasil ini hanya berlaku untuk tiga *seed* dan konfigurasi yang diuji, pengulangan dari titik awal lain tetap dapat bermanfaat pada percobaan berbeda.

Seluruh *final state* HC valid dan tidak memiliki tumpang tindih maupun kelebihan berat. Selisih hasil tertinggi dan terendah Steepest-Ascent, Sideways, serta Random-Restart adalah 170, sedangkan Stochastic memiliki selisih 66 dengan rata-rata hasil yang lebih rendah. Karena hanya ada tiga percobaan, angka

tersebut digunakan sebagai gambaran awal konsistensi, bukan kesimpulan statistik yang pasti.

2.7.3 Simulated Annealing

Parameter: T0 = 100, α = 0,9999, T_min = 0,1, stuck_window = 1000. Dengan parameter ini banyak iterasi setiap percobaan selalu 69.075.

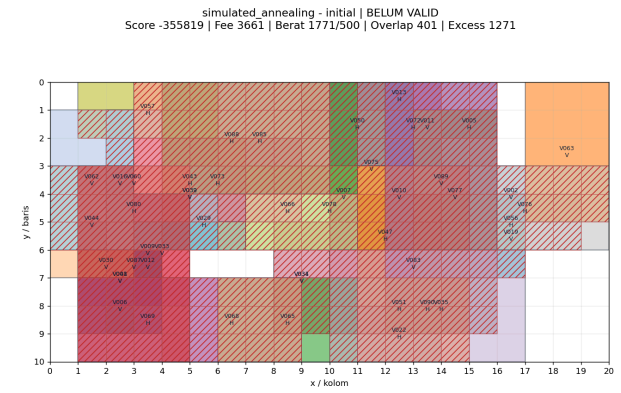
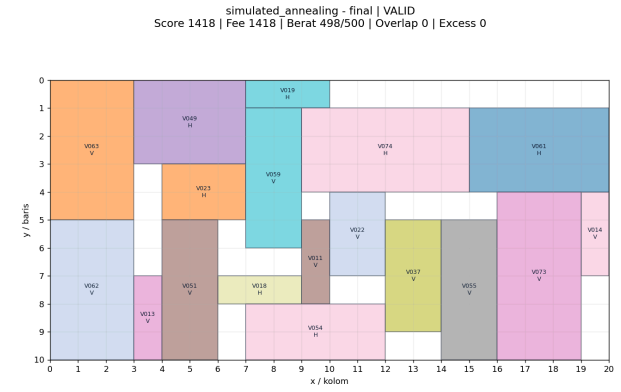
Tabel 2.7 Hasil Eksperimen Simulated Annealing

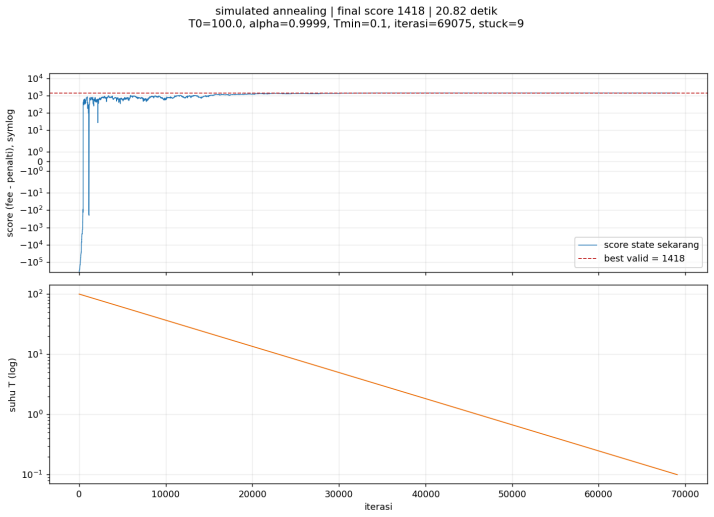
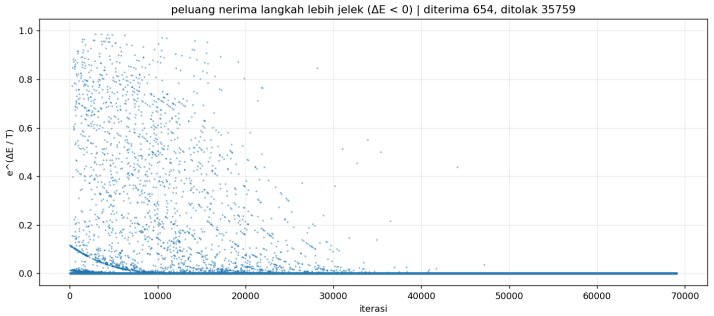
Percobaan (seed)	Score awal	Score akhir (best valid)	Score terminal	Frekuensi stuck	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	-355.819	1418	1417	9	18	498/500	20,82
2 (202)	-363.486	1398	1398	11	23	500/500	20,12
3 (303)	-258.960	1458	1458	14	23	497/500	20,19
Rata-rata	-326.088	**1424,67**	1424,33	11,33	21,33	498,33	20,38

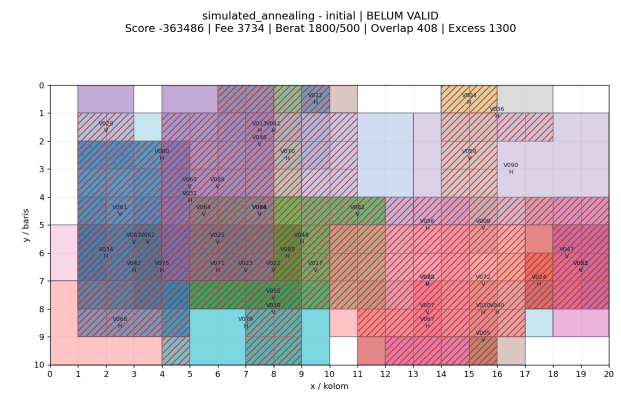
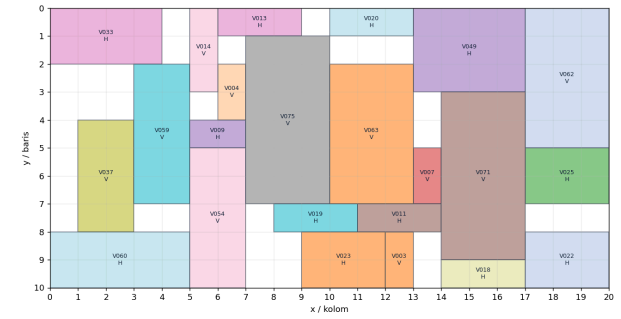
Tabel 2.8 Rincian Langkah Simulated Annealing

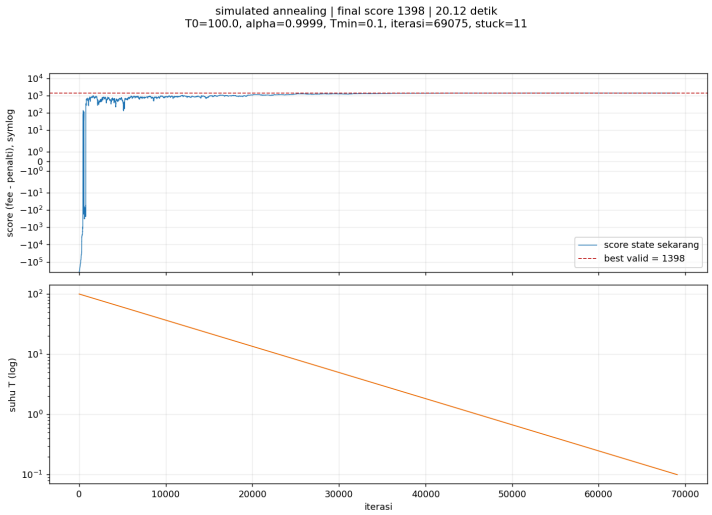
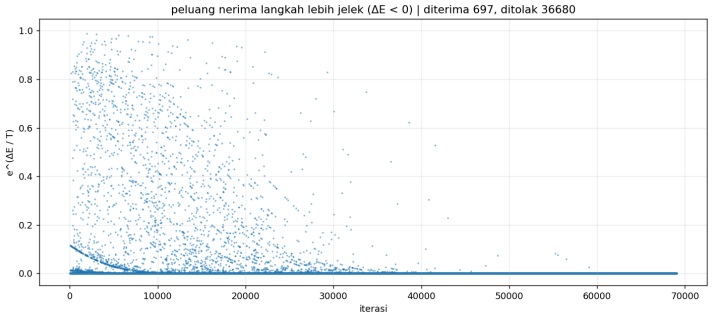
Percobaan (seed)	Lebih baik (diterima)	Sama (diterima)	Lebih buruk (diterima)	Lebih buruk (ditolak)	Iterasi valid pertama
1 (101)	723	31.939	654	35.759	sekitar 600
2 (202)	814	30.884	697	36.680	sekitar 830
3 (303)	765	30.149	688	37.473	sekitar 390

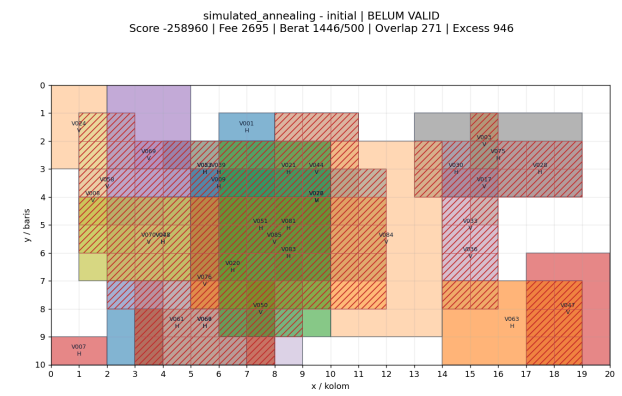
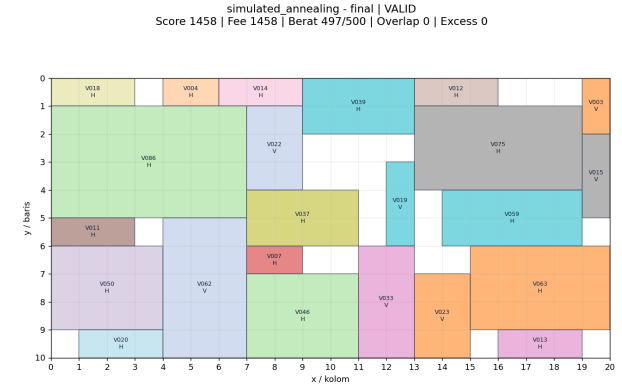
Kolom iterasi valid pertama diambil dari history yang dicatat setiap 10 iterasi, sehingga nilainya adalah perkiraan dengan ketelitian 10 iterasi.

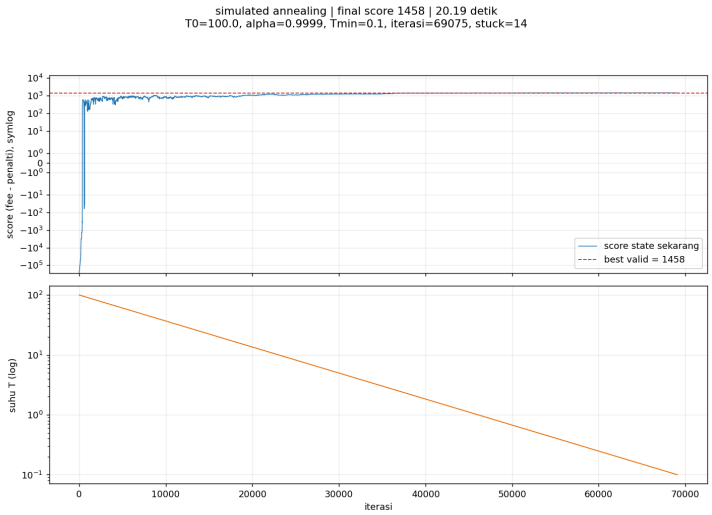
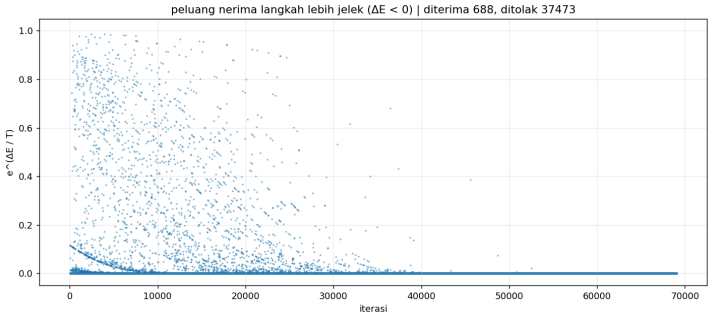
Percobaan 1 (seed 101)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(H), V003(H), V004(V), V014(V), V015(H), V017(V), V018(V), V020(H), V021(H), V023(V), V024(H), V025(V), V026(H), V027(H), V028(V), V036(H), V037(H), V038(V), V040(V), V042(H), V045(H), V046(V), V048(V), V049(H), V052(V), V053(V), V054(H), V055(V), V058(V), V059(H), V061(V), V064(H), V067(H), V070(H), V071(V), V074(V), V079(V), V081(V), V082(H), V084(V), V086(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.37 State Awal Simulated Annealing Percobaan 1**
State akhir	 Di dalam: 18 | Di luar: 72 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V003(H), V004(H), V005(V), V006(H), V007(H), V008(H), V009(V), V010(H), V012(V), V015(V), V016(H), V017(V), V020(H), V021(V), V024(H), V025(H), V026(V), V027(V), V028(V), V029(H), V030(V), V031(H), V032(V), V033(H), V034(H), V035(H), V036(H), V038(H), V039(H), V040(H), V041(H), V042(V), V043(V), V044(V), V045(H), V046(V), V047(H), V048(H), V050(V), V052(H), V053(H), V056(V), V057(H), V058(V), V060(H), V064(V), V065(V), V066(H), V067(V), V068(H), V069(H), V070(V), V071(V), V072(V), V075(H), V076(H), V077(H), V078(H), V079(V), V080(H), V081(V), V082(V), V083(V), V084(V), V085(H), V086(V), V087(V), V088(H), V089(V), V090(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.38 State Akhir Simulated Annealing Percobaan 1**

Plot objective function dan suhu terhadap iterasi	 **Gambar 2.39 Plot Objective Function Simulated Annealing Percobaan 1**
Plot e^(ΔE/T) terhadap iterasi	 **Gambar 2.40 Plot e^(ΔE/T) Simulated Annealing Percobaan 1**

Percobaan 2 (seed 202)	
State awal	 Di dalam: 49 | Di luar: 41 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V003(H), V006(H), V008(H), V010(H), V011(H), V015(V), V016(V), V018(V), V019(H), V020(H), V026(V), V027(V), V028(H), V032(H), V033(H), V035(H), V037(V), V039(V), V043(V), V044(V), V045(V), V049(V), V050(H), V051(V), V052(V), V054(H), V057(H), V059(H), V063(H), V065(V), V069(H), V074(V), V076(H), V077(H), V078(H), V083(V), V084(V), V086(V), V087(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.41 State Awal Simulated Annealing Percobaan 2**
State akhir	 Di dalam: 23 | Di luar: 67 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V005(H), V006(V), V008(V), V010(V), V012(H), V015(V), V016(H), V017(V), V021(V), V024(V), V026(H), V027(V), V028(H), V029(V), V030(V), V031(V), V032(V), V034(V), V035(H), V036(H), V038(H), V039(H), V040(H), V041(H), V042(V), V043(H), V044(H), V045(V), V046(V), V047(V), V048(H), V050(H), V051(V), V052(H), V053(V), V055(H), V056(V), V057(H), V058(V), V061(V), V064(V), V065(H), V066(H), V067(H), V068(V), V069(H), V070(V), V072(V), V073(H), V074(V), V076(H), V077(H), V078(V), V079(V), V080(V), V081(H), V082(V), V083(H), V084(V), V085(H), V086(V), V087(H), V088(V), V089(V), V090(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.42 State Akhir Simulated Annealing Percobaan 2**

Plot objective function dan suhu terhadap iterasi	 **Gambar 2.43 Plot Objective Function Simulated Annealing Percobaan 2**
Plot e^(ΔE/T) terhadap iterasi	 **Gambar 2.44 Plot e^(ΔE/T) Simulated Annealing Percobaan 2**

Percobaan 3 (seed 303)	
State awal	 Di dalam: 37 | Di luar: 53 Kendaraan di luar (orientasi tetap disimpan): V002(V), V004(V), V005(H), V006(H), V010(V), V011(H), V012(V), V013(H), V014(H), V015(V), V016(H), V018(V), V019(H), V022(H), V025(V), V027(H), V029(H), V031(H), V032(V), V034(H), V035(V), V037(H), V038(V), V040(V), V041(V), V042(H), V043(V), V045(V), V046(V), V049(V), V053(V), V054(V), V055(V), V056(V), V057(V), V059(V), V060(V), V062(V), V064(H), V065(H), V068(V), V071(V), V073(V), V074(V), V078(V), V079(V), V080(V), V082(V), V086(V), V087(V), V088(H), V089(V), V090(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.45 State Awal Simulated Annealing Percobaan 3**
State akhir	 Di dalam: 23 | Di luar: 67 Kendaraan di luar (orientasi tetap disimpan): V001(H), V002(H), V005(V), V006(V), V008(H), V009(V), V010(H), V016(V), V017(H), V021(H), V024(H), V025(V), V026(V), V027(H), V028(V), V029(V), V030(V), V031(V), V032(V), V034(H), V035(V), V036(V), V038(V), V040(V), V041(H), V042(V), V043(H), V044(H), V045(H), V047(V), V048(H), V049(H), V051(V), V052(V), V053(V), V054(H), V055(V), V056(H), V057(V), V058(V), V060(H), V061(H), V064(H), V065(V), V066(H), V067(H), V068(V), V069(H), V070(V), V071(H), V072(V), V073(V), V074(H), V076(V), V077(H), V078(H), V079(V), V080(H), V081(V), V082(V), V083(H), V084(V), V085(V), V087(V), V088(H), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.46 State Akhir Simulated Annealing Percobaan 3**

Plot objective function dan suhu terhadap iterasi	 **Gambar 2.47 Plot Objective Function Simulated Annealing Percobaan 3**
Plot e^(ΔE/T) terhadap iterasi	 **Gambar 2.48 Plot e^(ΔE/T) Simulated Annealing Percobaan 3**

Analisis Hasil Percobaan Simulated Annealing

Ketiga percobaan SA berakhir valid dengan *score* 1418, 1398, dan 1458, dengan rata-rata 1424,67. Pada percobaan 1, *score* terminal (1417) lebih kecil satu poin dari *best valid* (1418). Artinya SA sempat melewati *state* 1418, lalu menerima langkah yang sedikit lebih buruk dan tidak kembali lagi. Kasus ini menunjukkan pentingnya menyimpan *state* valid terbaik.

Dari plot *objective function*, SA sudah mencapai *state* valid pertama sekitar iterasi ke-390 sampai ke-830, jauh lebih cepat dibanding total iterasinya. Pada tahap ini SA terutama mengeluarkan kendaraan yang menyebabkan tumpang tindih dan kelebihan berat, karena penaltinya sangat besar. Setelah itu *score* naik turun di kisaran ratusan sampai sekitar 1400 selama suhu masih tinggi, lalu naik

perlahan dan baru mencapai 97% dari *score* akhirnya sekitar iterasi 31.000 sampai 39.000. Plot e^(ΔE/T) mendukung pengamatan ini. Di awal pencarian banyak titik bernilai mendekati 1, artinya langkah buruk sering diterima. Setelah iterasi 30.000, lebih dari 99,5% langkah buruk memiliki peluang di bawah 0,01, sehingga SA pada dasarnya sudah berperilaku seperti *Hill-Climbing*. Ada juga titik-titik di sekitar 0 sejak awal, yaitu langkah yang membuat tumpang tindih atau kelebihan berat. Karena penaltinya besar (215 per sel atau per unit berat), nilai ΔE untuk langkah ini sangat negatif dan peluangnya hampir 0 walaupun suhu masih tinggi.

Dari rincian langkah, hampir setengah dari seluruh iterasi (sekitar 30 ribu dari 69 ribu) adalah langkah dengan ΔE = 0. Langkah ini misalnya memutar kendaraan yang sedang di luar kapal atau menukar dua kendaraan yang sama-sama di luar. Langkah seperti ini tidak mengubah *score* sehingga SA banyak berjalan di *plateau*. Langkah buruk yang diterima hanya sekitar 650 sampai 700 kali, sedangkan yang ditolak sekitar 36 ribu kali. Frekuensi *stuck* berkisar 9 sampai 14 kali per percobaan. Kejadiannya tersebar dari sekitar iterasi 2.000 sampai 53.000, jadi SA tidak hanya tertahan di akhir pencarian, tetapi juga beberapa kali tertahan di tengah pencarian sebelum akhirnya menemukan *state* yang lebih baik lagi.

Pada *final state*, berat muatan hampir penuh (497 sampai 500 dari 500), sedangkan di geladak masih ada cukup banyak sel kosong. Ini menunjukkan bahwa pada dataset kami, batas yang paling menentukan adalah **kapasitas berat**, bukan luas geladak. Jadi solusi yang bagus adalah solusi yang memilih kendaraan dengan fee per unit berat yang tinggi. Jumlah kendaraan yang dimuat juga cukup berbeda antar percobaan (18 sampai 23) walaupun *score*-nya berdekatan, yang menunjukkan ada beberapa kombinasi kendaraan berbeda dengan total fee yang mirip.

Durasi ketiga percobaan hampir sama, yaitu 20,1 sampai 20,8 detik, karena banyak iterasinya ditentukan oleh jadwal suhu dan bukan oleh kondisi *state*. Walaupun setiap iterasi hanya mengevaluasi satu tetangga, setiap iterasi tetap memanggil evaluate secara lengkap dan menyalin *state*, sehingga 69.075 iterasi membutuhkan waktu sekitar 20 detik.

2.7.4 Genetic Algorithm

Parameter: T0 = 100, generations = 300, crossover_rate = 0,8, mutation_rate = 0,5, tournament_size = 3, elite_count = 2, repair = True.

Tabel 2.9 Hasil Eksperimen Genetic Algorithm

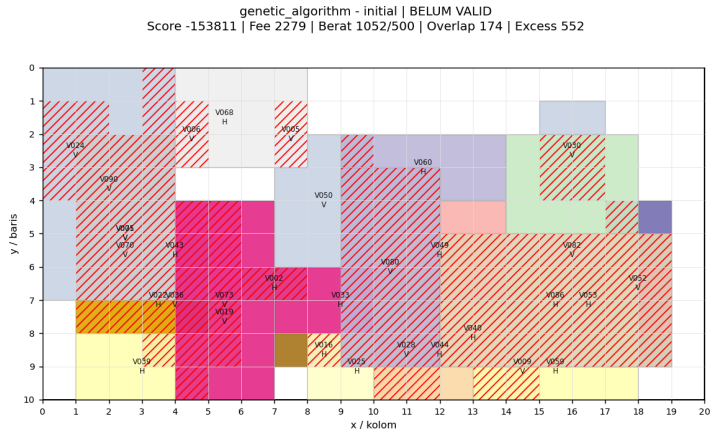
Percobaan (seed)	Score awal	Score akhir (best valid)	Score terminal	Kendaraan dimuat	Berat	Durasi (s)
1 (101)	-355.819	1335	1335	18	500/500	23,52
2 (202)	-363.486	1405	1405	20	499/500	29,27
3 (303)	-258.960	1356	1356	18	500/500	26,92
Rata-rata	-326.088	**1365,33**	1365,33	18,67	499,67/500	26,57

Tabel 2.10 Rincian Langkah Genetic Algorithm

Percobaan (seed)	Generasi valid pertama	Generasi rekor tercapai	Pembaruan rekor (kali	Individu valid di akhir	Fitness rata-rata akhir
1 (101)	1	97	21	100/100 (100%)	1317,07
2 (202)	1	298	33	100/100 (100%)	1368,60
3 (303)	1	289	32	100/100 (100%)	1329,45

Percobaan 1 (seed 101)

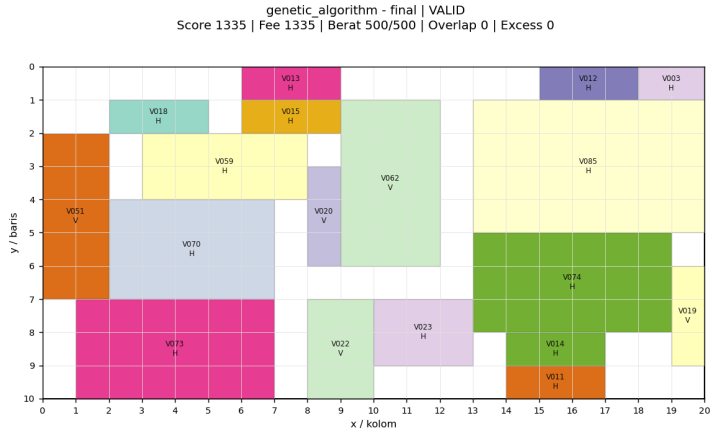
State awal



Di dalam: 32 | Di luar: 58
Kendaraan di luar (orientasi tetap disimpan):
V003(H), V004(H), V007(V), V008(H), V010(H), V011(V), V012(V), V013(V), V014(H), V015(V), V017(V), V018(V), V020(V), V021(H), V023(V), V026(V), V027(V), V029(V), V031(V), V032(V), V034(V), V035(V), V037(V), V038(V), V041(V), V042(V), V045(H), V046(V), V047(H), V048(H), V051(V), V054(V), V055(V), V056(V), V057(V), V058(V), V061(H), V062(V), V063(H), V064(H), V065(H), V066(V), V067(H), V069(V), V071(V), V072(V), V074(H), V076(V), V077(H), V078(H), V079(V), V081(H), V083(H), V084(H), V085(V), V087(H), V088(V), V089(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

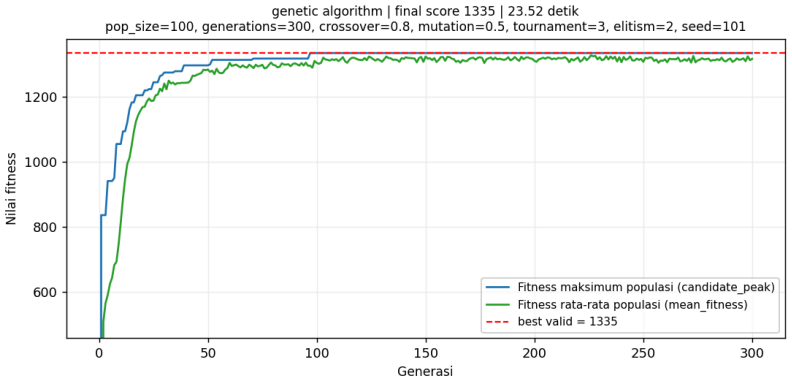
Gambar 2.49 State Awal Genetic Algorithm Percobaan 1

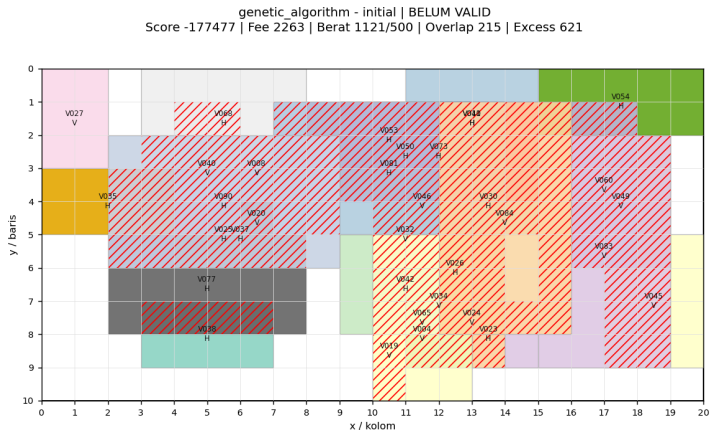
State akhir

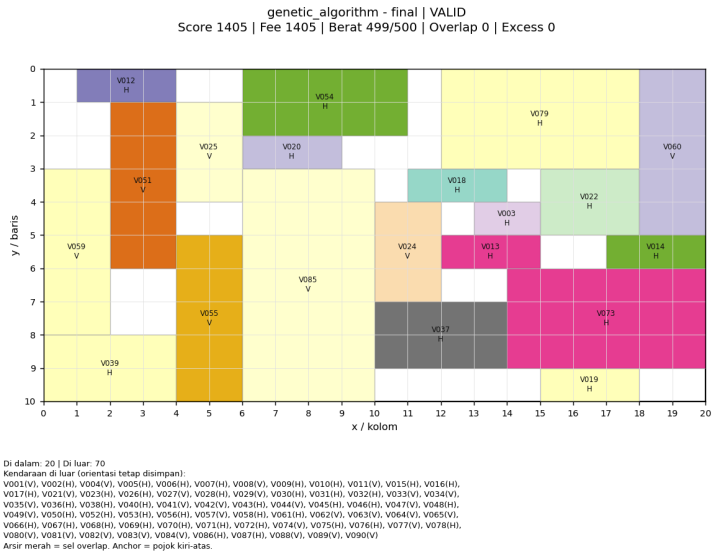
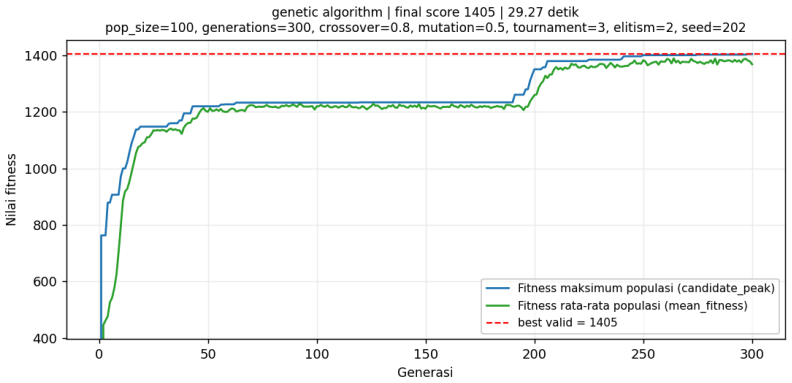


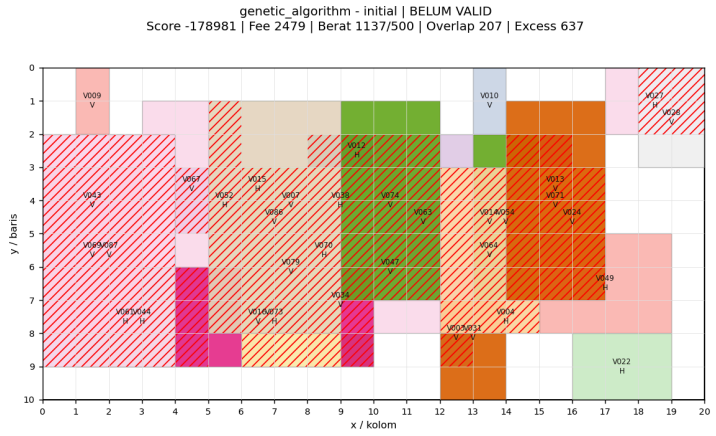
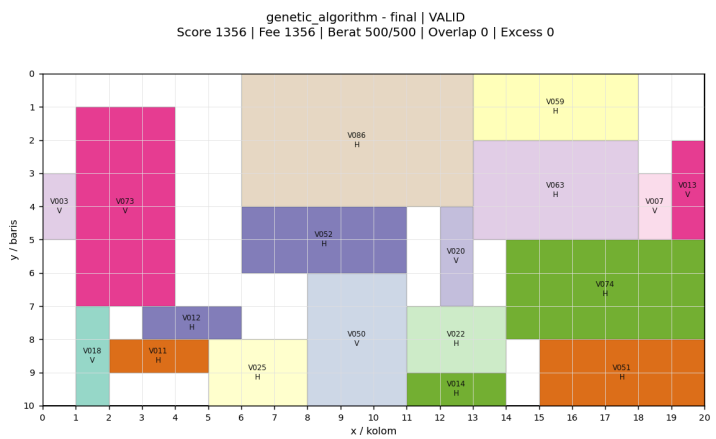
Di dalam: 18 | Di luar: 72
Kendaraan di luar (orientasi tetap disimpan):
V001(H), V002(V), V004(H), V005(V), V006(V), V007(V), V008(V), V009(H), V010(V), V016(V), V017(H), V021(V), V024(H), V025(H), V026(V), V027(V), V028(H), V029(V), V030(V), V031(H), V032(V), V033(H), V034(V), V035(V), V036(V), V037(H), V038(H), V039(H), V040(V), V041(H), V042(H), V043(H), V044(V), V045(V), V046(V), V047(H), V048(H), V049(H), V050(V), V052(V), V053(V), V054(V), V055(H), V056(H), V057(H), V058(H), V060(H), V061(H), V063(H), V064(H), V065(H), V066(H), V067(H), V068(V), V069(V), V071(V), V072(H), V075(V), V076(V), V077(V), V078(H), V079(H), V080(V), V081(H), V082(H), V083(H), V084(H), V086(V), V087(V), V088(V), V089(H), V090(H)
Arsir merah = sel overlap. Anchor = pojok kiri-atas.

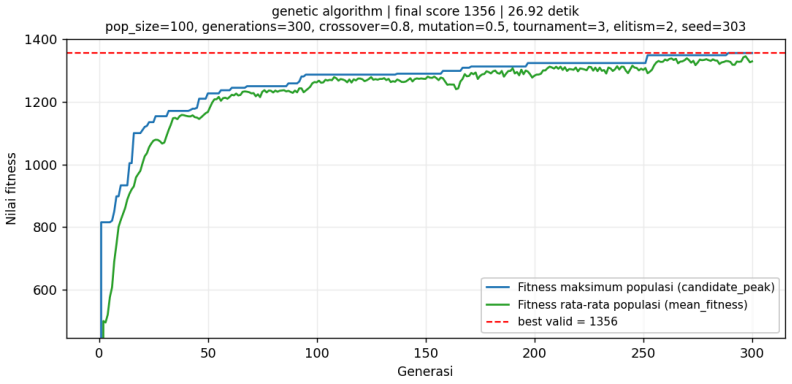
Gambar 2.50 State Akhir Genetic Algorithm Percobaan 1

Plot objective function dan suhu terhadap iterasi	 **Gambar 2.51 Plot Objective Function Genetic Algorithm Percobaan 1**

Percobaan 2 (seed 202)	
State awal	 Di dalam: 34 | Di luar: 56 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(V), V003(V), V005(V), V006(V), V007(H), V009(V), V011(H), V012(V), V013(H), V014(V), V015(V), V016(V), V017(H), V018(V), V021(H), V022(V), V028(H), V029(V), V031(H), V033(V), V036(V), V039(V), V043(H), V044(V), V047(V), V048(V), V051(V), V052(H), V055(V), V056(H), V057(H), V058(V), V059(H), V061(V), V062(V), V063(H), V064(H), V066(H), V067(H), V069(H), V070(V), V071(H), V072(V), V074(V), V075(H), V076(H), V078(H), V079(V), V080(H), V082(V), V085(H), V086(V), V087(H), V088(V), V089(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.52 State Awal Genetic Algorithm Percobaan 2**

State akhir	 **Gambar 2.53 State Akhir Genetic Algorithm Percobaan 2**
Plot objective function dan suhu terhadap iterasi	 **Gambar 2.54 Plot Objective Function Genetic Algorithm Percobaan 2**

Percobaan 3 (seed 303)	
State awal	 Di dalam: 35 | Di luar: 55 Kendaraan di luar (orientasi tetap disimpan): V001(V), V002(H), V005(V), V006(V), V008(H), V011(V), V017(V), V018(H), V019(V), V020(H), V021(H), V023(H), V025(H), V026(V), V029(V), V030(H), V032(V), V033(H), V035(V), V036(H), V037(V), V039(H), V040(V), V041(V), V042(H), V045(V), V046(H), V048(V), V050(H), V051(H), V053(V), V055(V), V056(V), V057(H), V058(H), V059(V), V060(H), V062(H), V065(V), V066(V), V068(V), V072(H), V075(H), V076(V), V077(V), V078(H), V080(H), V081(H), V082(H), V083(V), V084(V), V085(H), V088(V), V089(V), V090(H) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.55 State Awal Genetic Algorithm Percobaan 3**
State akhir	 Di dalam: 18 | Di luar: 72 Kendaraan di luar (orientasi tetap disimpan): V001(H), V002(V), V004(V), V005(V), V006(H), V008(H), V009(V), V010(H), V015(V), V016(V), V017(H), V019(H), V021(H), V023(H), V024(V), V026(H), V027(H), V028(H), V029(H), V030(V), V031(H), V032(H), V033(V), V034(H), V035(V), V036(H), V037(V), V038(V), V039(H), V040(H), V041(H), V042(V), V043(V), V044(V), V045(H), V046(H), V047(H), V048(V), V049(V), V053(V), V054(H), V055(V), V056(V), V057(V), V058(H), V060(H), V061(V), V062(V), V064(V), V065(V), V066(H), V067(V), V068(H), V069(H), V070(H), V071(H), V072(H), V075(H), V076(H), V077(H), V078(H), V079(V), V080(V), V081(H), V082(H), V083(H), V084(V), V085(H), V087(H), V088(H), V089(H), V090(V) Arsir merah = sel overlap. Anchor = pojok kiri-atas. **Gambar 2.56 State Akhir Genetic Algorithm Percobaan 3**

Plot objective function dan suhu terhadap iterasi	

Gambar 2.57 Plot Objective Function Genetic Algorithm Percobaan 3

Analisis Hasil Percobaan Simulated Annealing

Ketiga percobaan Genetic Algorithm dengan repair berakhir valid dengan score 1335, 1405, dan 1356. Seluruh percobaan menggunakan populasi 100 individu, 300 generasi, peluang crossover 0,8, peluang mutasi 0,5, ukuran turnamen 3, dan dua elite. Selisih antara hasil tertinggi dan terendah adalah 70 poin. Perbedaan ini menunjukkan bahwa meskipun parameternya sama, pengacakan populasi awal, pemilihan induk, crossover, dan mutasi dapat membawa pencarian menuju kombinasi muatan yang berbeda.

Pada generasi 0, belum ada individu yang valid dalam ketiga populasi. Fitness maksimum masih berada di range minus yang sangat negatif, yaitu -153.811, -177.477, dan -178.981. Setelah satu generasi, jumlah individu valid mencapai 98 dari 100 individu. Hal tersebut sesuai dengan pembentukan 98 anak yang menjalani repair, sedangkan dua elite dari populasi awal tetap dipertahankan tanpa perbaikan. Pada generasi 2, seluruh populasi sudah valid dan kondisi ini bertahan hingga akhir pencarian. Dengan itu, setelah semua tahap, kenaikan fitness mencerminkan peningkatan total tarif yang didapatkan.

Melalui riwayat fitness, perkembangan ketiga percobaan memiliki pola yang berbeda. Percobaan 1 sudah mencapai sekitar 97% dari hasil akhirnya pada generasi 39, lalu memperoleh score tertinggi 1335 pada generasi 97. Setelah itu, score tertinggi tidak bertambah hingga generasi 300. Sebaliknya, pada percobaan kedua dan ketiga mencapai sekitar 97% pada generasi 206 dan 197, serta

mendapatkan hasil tertinggi pada generasi 298 dan 289. Artinya, tambahan generasi bukan berarti merupakan opsi terbaik.

Periode tanpa peningkatan juga tidak berarti pencarian sudah selesai. Pada percobaan kedua, fitness maksimum bertahan di 1234 dari generasi 120 sampai 190, tetapi setelahnya meningkat drastis hingga 1405. Hal yang sama terjadi di percobaan ketiga, yang awalnya nyangkut pada angka 1324, tetapi naik di akhir menjadi 1356. Hasil ini menunjukkan bahwa penghentian hanya berdasarkan beberapa generasi tanpa peningkatan dapat melewatkan perbaikan yang muncul kembali.

Fitness maksimum tidak menurun pada seluruh percobaan karena dua elite dipertahankan pada setiap generasi. Sementara itu, rata-rata fitness masih berfluktuasi karena anak hasil crossover, mutasi, dan repair belum tentu lebih baik daripada induknya. Pada generasi terakhir, rata-rata fitness masing-masing populasi adalah 1317,07, 1368,60, dan 1329,45, dibandingkan fitness maksimum 1335, 1405, dan 1356. Selisih yang relatif kecil menunjukkan bahwa kualitas sebagian besar populasi sudah mendekati individu terbaik. Namun, kedekatan nilai fitness saja belum membuktikan bahwa seluruh individu memiliki susunan kendaraan yang sama.

Durasi ketiga percobaan adalah 23,52, 29,27, dan 26,92 detik. Dari kode, pekerjaan repair bergantung pada konflik dan kelebihan berat setiap anak, sedangkan pembangkitan neighbor dapat mengulang percobaan ketika perubahan tidak dapat dilakukan. Perbedaan kondisi komputasi juga dapat memengaruhi durasi. Karena itu, jumlah generasi yang sama menghasilkan batas pencarian yang sama, tetapi tidak menjamin waktu eksekusi identik.

Di luar ketiga percobaan tersebut, run tambahan dengan seed 0 menghasilkan solusi maksimal di 1438, dengan 21 kendaraan, berat 496, dan penggunaan geladak 165 dari 200 sel. Percobaan ini juga merupakan hasil fluktuasi setelah stagnasi yang lumayan lama. Hasil ini memperlihatkan bahwa pencarian masih dapat menemukan perbaikan setelah periode panjang tanpa kenaikan fitness.

2.8 Bonus: Objective Function Alternatif Berbasis ETA

2.8.1 Deskripsi

Pada bagian bonus, kami mengembangkan dua perluasan dari persoalan utama. Pertama, sistem dapat menggunakan *objective function* alternatif yang mempertimbangkan ETA kendaraan. Kedua, penempatan kendaraan tidak lagi terbatas pada satu kapal, tetapi dapat dilakukan pada armada yang berisi beberapa kapal. Kedua perluasan ini dapat digunakan bersama pada seluruh algoritma yang diimplementasikan, yaitu empat varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm.

Objective utama, shipping_fee, memberikan nilai kendaraan sebesar biaya pengirimannya. Pada objective alternatif, eta_priority, kendaraan dengan ETA lebih kecil memperoleh nilai lebih tinggi karena dianggap lebih mendesak. Untuk kendaraan (i), nilai yang digunakan adalah:

v_i^ETA = fee_i (1 + (ETA_max - ETA_i) / ETA_max)

Pada dataset yang digunakan, (ETA_max=14). Dengan demikian, kendaraan dengan ETA 0 memperoleh bobot 2, sedangkan kendaraan dengan ETA 14 memperoleh bobot 1. Apabila seluruh ETA bernilai 0, program menggunakan nilai *shipping fee* secara langsung agar tidak terjadi pembagian dengan nol. Objective ini tetap memakai atribut kendaraan yang telah tersedia; tidak ada atribut baru yang ditambahkan.

Pada perluasan armada, setiap kendaraan yang dimuat mempunyai posisi, orientasi, dan ID kapal tempatnya berada. Kendaraan yang tidak dimuat memiliki posisi dan ID kapal bernilai none. Setiap kapal dievaluasi berdasarkan ukuran geladak, kapasitas berat, dan tumpang tindihnya masing-masing. Nilai solusi merupakan jumlah nilai seluruh kapal:

f(s) = jumlah untuk setiap k dalam K dari (V_k - P, [O_k + E_k])

dengan (V_k) adalah jumlah nilai kendaraan pada kapal (k) menurut objective aktif, (O_k) adalah jumlah sel yang bertumpang tindih, (E_k) adalah kelebihan berat, dan (P=1+\max_i(v_i)) merupakan koefisien penalti. Selain nilai objective, program tetap menyimpan total *shipping fee* asli agar hasilnya dapat

diinterpretasikan secara terpisah. Solusi dinyatakan valid apabila setiap kapal tidak mengalami tumpang tindih maupun kelebihan kapasitas.

Konfigurasi armada pada eksperimen berisi dua kapal: S1 berukuran 10 × 20 dengan kapasitas berat 500, dan S2 berukuran 8 × 15 dengan kapasitas berat 350. Struktur masukan armada berupa daftar kapal, sehingga implementasinya tidak dibatasi khusus untuk dua kapal.

2.8.2 Source Code

Source code perhitungan nilai kendaraan pada kedua objective (src/Core/objectives.py)

```
def vehicle_values(vehicles, objective="shipping_fee"):
 objective = normalize_objective(objective)
 if objective == "shipping_fee":
 return {vid: v["shipping_fee"] for vid, v in
vehicles.items()}
 for vid, vehicle in vehicles.items():
 if type(vehicle.get("eta")) is not int or vehicle["eta"] <
0:
 raise ValueError(f"{vid}.eta harus integer >= 0 untuk
eta_priority")
 eta_max = max((v["eta"] for v in vehicles.values()),
default=0)
 if eta_max == 0:
 return {vid: v["shipping_fee"] for vid, v in
vehicles.items()}
 # Rumus bonus branch tim: urgensi berada pada rentang [1,
2].
 return {
 vid: v["shipping_fee"] * (1 + (eta_max - v["eta"]) /
eta_max)
 for vid, v in vehicles.items()
 }
```

Source code menerima konfigurasi satu kapal maupun daftar kapal, lalu memvalidasi ID, dimensi, dan kapasitas setiap kapal. (src/Core/fleet.py)

```
def ships_by_id(ship):
 """Legacy ship memakai ID internal 'ship_1'; fleet memakai
ID dari input."""
 if not isinstance(ship, dict):
 raise ValueError("Konfigurasi kapal harus object")
 rows = ship["ships"] if is_fleet(ship) else [ship]
 if not isinstance(rows, list) or not rows:
 raise ValueError("ships harus list tidak kosong")
 result = {}
 for row in rows:
 if not isinstance(row, dict):
 raise ValueError("Setiap kapal harus object")
 sid = row.get("id") if is_fleet(ship) else "ship_1"
 if not isinstance(sid, str) or not sid.strip() or sid in
result:
 raise ValueError("ID kapal harus string nonkosong dan unik")
 for field in ("w", "l", "max_capacity"):
 if type(row.get(field)) is not int or row[field] < 1:
 raise ValueError(f"{sid}.{field} harus integer >= 1")
 result[sid] = row
 return result
```

Source code perhitungan score dan validitas pada setiap kapal (src/Core/evaluation.py)

```
for metrics in per_ship.values():
 metrics["objective_value"] =
round(metrics["objective_value"], 10)
 metrics["overlap"] = sum(max(0, n - 1) for row in
metrics["grid"] for n in row)
 metrics["excess_weight"] = max(0, metrics["total_weight"] -
metrics["max_capacity"])
 metrics["score"] = round(metrics["objective_value"] -
```

```
penalty * (
 metrics["overlap"] + metrics["excess_weight"]), 10)
 metrics["is_valid"] = metrics["overlap"] == 0 and
metrics["excess_weight"] == 0
```

2.8.3 Penjelasan

Berbeda dari implementasi lama yang menggunakan fungsi evaluasi ETA terpisah, implementasi saat ini memakai satu fungsi evaluate() dengan parameter objective. Nilai kendaraan dihitung oleh vehicle_values(), kemudian evaluasi menghitung nilai objective, *shipping fee* asli, berat, tumpang tindih, dan penalti untuk setiap kapal. Hal ini membuat aturan penilaian konsisten pada Hill Climbing, Simulated Annealing, dan Genetic Algorithm.

Dalam konfigurasi armada, batas berat dan tumpang tindih diperiksa secara terpisah pada tiap kapal, bukan hanya terhadap kapasitas gabungan. Karena itu, total berat yang masih di bawah jumlah kapasitas armada belum cukup untuk menyatakan suatu solusi valid; distribusi berat dan penempatan pada masing-masing kapal juga harus memenuhi batasannya. Penalti membantu mengarahkan pencarian menjauhi solusi yang melanggar batasan, sementara hasil akhir dipilih dari solusi valid yang ditemukan oleh algoritma.

Untuk menjalankan eksperimen bonus, program menerima opsi --fleet dan --objective eta_priority. Opsi objective tersebut tersedia untuk seluruh algoritma, bukan hanya Simulated Annealing. Konfigurasi --algorithm all menjalankan keenam algoritma pada setiap seed dan menyimpan ringkasan dalam summary.csv, rincian pada result.json, serta gambar state dan riwayat pencarian.

2.8.4 Hasil Eksperimen

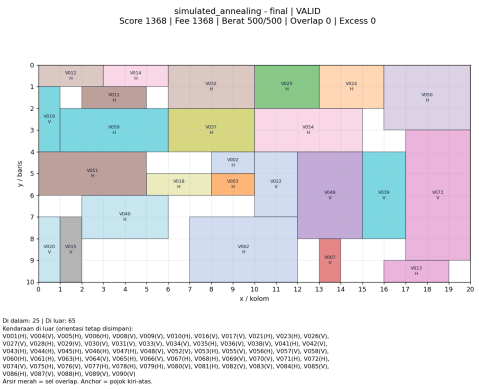
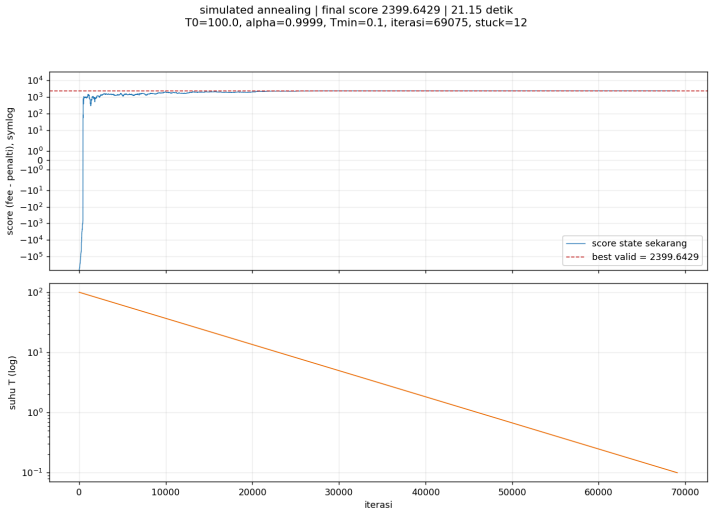
Eksperimen bonus menggunakan dataset 90 kendaraan yang sama, armada S1 dan S2, objective eta_priority, serta tiga seed: 101, 202, dan 303. Enam algoritma dijalankan untuk setiap seed sehingga terdapat 18 hasil. Tabel berikut merangkum *score* akhir menurut objective ETA, rata-rata *shipping fee* asli, dan rata-rata durasi. Seluruh solusi akhir pada 18 percobaan berstatus valid.

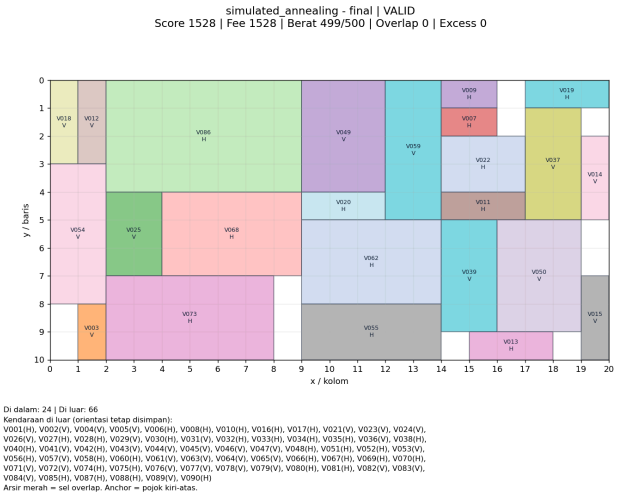
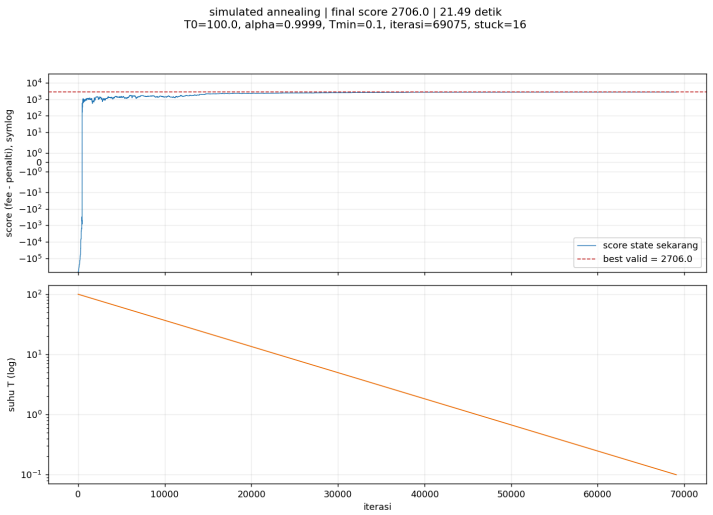
Tabel 2.11 Hasil Simulated Annealing dengan Objective ETA

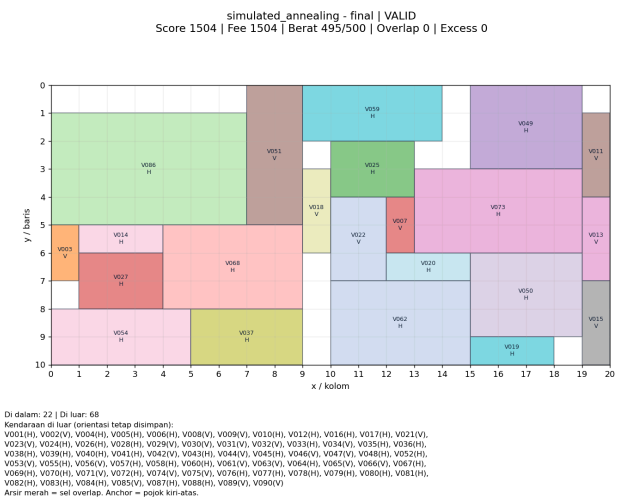
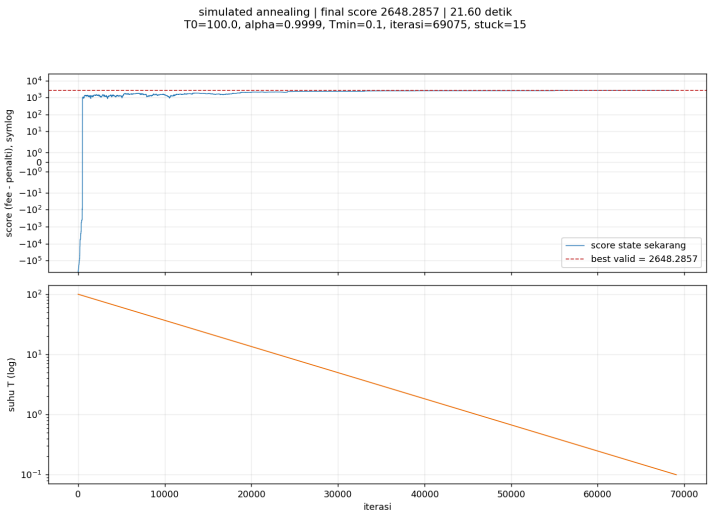
Percobaan (seed)	Score ETA seed 101	Seed 202	Seed 303	Rata-rata score ETA	Rata-rata fee asli	Rata-rata Durasi (s)
HC Steepest	3248,07	3401,36	3290,21	3313,21	2010,00	9,05
HC Stochastic	3058,50	3356,57	3078,93	3164,67	1920,67	32,18
HC Sideways	3305,50	3401,36	3362,21	3356,36	2024,33	13,91
HC Random Restart	3301,86	3428,29	3430,21	3386,79	2027,67	37,46
Simulated Annealing	4015,71	3946,29	4040,79	4000,93	2342,00	78,65
Genetic Algorithm	3157,07	3168,43	3194,86	3173,45	1889,67	37,15

Tabel 2.12 Perbandingan Objective Fee dan Objective ETA

Percobaan (seed)	Fee asli (obj. fee)	Fee asli (obj. ETA)	Rata-rata ETA (obj. fee)	Rata-rata ETA (obj. ETA)	Kendaraan (fee / ETA)
1 (101)	1418	1368	4,33 hari	3,96 hari	18 / 25
2 (202)	1398	1528	5,96 hari	3,88 hari	23 / 24
3 (303)	1458	1504	5,96 hari	3,82 hari	23 / 22

Percobaan 1 (seed 101), objective ETA	
State akhir	 **Gambar 2.58 State Akhir Objective ETA Percobaan 1**
Plot objective function dan suhu terhadap iterasi	 **Gambar 2.59 Plot Objective Function Objective ETA Percobaan 1**

Percobaan 2 (seed 202), objective ETA	
State akhir	 **Gambar 2.60 State Akhir Objective ETA Percobaan 2**
Plot objective function dan suhu terhadap iterasi	 **Gambar 2.61 Plot Objective Function Objective ETA Percobaan 2**

Percobaan 3 (seed 303), objective ETA	
State akhir	 **Gambar 2.62 State Akhir Objective ETA Percobaan 3**
Plot objective function dan suhu terhadap iterasi	 **Gambar 2.63 Plot Objective Function Objective ETA Percobaan 3**

Sebagai catatan, judul gambar *state* akhir menampilkan *score* dengan *objective function* utama (sama dengan fee asli karena *state*-nya valid), sedangkan *score* ETA ditampilkan pada judul plot *objective function*. *State* awal tidak ditampilkan ulang karena sama dengan *state* awal pada subbab 2.7.3.

2.8.5 Analisis

Dalam tiga seed yang diuji, Simulated Annealing memperoleh rata-rata *score* ETA tertinggi, yaitu 4000,93, sekaligus rata-rata *shipping fee* asli tertinggi, yaitu 2342. Namun, durasi rata-ratanya juga paling lama, 78,65 detik. HC Steepest memiliki durasi rata-rata paling singkat, 9,05 detik, dengan rata-rata *score* 3313,21. Hasil ini menunjukkan perbedaan kualitas solusi dan waktu komputasi pada konfigurasi eksperimen tersebut; tiga seed belum cukup untuk menyimpulkan keunggulan algoritma secara umum.

Nilai *score* ETA tidak sama dengan pendapatan karena setiap kendaraan memperoleh bobot menurut ETA. Pada seluruh solusi akhir yang valid, tidak terdapat penalti sehingga *score* sama dengan jumlah nilai kendaraan berbobot ETA. *Shipping fee* asli tetap dilaporkan sebagai metrik terpisah. Oleh sebab itu, *score* ETA tidak boleh dibandingkan langsung dengan *score* eksperimen satu kapal yang menggunakan objective shipping_fee.

Contoh HC Random Restart seed 303 memperlihatkan bahwa total berat 846 masih berada di bawah kapasitas gabungan 850, tetapi pemeriksaan validitas tetap harus dilakukan per kapal. S1 menampung berat 499 dari batas 500, sedangkan S2 menampung berat 347 dari batas 350; keduanya tidak memiliki sel yang bertumpang tindih. Hasil ini menunjukkan bahwa implementasi armada tidak sekadar menjumlahkan kapasitas kapal, melainkan benar-benar mengevaluasi susunan masing-masing geladak.

BAB III
ANALISIS

3.1 Kedekatan dengan Global Optimum

Global optimum persoalan ini tidak bisa diketahui secara pasti karena ruang pencariannya terlalu besar. Sebagai pembanding, kami menghitung **batas atas** (*upper bound*) dengan *dynamic programming knapsack* dua dimensi, yaitu memaksimalkan total fee dengan syarat total berat ≤ 500 dan total luas ≤ 200 sel. Perhitungan ini mengabaikan bentuk kendaraan dan susunannya di grid, sehingga tidak ada susunan nyata yang bisa melebihi nilainya. Hasilnya adalah **1673**. Kalau hanya batas berat yang dipakai, batas atasnya 1989, sehingga batas luas geladak tetap ikut membatasi.

Selain batas atas, kami juga punya batas bawah dari hasil bonus pada subbab 2.8, yaitu susunan valid dengan fee asli 1528. Jadi *global optimum* yang sebenarnya pasti berada di antara **1528 dan 1673**.

Tabel 3.1 Perbandingan Hasil Terhadap Upper Bound (1673)

Algoritma	Score terbaik	Rata-rata
Random Restart Hill-Climbing	1503	1417,00
Hill-Climbing with Sideways Move	1503	1430,33
Genetic Algorithms	1253	1231,33
Steepest Ascent Hill-Climbing	1503	1417,00
Stochastic Hill-Climbing	1277	1246,00
Simulated Annealing	1458	1424,67

Hill Climbing Sideways menghasilkan rata-rata tertinggi di antara algoritma yang dibandingkan, yaitu 1430,33 atau 85,50% dari batas atas. Hasil terbaiknya, 1503, mencapai 89,84% dari batas atas dan menjadi batas bawah terbaik yang ditemukan dalam eksperimen utama. Namun, jarak 170 poin terhadap batas atas belum dapat diartikan sebagai jarak sebenarnya terhadap global optimum, sebab batas atas tersebut mungkin tidak dapat diwujudkan sebagai susunan kendaraan pada geladak.

Hill Climbing Steepest dan Hill Climbing Random Restart juga mencapai score terbaik 1503, tetapi rata-ratanya lebih rendah daripada Hill Climbing Sideways. Pada tiga seed yang diuji, Random Restart tidak menemukan solusi akhir yang lebih baik daripada Steepest meskipun menjalankan pencarian tambahan. Hill Climbing Stochastic memiliki score terbaik 1277 dan rata-rata 1246, paling rendah di antara varian HC. Pemilihan tetangga secara acak membuat jalur pencariannya berbeda, tetapi pada percobaan ini tidak menghasilkan solusi yang lebih baik daripada pemeriksaan tetangga secara menyeluruh.

Simulated Annealing mencapai sekitar 85,16% dari batas atas secara rata-rata. Hasil terbaiknya, 1458, masih lebih rendah daripada susunan valid bernilai 1503. Artinya, SA belum mencapai *global optimum*. Berdasarkan rincian langkah Simulated Annealing, sekitar 30 ribu dari 69.075 iterasinya memiliki $\Delta E = 0$ sehingga banyak langkah tidak mengubah score. Selain itu, ketika suhu semakin rendah, peluang menerima langkah yang lebih buruk menjadi sangat kecil. Kedua hal tersebut dapat membatasi kesempatan SA untuk berpindah ke bagian ruang solusi yang lebih baik.

Genetic Algorithm pada konfigurasi pembanding mencapai rata-rata 1231,33 atau 73,60% dari batas atas, dengan hasil terbaik 1253. Karena hasil terbaiknya juga berada di bawah solusi valid 1503, konfigurasi Genetic Algorithm Ini belum mencapai *global optimum*. Hasil percobaan variasi parameter pada subbab 3.5 perlu dibedakan dari konfigurasi pembanding ini, peningkatan jumlah populasi atau generasi dapat mengubah kualitas hasil, tetapi tidak mengubah angka Genetic Algorithm Pada Tabel 3.1 dan 3.2.

3.2 Perbandingan Hasil Antar Algoritma

Dilihat dari rata-rata tiga percobaan, urutan hasil tertinggi adalah Hill Climbing Sideways (1430,33), Simulated Annealing (1424,67), Hill Climbing Steepest dan Hill Climbing Random Restart (masing-masing 1417,00), Hill Climbing Stochastic (1246,00), lalu Genetic Algorithm (1231,33). Selisih rata-rata Hill Climbing Sideways dan Simulated Annealing hanya 5,66 poin. Oleh karena itu, hasil tiga seed ini menunjukkan keduanya berkinerja berdekatan, bukan membuktikan salah satunya selalu lebih unggul.

Perbandingan per seed memperlihatkan bahwa peringkat algoritma dapat berubah. Pada seed 101, SA memperoleh 1418, lebih tinggi daripada seluruh varian Hill Climbing dan Genetic Algorithm. Pada seed 202, Hill Climbing Steepest, Sideways, dan Random Restart sama-sama mencapai 1503, melampaui Simulated Annealing yang memperoleh 1398. Pada seed 303, Simulated Annealing memperoleh 1458, hanya tiga poin di atas Hill Climbing Sideways yang mencapai 1455. Dengan demikian, kesimpulan berdasarkan satu percobaan saja dapat berbeda dari kesimpulan berdasarkan rata-rata.

Perbedaan antarvarian Hill Climbing juga memberikan informasi mengenai strategi pencarian. Sideways menghasilkan score yang sama dengan Steepest pada seed 101 dan 202, tetapi meningkat dari 1415 menjadi 1455 pada seed 303. Ini menunjukkan bahwa langkah dengan score setara dapat membantu pencarian pada keadaan tertentu, walaupun tidak selalu memberi perbaikan. Random Restart menghasilkan score akhir yang sama dengan Steepest pada ketiga seed. Tambahan titik awal pada konfigurasi ini belum berhasil meningkatkan hasil terbaiknya.

Genetic Algorithm memperoleh hasil yang lebih rendah pada konfigurasi pembanding, tetapi mekanismenya berbeda dari pencarian satu state pada Hill Climbing dan Simulated Annealing. Genetic Algorithm mempertahankan populasi dan membentuk solusi baru melalui seleksi, *crossover*, serta mutasi. Hasil pada Tabel 3.1 hanya menggambarkan konfigurasi 100 individu dan 300 generasi, sehingga pengaruh jumlah populasi dan generasi dibahas lebih lanjut pada subbab 3.5. Seluruh score yang dibandingkan pada subbab ini berasal dari solusi akhir valid, sehingga pada objective `shipping_fee` nilainya sama dengan total biaya pengiriman kendaraan yang dimuat

3.3 Perbandingan Durasi

Berdasarkan tabel Hill Climbing Steepest mempunyai rata-rata durasi terpendek, yaitu 4,54 detik. Hill Climbing Sideways membutuhkan 6,55 detik karena dapat melanjutkan pencarian melalui langkah dengan score setara. Hill Climbing Stochastic membutuhkan 17,74 detik, sedangkan Hill Climbing Random Restart membutuhkan 17,43 detik karena menjalankan beberapa pencarian dari

state awal yang berbeda. Pada tiga seed ini, waktu tambahan Random Restart tidak disertai peningkatan score dibandingkan Steepest.

Simulated Annealing membutuhkan rata-rata 20,38 detik. Ketiga percobaannya sama-sama menjalankan 69.075 iterasi sesuai jadwal suhu, sehingga durasinya relatif berdekatan. Genetic Algorithm dengan 100 individu dan 300 generasi membutuhkan rata-rata 28,94 detik. Jumlah populasi dan generasi menentukan banyaknya individu yang perlu dibentuk dan dievaluasi, sehingga peningkatan kedua parameter tersebut dapat menambah durasi.

Angka durasi perlu ditafsirkan secara hati-hati karena eksperimen algoritma dijalankan pada perangkat yang berbeda. Perbedaan perangkat dan kondisi eksekusi dapat memengaruhi waktu yang tercatat. Karena itu, tabel durasi menunjukkan biaya komputasi yang teramati pada eksperimen ini, tetapi belum cukup untuk menyatakan perbandingan kecepatan antarmetode secara adil. Untuk pengukuran yang lebih kuat, seluruh algoritma perlu dijalankan ulang pada perangkat dan lingkungan yang sama.

3.4 Konsistensi Hasil

Konsistensi awal diukur melalui selisih score tertinggi dan terendah pada tiga seed. SA memiliki selisih 60 poin, dari 1398 sampai 1458, atau sekitar 4,21% dari rata-ratanya. GA memiliki selisih 63 poin, dari 1190 sampai 1253, atau sekitar 5,12% dari rata-ratanya. HC Stochastic memiliki selisih 66 poin, sedangkan HC Steepest, Sideways, dan Random Restart masing-masing memiliki selisih 170 poin.

Rentang yang lebih besar pada HC Steepest, Sideways, dan Random Restart terutama dipengaruhi hasil seed 202 yang mencapai 1503, sementara hasil seed lainnya lebih rendah. Hal ini sesuai dengan sifat HC yang bergerak berdasarkan lingkungan state saat ini: titik awal berbeda dapat membawa pencarian ke *local optimum* yang berbeda. Sideways dapat memberi perbaikan pada seed tertentu, tetapi belum membuat hasil antarseed menjadi seragam. Random Restart pada konfigurasi yang diuji juga belum mengurangi rentang hasil dibandingkan Steepest.

Rentang SA lebih kecil daripada ketiga varian HC tersebut. Penerimaan langkah yang lebih buruk ketika suhu masih cukup tinggi memberi SA kesempatan berpindah dari jalur pencarian awalnya, walaupun data tiga seed tidak cukup untuk memastikan bahwa mekanisme ini selalu menghasilkan variasi yang lebih rendah. GA juga menunjukkan rentang relatif kecil, tetapi rentang kecil tidak sama dengan kualitas solusi tinggi: rata-rata GA tetap lebih rendah daripada HC Sideways dan SA. Oleh karena itu, konsistensi perlu dinilai bersama kualitas score, bukan secara terpisah. Karena setiap algoritma baru diuji pada tiga seed, hasil ini merupakan pengamatan awal, bukan ukuran statistik yang kuat.

3.5 Pengaruh Banyak Iterasi dan Jumlah Populasi pada Genetic Algorithm

Berdasarkan percobaan yang dilakukan, jumlah generasi dan ukuran populasi mempengaruhi kualitas solusi akhir dari Genetic Algorithm. Tetapi, peningkatannya tidak selalu memberikan manfaat yang sama besar. Pada pengujian GA tanpa repair dengan populasi tetap 100 individu, penambahan generasi dari 100 menjadi 300 meningkatkan rata-rata score dari 1157,67 menjadi 1312,33, atau sekitar 13,36%. Ketika jumlah generasi ditambah menjadi 500, rata-ratanya kembali meningkat menjadi 1333,67, tetapi kebaikannya hanya sekitar 1,63%. Artinya, tambahan generasi masih membantu pencarian menemukan kombinasi yang lebih baik, tetapi peningkatannya mulai melambat.

Pengaruh jumlah generasi juga terlihat apda riwayat tiga percobaan utama Genetic Algorithm dengan repair. Pada percobaan pertama, fitness maksimum sudah mencapai 1335 pada generasi 97, darisitu tidak kunjung membaik. Sedangkan di kedua percobaan lainnya, hasil terbaik ada pada generasi akhir-akhir, sekitar 280 ke atas. Dengan demikian, grafik yang mendatar selama beberapa generasi belum tentu menunjukkan bahwa pencarian tidak dapat membaik lagi. Namun, penambahan generasi juga tidak akan menjamin peningkatan.

Ukuran populasi menunjukkan pola yang serupa. Pada pengujian awal tanpa repair dengan 300 generasi, peningkatan populasi dari 50 menjadi 100 individu menaikkan rata-rata score dari 1225,67 menjadi 1312,33, atau sekitar 7,07%. Akan tetapi, ketika populasi diperbesar menjadi 200 individu, rata-ratanya menjadi 1304,33, sedikit lebih rendah daripada populasi 100. Memang populasi

yang lebih besar menyediakan banyak alternatif, tetapi setiap alternatif harus melalui banyak proses. Karena itu, memperbesar populasi tidak otomatis menghasilkan solusi yang lebih baik, sementara jumlah individu yang harus diproses pada setiap generasi terus bertambah.

Dari grafik tiga percobaan utama, rata-rata fitness cenderung mendekati fitness maksimum pada tahap akhir. Hal ini menunjukkan bahwa kualitas individu dalam populasi semakin berdekatan, meskipun belum membuktikan bahwa susunan dari Genetic Algorithm ini mencapai global optimum atau tidak. Garis fitness maksimum tidak menurun karena dua elite selalu dipertahankan, tetapi nilainya tetap dapat meningkat ketika ditemukan anak yang lebih baik. Maka dari itu, nilai fitness akan terus dinamis, karena nilai anak akan berubah setiap adanya populasi tambahan.

Peluang mutasi pada tiga percobaan utama dipertahankan sebesar 0,5, sehingga pengaruh perubahan mutasi tidak dapat disimpulkan dari ketiga run tersebut. Mutasi tetap berperan dalam menghasilkan variasi penempatan yang belum dimiliki induk, sedangkan repair memperbaiki pelanggaran pada anak sebelum dimasukkan ke populasi berikutnya. Berdasarkan hasil yang tersedia, populasi 100 dan 300 generasi digunakan sebagai kompromi antara kualitas hasil dan kebutuhan komputasi. Pengaturan tersebut belum dapat disebut optimal, karena hasil pencarian masih bergantung pada pengacakan dan tambahan generasi masih memberikan perbaikan pada sebagian percobaan.

3.6 Analisis Tambahan

Dari eksperimen *Simulated Annealing* dan bonus ETA, ada dua hal yang menurut kami menarik untuk dicatat.

1. **Kapasitas berat adalah batasan yang paling menentukan.** Hampir semua *final state* SA memiliki berat 495 sampai 500 dari 500, sementara geladak masih memiliki sel kosong. Hal ini juga terlihat dari batas atas, yaitu 1989 kalau hanya memakai batas berat dan 1673 kalau batas luas ikut dipakai. Jadi untuk dataset ini, memilih kendaraan dengan rasio fee terhadap berat yang tinggi lebih penting daripada menyusun kendaraan serapat mungkin.

2. **Bentuk *objective function* mempengaruhi arah pencarian.** *Objective* ETA pada bonus menemukan susunan dengan fee asli yang lebih tinggi dari *objective* fee pada dua dari tiga percobaan. Ini menunjukkan bahwa *landscape* yang berbeda bisa membawa pencarian ke daerah ruang solusi yang berbeda, dan hasil satu *objective* tidak selalu menjadi batas terbaik untuk *objective* lain.
3. **Populasi valid belum tentu menghasilkan solusi terbaik.** Pada seluruh proses percobaan Genetic Algorithm, populasi sudah sepenuhnya valid mulai dari generasi kedua mereka. Tetapi peningkatan score masih berlangsung hingga generasi 298 pada salah satu run. Artinya, algoritma tetap perlu mencari kombinasi kendaraan yang lebih menguntungkan. Berat muatan yang penuh juga tidak cukup menjadi ukuran kualitas.

BAB IV
KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan implementasi dan eksperimen yang sudah dilakukan, kesimpulan yang kami dapatkan adalah sebagai berikut.

1. Persoalan penempatan kendaraan pada geladak kapal berhasil dimodelkan sebagai persoalan *local search*. *State* direpresentasikan dengan *dictionary* posisi dan orientasi, dan perhitungan nilai berbasis penalti dengan P = 1 + fee terbesar menjamin *state* terbaik selalu valid.
2. Pada eksperimen utama satu kapal dengan objective shipping fee, Hill Climbing Sideways menghasilkan rata-rata score tertinggi dari enam algoritma yang dibandingkan, yaitu 1430,33. Hill Climbing Steepest, Sideways, dan Random Restart sama-sama menemukan score terbaik 1503 pada seed 202. Hill Climbing Steepest mempunyai rata-rata durasi tercatat paling singkat, sedangkan langkah *sideways* memberi peningkatan hasil pada seed 303. Random Restart belum memberikan peningkatan score dibandingkan Steepest pada tiga seed yang diuji.
3. *Simulated Annealing* menghasilkan *score* 1418, 1398, dan 1458 (rata-rata 1424,67) dengan durasi sekitar 20 detik per percobaan. Hasilnya cukup konsisten dengan selisih 60 dan selalu valid, tetapi masih di bawah batas bawah *global optimum* yang kami temukan (1528).
4. Penerimaan langkah buruk pada SA paling berperan di awal pencarian. Setelah sekitar iterasi 30.000, hampir semua langkah buruk ditolak, dan hampir setengah iterasi SA berupa langkah di *plateau* dengan $\Delta E = 0$.
5. Pada dataset kami, batasan yang paling menentukan adalah kapasitas berat, bukan luas geladak.
6. *Objective function* alternatif berbasis ETA berhasil memprioritaskan kendaraan yang lebih mendesak, dengan rata-rata ETA turun menjadi sekitar 3,9 hari. Bonus ini juga menemukan susunan dengan fee 1528, sehingga *global optimum* berada di antara 1528 dan 1673.
7. Genetic Algorithm dengan repair menghasilkan score 1335, 1405, dan 1356 dengan rata-rata 1365,33 serta rata-rata durasi 26,57 detik. Seluruh solusi akhir

valid. Pada ketiga percobaan, seluruh populasi sudah valid mulai generasi 2, tetapi pencarian tetap diperlukan untuk meningkatkan total tarif muatan.

8. Pada eksperimen awal Genetic Algorithm, penambahan generasi meningkatkan rata-rata hasil dengan manfaat tambahan yang semakin kecil. Penambahan populasi dari 50 menjadi 100 memperbaiki rata-rata hasil. Sedangkan, peningkatan menjadi 200 tidak memberikan perbaikan. Jadi, memperbesar parameter pencarian tidak selalu menghasilkan peningkatan kualitas yang sebanding.

4.2 Saran

Kedepannya, untuk memperoleh hasil yang lebih akurat, dapat dilakukan eksperimen dengan batas maksimum iterasi yang lebih tinggi sehingga algoritma akan lebih banyak melakukan eksplorasi di *neighbor* dan pada akhirnya mengeluarkan hasil yang efektif. Sama halnya dengan temperatur, *cooling rate,* jumlah *restart,* dan jumlah *sideways move,* dapat diatur kembali agar lebih optimal.

BAB V
PEMBAGIAN TUGAS

Pembagian tugas setiap anggota kelompok dapat dilihat pada tabel berikut.

Tabel 5.1 Pembagian Tugas Anggota Kelompok

NIM	Nama	Tugas
18224068	Tiffany Viriya Chu	1. Membuat struktur dokumen 2. Membuat penjelasan awal pada dokumen 3. Membuat constraints, initial state, dan random state 4. Mengerjakan bonus objective function ETA beserta eksperimennya 5. Mengerjakan simulated annealing 6. Merapikan dokumen akhir
18224072	Francis Galton	1. Menginisiasi pembuatan repository 2. Pembuatan database 3. Mengerjakan steepest ascend hill climbing, stochastic hill climbing, sideways move hill climbing, dan random restart hill-climbing 4. Membuat UI untuk seluruh local search 5. Mengerjakan bonus 3
18224092	Zahrah Nur Azizah	1. Membuat constraints, initial state, dan random state 2. Membuat penjelasan awal pada dokumen 3. Membuat source code general untuk ketiga local search 4. Mengerjakan genetic algorithm 5. Merapikan dokumen akhir

REFERENSI

M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. Video. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8. Accessed: Sep. 16, 2026.

M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. Video. Available: https://www.youtube.com/watch?v=_6rla37QorM. Accessed: Sep. 16, 2026.

M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. Video. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. Accessed: Sep. 16, 2026.

M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. Video. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk. Accessed: Sep. 16, 2026.

M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. Video. Available: https://www.youtube.com/watch?v=aI41TGo9J9c. Accessed: Sep. 16, 2026.

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671-680, 1983.

Tim Asisten Laboratorium Inteligensi Artifisial, "Spesifikasi Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial 2026/2027," Institut Teknologi Bandung, 2026.

J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

LAMPIRAN

Lampiran A. Form Penggunaan Generative AI

- **Lampiran Penggunaan AI - Tugas Besar DAI 1.pdf**

Lampiran B. Lampiran Lainnya

Repository GitHub: https://github.com/gelatonnn/IF3070_G19_MantaplahPokoknya

Proses Pengerjaan		
